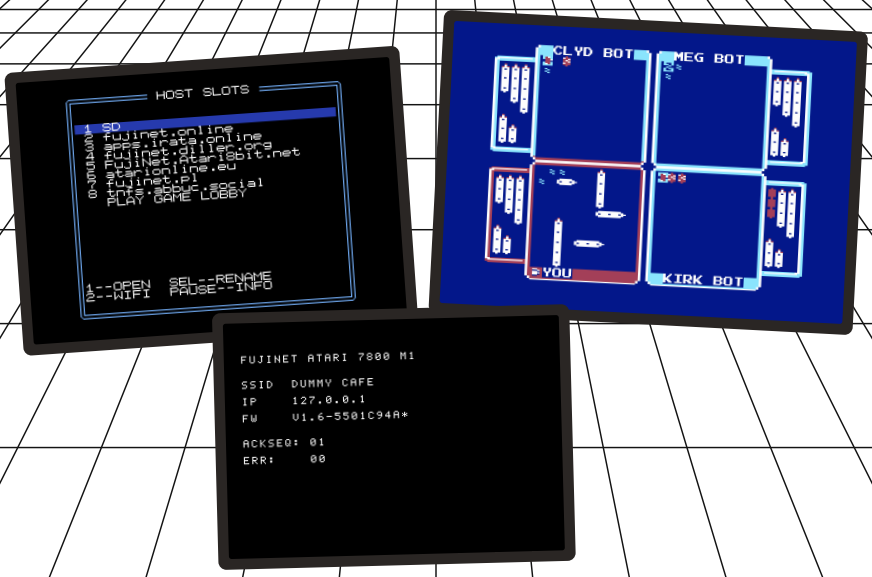
FN7800-PH

FUJINET 7800

ProSystem



Programmer's Handbook

FujiNet 7800 ProSystem Programmer's Handbook. *First edition, 2026.*

Typeset after the 1986 ATARI 7800 ProSystem Owner's Manual *as a tribute. FujiNet is an independent project and is not affiliated with, endorsed by, or sponsored by Atari. "Atari", "7800" and "ProSystem" are trademarks of their owner and are used here to identify the console this handbook is about.*

Every address, register, command and code fragment in this handbook is transcribed from the live project sources and verified in the FujiNet MAME cartridge model against a running FujiNet adapter; the sources and their commits are listed in the Parts List at the back.

Text and diagrams © 2026 the FujiNet Project, CC BY-SA 4.0. Program listings carry the licence of their repositories. 7800basic and cc65 are © their authors and are distributed under their own licences.

Table of Contents

1 Your FujiNet Cartridge 1
2 Install the Tools 3
3 The Memory Map 6
4 How the Mailbox Works 10
5 The Text Display 16
6 Living in 4K 21
7 The 6502 Library 23
8 C and fujinet-lib 27
9 7800basic on the Mailbox 31
10 First Contact 35
11 The Network Device 38
12 Directories, Mount and Boot 43
13 Keys, Clock and Utilities 51
14 Banks and SuperGame 57
15 The Tile Engine 62
16 Command Reference: The Fuji Device 66
17 Command Reference: The Network Device 86
18 Command Reference: Disk, Printer, Clock, Modem 96
19 Command Reference: The Cartridge Itself 101
20 Battleship 106
21 CONFIG 111
22 Things That Cost Real Time 114
23 Troubleshooting Checklist 117
24 Parts List 122
A Mailbox Quick Reference 124
B Error and Status Codes 126
C Program Listings 128
D Battleship Listing 168

1 Your FujiNet Cartridge

The FujiNet cartridge for the ATARI 7800 ProSystem is a network adapter that plugs into the cartridge slot. Inside the shell are two computers and a memory chip. An RP2354B microcontroller and 512K of static RAM sit on the cartridge edge. The console reads the RAM itself: its address lines A0–A12 and its data lines go straight to the chip, and the RP2354B only decides, through a table in its PIO, which 8K page of the RAM each slot of the console's address space shows. Nothing stands in a game's way, so MARIA can read the cartridge as fast as it reads any other.

Behind the RP2354B, over a USB cable, an ESP32-S3 runs the FujiNet firmware. It carries the WiFi radio, the SD card, the network protocols and everything else a FujiNet does on every other computer it has been built for.

The console never sees the ESP32-S3. It sees a signed boot block when the power comes on, a 512-byte loader at `$0600`, a mailbox from `$0800` to `$0FFF`, a POKEY at `$0450`, a High Score Cart when one is installed, and up to 448K of program in the RAM, laid out the way any of ten real 7800 cartridge boards would lay it out. This handbook is about all of them.

What you should have

- A FujiNet cartridge, or the FujiNet cartridge model grafted into MAME, which is what every program in this handbook was verified on.
- A FujiNet adapter to talk to: an ESP32-S3 running the `fujiversal-atari7800` firmware, or `fujinet-pc` on the same machine as MAME, listening on bus-over-IP.
- cc65 2.19: `ca65` and `ld65` for the assembly programs, `cc65` for the C ones.
- fujinet-lib, built for its `atari7800` target, for the C programs.
- 7800basic, for the BASIC programs.
- This handbook.

Note: The cartridge edge carries no reset line. The console's RESET switch is a bit in `SWCHB` that your program reads like a joystick; it does not reach the cartridge. Only a power cycle sends the cartridge back to its boot block.

Important: The FujiNet 7800 board is a Rev0 design that has not yet been built. Everything in this handbook ran in host tests and in the MAME cartridge model. The first thing a real board must prove is that the console lets the cartridge answer at `$0600`–`$0FFF`, where the mailbox lives.

How this handbook is organised

Sections 3 through 9 are the machine: the memory map, the mailbox, the display, the memory you have to work in, and the three languages. Sections 10 through 15 are programs, each one written three times: in 6502 assembly, in C, and in 7800basic. Sections 16 through 19 are the command reference: every command the adapter answers, and everything the cartridge itself does. Section 20 walks through Battleship, a complete networked game, and Section 21 through CONFIG. The rest is what went wrong on the way, and how to tell when it is going wrong for you.

2 Install the Tools

Every program in this handbook is built three ways, and every build runs on the same bench: the FujiNet cartridge grafted into MAME's Atari 7800, talking over bus-over-IP to a FujiNet adapter running on the same computer. Install the pieces in this order.

cc65

The assembler, linker and C compiler are all cc65's: `ca65`, `ld65` and `cc65`, with `cl65` to drive them. Everything here was built with version 2.19. cc65's own `atari7800` target supplies the C library; the FujiNet library supplies the start-up code and the linker configuration, because cc65's start-up locks INPTCTRL (Section 3 explains why that matters).

```
git clone https://github.com/cc65/cc65.git
cd cc65 && make && sudo make install PREFIX=/usr/local
cc65 --version           # cc65 V2.19
```

fujinet-lib

The C programs call fujinet-lib, the same library every FujiNet C program on every platform uses. Its `atari7800` target adds the mailbox transport, a start-up file that never touches INPTCTRL, a linker configuration and a checker. Until it is merged, build it from the branch that adds it:

```
git clone -b add-atari7800 https://github.com/tschak909/fujinet-lib-experimental.git
cd fujinet-lib-experimental && make atari7800
ls r2r/atari7800        # fujinet.atari7800.lib
```

The library is the `.lib`; its headers are `include/` and `bus/atari7800/`. The build scripts in this handbook find them through `FNLIB`.

7800basic

7800basic is a compiled BASIC for the 7800: it turns a program into 6502 assembly and assembles it with dasm. Its release runs on wasmtime; it also builds natively, which is what this handbook used.

```
git clone https://github.com/7800-devtools/7800basic.git
cd 7800basic
make LEX=flex 7800preprocess    # the two lexers share lex.yy.c:
make LEX=flex 7800optimize      #   build them one at a time
make LEX=flex                   # then the rest
export bas7800dir=$PWD PATH=$PWD:$PATH
```

dasm 2.20.16 must be on the `PATH` too. The handbook's `listings/bas/build.sh` runs 7800basic's steps itself, because the native `7800basic.native.sh` runs the compiler under valgrind, and then adds the FujiNet claim (Section 9).

MAME with the FujiNet cartridge

The cartridge model lives in the firmware's `pico/atari-7800/emu/`. Its `apply.sh` grafts it into a MAME source tree; a build of the 7800 driver alone is enough.

```
git clone https://github.com/mamedev/mame.git
fujinet-firmware/pico/atari-7800/emu/apply.sh mame
cd mame
make SUBTARGET=a7800 SOURCES=src/mame/atari/a7800.cpp REGENIE=1 -j$(nproc)
./a7800 -listslots | grep fujinet
```

The last line should print `cartslot  fujinet  Atari 7800 FujiNet Cartridge`. The 7800 BIOS images go where MAME's `-rompath` finds them. The cartridge connects to a FujiNet adapter at `127.0.0.1:9995` unless the environment variable `FUJINET_TCP` names another address.

Important: MAME must run from its own tree, or it ignores `-autoboot_script` and every headless check in this handbook does nothing. With no display, set `SDL_VIDEODRIVER=dummy`.

A FujiNet adapter

The adapter is `fujinet-pc`, the FujiNet firmware built to run on a computer, in its RS-232 flavour, which speaks bus-over-IP. Build it from the firmware:

```
cd fujinet-firmware
./build.sh -p RS232
cd build/dist && ./run-fujinet
```

In its `fnconfig.ini`, the `[BOIP]` section names the port it listens on; make it the port the cartridge connects to. Its `SD/` directory is the SD card, host slot 0. Only one MAME can talk to one `fujinet-pc` at a time.

Running a program

Each language has a build script: `listings/asm/build.sh`, `listings/c/build.sh` and `listings/bas/build.sh`. Each writes the program twice: a bare image (`.bin`), which is what the cartridge runs, and the same image with the 128-byte header MAME needs (`.a78`), which the cartridge strips.

```
listings/c/build.sh acfg                    # build/acfg.bin, build/acfg.a78
emu/run.sh listings/c/build/acfg.a78        # in a window
SHOT_AT=6 TEXT_WANT="DUMMY CAFE" \
  emu/run.sh listings/c/build/acfg.a78 screen   # headless, and checked
emu/verify.sh                               # all 24 programs
```

`emu/screen.lua` reads the screen the way MARIA does, by walking the display lists, so one check serves all three languages. It prints the cartridge's status and the text on the screen, saves a snapshot, and with `TEXT_WANT` passes only if every piece appears.

FujiNet Go Atari 7800

FujiNet Go Atari 7800 is a desktop application with all of this inside it: MAME's 7800, the cartridge model, and a FujiNet adapter, with no BIOS required. Its **Open Cartridge…** menu runs any `.a78` file, and a program that carries the FujiNet claim keeps the mailbox, so every program in this handbook runs there unchanged. It is the quickest way to try one.

3 The Memory Map

The 7800 gives a cartridge all of `$4000`–`$FFFF`, and the FujiNet cartridge gives nearly all of that to your program. Its own business happens lower down, in space the console's design leaves to the cartridge and no other cartridge ever used: a 512-byte loader at `$0600`, and the mailbox from `$0800` to `$0FFF`.

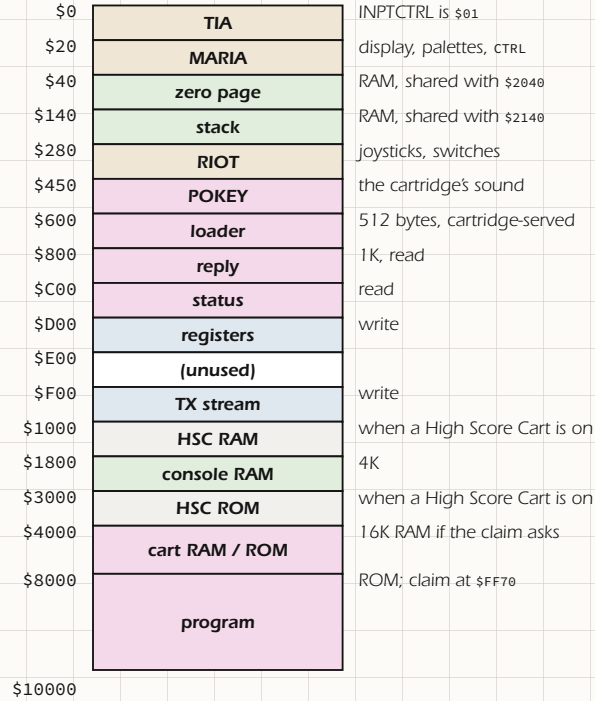


Figure 1. The 7800 as a FujiNet program sees it. Magenta tints are the cartridge's; blue tints are pages it only listens to.

The console's own pieces are the usual ones. TIA and MARIA share the bottom page. The 4K of RAM at `$1800`–`$27FF` is also seen at `$0040`–`$00FF` (the zero page) and `$0140`–`$01FF` (the stack). The RIOT's joystick and switch ports sit at `$0280`.

Four modes

The cartridge serves different things at different times. The status page at `$0C12` says which mode it is in.

Range	BOOT (0)	LOAD (1)	GAME (2)	APP (3)
`$0450`	—	—	POKEY, if the game has one	POKEY
`$0600`	loader	loader	—	loader
`$0800–$0FFF`	mailbox	mailbox	—	mailbox
`$4000–$7FFF`	—	the load window	as the board maps it	as the board maps it; RAM if claimed
`$8000–$EFFF`	—	—	the image	the image
`$F000–$FFFF`	signed boot block	—	the image	the image

- **BOOT** is power-on. The cartridge serves a 4K block the console's BIOS can verify; the block jumps to the loader.
- **LOAD** is the loader filling the cartridge's RAM, one 1K slice at a time, through the window at `$4000`.
- **GAME** is an ordinary 7800 cartridge: an image in the RAM, laid out as its own board would lay it out, and the mailbox gone. Nothing the game does can disturb the FujiNet.
- **APP** is a FujiNet program. The image runs exactly as a game would, but the loader and the mailbox stay where they are.

What separates GAME from APP is ten bytes in the program.

The claim

A program that wants the mailbox says so at `$FF70`, just below the area the BIOS's signature check uses:

F	U	J	I	ver 1	kind+1	flags	0	0	0

- **Version** is 1.
- **Kind** names the board layout the image wants, plus one; 0 lets the cartridge decide. A program bigger than 48K says which bank scheme it uses here (Section 14).
- **Flags** bit 0 asks for 16K of cartridge RAM at `$4000–$7FFF`. fujinet-lib's C programs keep their variables and stack there; the bring-up's assembly programs keep their screen there.

The cartridge looks for the claim when it loads an image. If it is there, the image runs in APP mode, every FujiNet program may play sounds through the cartridge's POKEY at `$0450`, and the mailbox stays.

Note: The claim is all it takes. The same cartridge runs a commercial game and a FujiNet program the same way; the only difference is whether the program asked to keep the mailbox.

The image

An image is a whole number of kilobytes, laid so that it ends at `$FFFF`: a 32K program occupies `$8000`–`$FFFF`. Its reset vector must point at `$4000` or above. MAME wants the 128-byte `.a78` header in front; the cartridge takes either and strips the header.

How the cartridge maps a bigger image depends on what it can learn about it, in this order: a database of every known 7800 game by CRC; a `mapper=` line in a `.cfg` file beside the image; the `.a78` header's cartridge type; and the image's size. The claim's kind byte, when it is not 0, overrides all but the first two. Section 12 lists the ten board layouts.

No reset line

The 7800's cartridge edge carries the address and data buses, R/W, the clock, HALT, an interrupt line and audio. It has no reset. The console's RESET switch is a bit in `SWCHB` that your program reads. The cartridge notices only a power cycle (the clock stopping for 50 milliseconds), and then it goes back to BOOT and serves the boot block.

A program therefore cannot assume anything about the cartridge's state when it starts, except that it is in APP mode and its mailbox works. In particular, the sequence number in the status page survives a power cycle of the console, because the cartridge was not reset; Section 4 explains why that matters.

What every program must not do

A. **Never store to `$00`–`$1F` or its mirrors** until you mean to lock INPTCTRL. Until the first write that locks it, every store to the TIA's range rewrites INPTCTRL, which can switch off MARIA or switch the BIOS back on. A locked INPTCTRL also stops the cartridge from handing a game to the console's BIOS later (Section 12). So a FujiNet program plays its sound through the cartridge's POKEY, uses MARIA's `WSYNC` at `$24`, and leaves INPTCTRL alone. Reading the TIA's input ports is fine.

B. **Never use a read-modify-write instruction on `$0D00`–`$0FFF`.** Section 4.

C. **Never `JMP ($xxFF)`.** The 6502 fetches the high byte from the wrong page.
D. **Set your own stack.** The loader leaves S at `$16`; the first `JSR` before `LDX #$FF` / `TXS` lands in the TIA.

The build scripts check the first three rules on every program they build. `tools/checkrom.py` (from the cartridge's own tree) and fujinet-lib's `atari7800-romstamp.py` disassemble the program's code segments from the linker's map and refuse an absolute store into the TIA's range, a read-modify-write aimed at the mailbox, an indirect jump through `$xxFF`, a missing claim, or a reset vector below `$4000`. An indexed store into the TIA's range cannot be seen that way; the cartridge's status page reports INPTCTRL's state at run time (`$0C1C` and `$0C1D`).

4 How the Mailbox Works

The 7800's cartridge edge has a read/write line, so talking to the FujiNet is ordinary memory traffic. The console **stores** to pages the cartridge watches, and **reads** pages the cartridge paints. Four pages do all the work.

Address	Direction	What it is
`$0800–$0BFF`	read	The reply window: up to 1024 bytes of the last reply.
`$0C00–$0CFB`	read	The status page: sequence number, error, reply length, boot and load progress, the cartridge's mode.
`$0D00–$0D7F`	write	The registers. A store of V to `$0D00`+n sets register n to V.
`$0F00–$0FFF`	write	The TX stream. Every store anywhere in the page appends its byte.

Reading a write page returns nothing useful, and `$0E00–$0EFF` is not used at all. Everything a program sends is a store; everything it learns is a load.

A register is one store

There is no register-select dance. To set the device, the command and the parameter count, you store three bytes, then a fourth to rewind the TX stream:

6502 ASSEMBLY (ca65)

```
        lda     #$70            ; the FUJI device
        sta     FN_REGSEL+FN_REG_DEVICE
        lda     #$C4            ; GET_ADAPTERCONFIG_EXTENDED
        sta     FN_REGSEL+FN_REG_CMD
        lda     #0              ; no parameters
        sta     FN_REGSEL+FN_REG_NPARAM
        sta     FN_REGSEL+FN_REG_DATA_RST
```

C (cc65 + fujinet-lib)

```
fn_regwr(FNR_DEVICE, 0x70);      /* the FUJI device */
fn_regwr(FNR_CMD, 0xC4);         /* GET_ADAPTERCONFIG_EXTENDED */
fn_regwr(FNR_NPARAM, 0);         /* no parameters */
fn_regwr(FNR_DATA_RST, 0);
```

7800basic

```
FN_DEVICE = $70
FN_CMD = $C4
FN_NPARAM = 0
FN_DATA_RST = 0
```

In C, `fn_regwr()` is fujinet-lib's one-store register write; writing `FN_REGSEL[FNR_CMD] = 0xC4` directly compiles to the same `STA $0D01`. In 7800basic, the name of a register is its address (Section 9).

Register	Store to	Meaning
`$00` DEVICE	`$0D00`	The FujiBus device: `$70` the FujiNet itself, `$71`–`$78` the network devices, `$45` the clock.
`$01` CMD	`$0D01`	The command.
`$02` NPARAM	`$0D02`	How many parameters the TX stream holds.
`$05` DATA_RST	`$0D05`	Any value: rewind the TX stream to its start.
`$06` RXSLICE	`$0D06`	Which 1K slice the reply window shows. There is one slice; leave it 0.
`$10` SEQ	`$0D10`	Launch the transaction (below).
`$11` BOOTLOCK	`$0D11`	`$B5`: arm a pushed image for the loader (Section 12).
`$12, $13`	`$0D12`–`$0D13`	`$B5` then `$4A`: reboot the cartridge's RP2354B for a firmware update.
`$14` SLICE_ACK	`$0D14`	The loader's: a slice has been copied.
`$15` TV	`$0D15`	Tell the cartridge NTSC (0) or PAL (non-zero).
`$16` HSC	`$0D16`	The High Score Cart: 0 off, 1 on, 2 forget, 3 install (Section 13).

Stores to `$0D80`–`$0DFF` are not registers. Four of them are one-shot operations the loader uses, listed in Section 19; the rest do nothing.

Warning: Never use a read-modify-write instruction on `$0D00`–`$0FFF`. `INC`, `DEC`, `ASL`, `LSR`, `ROL` and `ROR` write the old value back before the new one, and the cartridge counts both writes. In C, never `++` or `|=` a mailbox address; in 7800basic, never `FN_TX = FN_TX + 1`. An indexed store such as `STA $0F00,X` is safe as long as it does not cross a page.

The TX stream

Parameters and payload go into `$0F00`, one store per byte. Each parameter is its size (1, 2 or 4) and then its value, low byte first. The payload, if the command takes one, follows the last parameter.

size	value…	size	value…	…	payload…

The stream holds 320 bytes and at most eight parameters. A parameter of any other size fails the transaction with error 3 before anything is sent; bytes past 320 are dropped. `DATA_RST` rewinds the stream, so write it before the first byte

of every transaction. fujinet-lib writes it first of all; the assembly library writes it last of the four registers; the cartridge does not mind which.

Launching, and the sequence number

A transaction is launched by one more store: the next sequence number into register `$10`. The cartridge sends the request to the FujiNet, waits for the answer, paints the reply into `$0800`, the error and the reply length into the status page, and then, ***last***, copies your sequence number into `ACKSEQ` at `$0C00`. The moment `ACKSEQ` equals your number, everything else is in place.

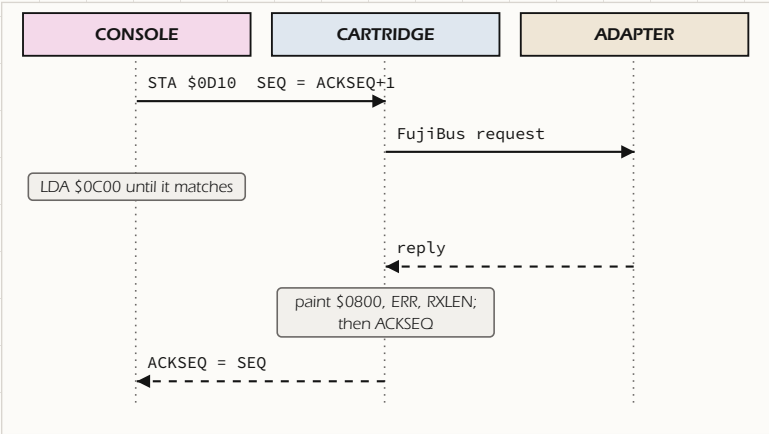


Figure 2. A transaction. The console only ever stores and loads.

The next number is always the cartridge's own `ACKSEQ` plus one, wrapping from 255 to 1. Zero is reserved, and a number equal to the last one does nothing at all, so a stray repeat cannot send a request twice.

Important: Never keep the sequence number in your own RAM. The console can be switched off and on without the cartridge being reset (there is no reset line), and a program that starts counting from 1 again will send a number the cartridge has just seen. It will be ignored, and the old reply will still be in the window. Read `ACKSEQ`, add one.

The status page

Address	Name	Meaning
$0C00	ACKSEQ	The last sequence number answered. Painted last.
$0C01	STATUS	Bit 0: the link to the FujiNet is up.
$0C02	ERR	How the transaction went: 0 completed, 1 no link, 2 timed out, 3 bad frame, 4 too big.
$0C03	REPLY_CMD	$06 the FujiNet said yes (ACK), $15 no (NAK), 0 if nothing came back.
$0C04–$0C05	RXLEN	The reply's length, low byte first; at most 1024.
$0C06–$0C08	BOOT_STATE, BOOT_PCT, BOOT_ERR	A pushed image's progress (Section 12).
$0C09–$0C0A	MAGIC	'F', 'N': a FujiNet cartridge is answering.
$0C0B	PROTO_VER	1.
$0C0D–$0C11	LOAD_…	The loader's handshake (Section 19).
$0C12	MODE	0 BOOT, 1 LOAD, 2 GAME, 3 APP.
$0C14	MAPPER	The board layout of the running image.
$0C16–$0C1B	BOOT_GOT, BOOT_TOT	Bytes of a push received, and expected; 24 bits each.
$0C1C–$0C1D	INPTCTRL, INPT_LOCK	The cartridge's model of INPTCTRL, and 1 if it has been locked.
$0C1E	TV	0 NTSC, 1 PAL.
$0C1F–$0C24	STAGED…	The pushed image: flags, kind and CRC (Section 19).
$0C25	HANDOVER	How the last image was started: 1 by the BIOS, 2 by the loader.
$0C26	HSC	The High Score Cart's state (Section 13).

The magic bytes are the presence test: a program that finds anything but 'F', 'N' at $0C09 is not running on a FujiNet cartridge, or not in APP mode, and must not wait for a reply.

The reply window

The reply is at $0800, RXLEN bytes of it, read in place: it stays there until the next transaction. A reply never needs copying unless MARIA has to display it, since

MARIA cannot read the mailbox (Section 5), or unless the next transaction would overwrite it before you are done.

How long to wait

The cartridge gives each transaction a budget, and reports error 2 if the FujiNet has not answered by then:

Transaction	Cartridge's budget
Anything not below	5 seconds
MOUNT_IMAGE (`$F8`), COPY_FILE (`$D8`)	60 seconds
Network devices `$71`–`$78`: OPEN, CLOSE, STATUS, READ	90 seconds
If the link is down	3 more seconds, first

A program's own wait must be longer, so that the cartridge's timeout is the one reported. fujinet-lib does this for you; the assembly library waits about 11 seconds unless you raise `FN_TMO`; the 7800basic library waits about 13 unless you raise `fn_tmo`. Each section says when to raise it.

Note: In MAME the whole transaction happens inside the store to `$0D10`: the very next instruction already sees `ACKSEQ` change. On a real cartridge it takes milliseconds, or seconds for a network request. Write the wait loop as if it mattered, and it will.

The recipe

Everything a program does with the FujiNet is this sequence. Here it is in full, with nothing but the stores and loads, asking for the adapter's configuration. It waits forever; each language's library (Sections 7, 8 and 9) wraps the same steps with a timeout, and the rest of the handbook uses the libraries.

6502 ASSEMBLY (ca65)

```
        lda     #$70            ; device, command, no parameters
        sta     FN_REGSEL+FN_REG_DEVICE
        lda     #$C4
        sta     FN_REGSEL+FN_REG_CMD
        lda     #0
        sta     FN_REGSEL+FN_REG_NPARAM
        sta     FN_REGSEL+FN_REG_DATA_RST
        lda     FN_ACKSEQ       ; the next sequence number
        clc
        adc     #1
        bne     @seq
        lda     #1              ; 0 is reserved
@seq:   sta     FN_REGSEL+FN_REG_SEQ   ; launch
@wait:  cmp     FN_ACKSEQ       ; painted last
        bne     @wait
        lda     FN_ERR          ; did it complete?
        bne     failed
        lda     FN_REPLYCMD     ; did the FujiNet say yes?
        cmp     #$06
        bne     refused         ; the reply is at FN_REPLY
```

C (cc65 + fujinet-lib)

```
uint8_t seq;

fn_regwr(FNR_DEVICE, 0x70);      /* device, command, no parameters */
fn_regwr(FNR_CMD, 0xC4);
fn_regwr(FNR_NPARAM, 0);
fn_regwr(FNR_DATA_RST, 0);
seq = FN_ACKSEQ + 1;             /* the next sequence number */
if (seq == 0)
    seq = 1;                     /* 0 is reserved */
fn_regwr(FNR_SEQ, seq);          /* launch */
while (FN_ACKSEQ != seq)         /* painted last */
    ;
if (FN_ERRCODE != FN_OK)         /* did it complete? */
    failed();
if (FN_REPLYCMD != 0x06)         /* did the FujiNet say yes? */
    refused();                   /* the reply is at FN_REPLY */
```

7800basic

```
 FN_DEVICE = $70
 FN_CMD = $C4
 FN_NPARAM = 0
 FN_DATA_RST = 0
 fn_seq = FN_ACKSEQ + 1
 if fn_seq = 0 then fn_seq = 1
 FN_SEQ = fn_seq
wait_ack
 if FN_ACKSEQ <> fn_seq then goto wait_ack
 if FN_ERR <> 0 then goto failed
 if FN_REPLYCMD <> $06 then goto refused
 rem the reply is at FN_REPLY
```

5 The Text Display

MARIA draws nothing by itself. Every line of the picture, it reads a list of objects from memory, and every object tells it where to find its pixels. A FujiNet program that wants to show text builds those lists once, puts character codes where an object points, and from then on changing the text is just storing bytes.



Figure 3. How MARIA finds a character. Every box is memory MARIA reads by DMA.

The **display list list** (DLL) divides the screen into zones of a few lines each and points each zone at its **display list**. A display list holds objects; a text row is one or two **indirect** objects, which point not at pixels but at a row of character codes. MARIA looks each code up in the font whose page `CHARBASE` names. All three roads in this handbook draw text this way.

Important: Never point MARIA at the mailbox. The cartridge answers at `$0600`–`$0FFF` from its microcontroller, not from RAM, and cannot keep up with MARIA's DMA. A string in the reply window must be copied to RAM — the cartridge's or the console's — before MARIA can show it. Every program in this handbook does.

Three roads

	Assembly	C	7800basic
Library	`fujidisp.s` (the bring-up's)	`maria.s` (CONFIG's tile engine)	7800basic itself
Mode	320A, one byte a character	320B indirect, two bytes a character	320A, one byte a character
Cells	40 × 24	32 × 24	as you plot them
Text lives	`$4000`, cartridge RAM	`$4000`, cartridge RAM	`$2200`–`$27FF`, console RAM
Font	`mkfont.py`: 5 × 7, capitals	`font.txt`: 7 × 7, upper and lower case	`mkbasfont.py`: the assembly font
Colours	one: palette 0	three per cell, two palettes	palette per plot

The assembly and 7800basic programs use the same glyphs, made from one table, so their screens look alike; lower case comes out as capitals. The C programs use CONFIG's own font, which has both.

In assembly: fujidisp

`fujidisp.s` is the bring-up's text screen: 40 columns by 24 rows in 320A. It keeps its screen buffer, its display lists and its DLL in the cartridge's RAM, which is why every bring-up program's claim sets the RAM flag:

Address	Size	What
`$4000`	960 bytes	The screen: 24 rows of 40 character codes. Code = ASCII; + `$80` is the same character in reverse.
`$4400`	24 × 12	The display lists: two 20-character indirect objects per row.
`$4600`	—	The DLL: 25 blank lines, the 24 rows of 8 lines, then blank lines to the bottom.
`font`	2K	The font, page-aligned in ROM; `CHARBASE` points at it.

Routine	What it does
`disp_init`	Clears the screen, builds the lists, sets the palette. Display still off.
`disp_on` / `disp_off`	MARIA's DMA on (at a frame edge) or off.
`disp_vbl`	Waits for the start of the next vertical blank.
`disp_at`	Cursor to row A, column X.
`disp_putc`	One character at the cursor.
`disp_str`	The NUL-terminated string at `FN_PTR`, at most 40 characters.
`disp_rpl`	Up to Y bytes of the reply window from offset X, stopping at a NUL. This is the copy the Important note asks for: from the mailbox into the screen buffer.
`disp_hex`, `disp_dec`	A as two hex digits, or up to three decimal ones.

The start-up code `a78init.s` calls `disp_init` before `main`, so a program starts with a blank screen and the display off; it turns the display on when it has something to show.

In C: the tile engine

The C programs use the MARIA engine CONFIG and all five FujiNet games share, `maria.s` and `maria.h`. It is a 32 × 24 grid of 8 × 8 tiles in 320B, three colours and the background per tile, two palettes, with the map and the display lists in the cartridge's RAM and the tiles in ROM. Text is just tiles whose

numbers are ASCII codes. Section 15 takes it apart; for text, five calls are enough:

Call	What it does
`mt_init(blank)`	Measures the TV standard, clears to tile `blank`, builds the lists, turns the display on.
`mt_palette(p, c1, c2, c3)`, `mt_background(c)`	Colours. In CONFIG's font, colour 1 is the ink and colour 2 the paper.
`mt_at(col, row)`	The write cursor.
`mt_puts(s)`, `mt_put(t)`	Tiles from a string, or one tile.
`mt_text(s, max)`	Like `mt_puts`, but at most `max` characters, and anything outside 32–126 shows as a space: the call for text that came from the network.
`mt_sync()`	Once a frame: waits for the vertical blank and rebuilds any row whose palette changed.

`mt_text` copies into the map, so it is also the copy out of the mailbox; fujinet-lib's calls copy their replies into your buffers anyway.

In 7800basic

7800basic has its own display system: zones of `zoneheight` lines, objects plotted into them each frame or once, and characters from a PNG. Three statements set it up for text, and the rest of the handbook's BASIC programs begin with them:

```
displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F
```

`fujifont.png` is the assembly font, drawn by `tools/mkbasfont.py` as 128 cells side by side; `alphachars ASCII` makes a character's code its cell. Then:

- `plotchars 'TEXT' palette x y` puts a quoted string on the screen. `x` counts in units of two pixels, so a column is 4; `y` is the zone, which with 8-line zones is the text row.

- `plotchars array palette x y count` plots `count` bytes of a RAM array. MARIA reads the array every frame, so storing into it later changes the screen; there is no need to plot again.
- `plotmap array palette x y width height` plots a whole block of an array as rows, `width` characters each: a text screen in one statement.

A 7800basic program copies a reply out of the mailbox with a loop into an array `dim`'d in the console's free RAM, `$2200`–`$27FF`, and plots the array. Section 9 has the rules for those loops.

Hello, three times

The smallest program in each language puts some text up and checks that a FujiNet cartridge is there. None of them talks to the FujiNet yet.

6502 ASSEMBLY (ca65) · hello.s

```
.proc main
        PUTS    2, 2, title
        PUTS    4, 2, hello
        PUTS    6, 2, m0
        PUTS    8, 2, tinpt
        PUTS    20, 2, frames_lbl
```

C (cc65 + fujinet-lib) · hello.c

```
void main(void)
{
    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);    /* ink white, paper = background */
    mt_at(2, 2);
    mt_puts("FUJINET ATARI 7800");
    mt_at(2, 4);
    mt_puts("HELLO, WORLD");
    mt_at(2, 6);
    mt_puts(fuji_a7800_present() ? "A FUJINET CARTRIDGE ANSWERS"
                                 : "NO FUJINET CARTRIDGE");
    mt_at(2, 7);
    mt_puts(fuji_a7800_mode() == FUJI_A7800_MODE_APP ? "RUNNING AS AN APP" : "");
```

7800basic · hello.bas

```
 clearscreen
 plotchars 'FUJINET ATARI 7800' 0 8 2
 plotchars 'HELLO, WORLD' 0 8 4
 gosub fn_chk
 if fn_res <> FN_OK then goto nocart
 plotchars 'A FUJINET CARTRIDGE ANSWERS' 0 8 6
 if FN_MODE = 3 then plotchars 'RUNNING AS AN APP' 0 8 7
```

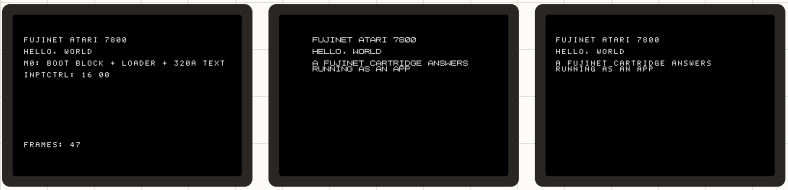


Figure 4. Hello in assembly, C and 7800basic. The assembly one also shows the cartridge's model of INPTCTRL, unlocked, and counts frames.

PAL

A PAL 7800 draws 50 frames a second and 50 more lines than an NTSC one. The tile engine measures the console when it starts, keeps the answer in `mt_pal`, and puts 50 blank lines above the picture instead of 25, so it sits in the middle of either screen. fujidisp's list is the same on both, so on PAL its text sits higher. The cartridge knows too (it watched which BIOS ran) and says so at `$0C1E`; 7800basic measures at start-up and keeps the answer in `paldetected`.

6 Living in 4K

The 7800 has 4K of RAM, at `$1800–$27FF`. Two pieces of it are also seen low in memory: `$2040–$20FF` is the zero page at `$0040–$00FF`, and `$2140–$21FF` is the stack page at `$0140–$01FF`. The bottom of both pages belongs to TIA and MARIA. So the zero page, the stack and everything else share the same 4K, and MARIA's display lists usually want a good part of it.

The FujiNet cartridge helps in two ways. A program that sets the claim's RAM flag gets 16K more at `$4000–$7FFF`, in the cartridge, which MARIA can read like any other memory. And the reply window is 1K of memory the program never has to give up RAM for: a reply is read where it lies.

Who uses what

	Assembly	C	7800basic
Zero page	`$40–$DF` the program's; `$E0–$EB` the library's	`$40–$FF`: cc65's registers and the program's `ZEROPAGE`	`$40–$4A` its temporaries; `a–z` at `$E6–$FF`
Stack page	the 6502 stack	the 6502 stack	`var0–var99` at `$140–$1A3`, its own variables above, then the stack from `$1C2`
`$1800–$27FF`	free	`CONSOLE` segment (optional)	`$1800` the DLL, `$1880–$1FFF` display lists, its variables; `$2200–$27FF` free
`$4000–$7FFF`	screen, lists, DLL; variables (`BSS`)	data, `BSS`, heap; the tile map and lists; the C stack (2K) at the top	not used unless `set romsize 32kRAM`

The handbook's 7800basic library keeps its ten variables in `var90–var99`, so a program has `var0–var89`, `a–z` and the free RAM from `$2200`. A string that MARIA must show, and anything else bigger than a byte, goes in that free RAM with `dim name = $2200`.

Note: The loader runs from the cartridge and borrows `$80–$89` while it copies an image in. A program finds them dirty, along with S at `$16` and MARIA's DMA off. Clear what you use, and set the stack before anything else.

The reply is free memory

The reply window at `$0800` is the biggest buffer a FujiNet program has, and it costs no RAM. Read a reply where it lies: walk it with an index, compare against it, take bytes from it into the screen. Copy only what must outlive the next transaction, or what MARIA must draw.

Language	Reading the reply in place
Assembly	`LDA FN_REPLY,X` — or `(ptr),Y` for more than 256 bytes.
C	`FN_REPLY[i]` from the bus header; fujinet-lib's calls copy into your buffer for you.
7800basic	`FN_REPLY[ix]`; for a field past the first 256 bytes, or for clarity, `dim` a name at its address: `dim FN_R_IP = $088C`.

What survives what

	RESET switch	Power off and on
Your program	keeps running: RESET is a bit in `SWCHB` you read	gone: the cartridge boots CONFIG
Console RAM	kept	lost
Cartridge RAM (`$4000`)	kept	cleared by the loader when CONFIG loads
The cartridge's `ACKSEQ`	kept	kept: the cartridge was never reset
The FujiNet adapter	untouched	untouched

The last two rows are why a program must take its sequence number from `ACKSEQ` and never count for itself (Section 4). A FujiNet program also cannot leave anything in the adapter that it expects to be gone: a network connection opened before the power went off is still open, and a program that may have been interrupted should close what it opens before it opens it again.

7 The 6502 Library

The assembly programs in this handbook are built on the bring-up's own client code, copied unchanged from the cartridge's `testrom/` directory into `listings/common/`. Four files do the work, and two more of the handbook's own name the commands the bring-up never needed.

File	What it is
`fujinet.inc`	The mailbox's addresses and registers, the 7800's hardware registers, the library's zero page and error codes. Mirrored by hand from the cartridge's `fuji_mailbox.h`.
`fujilib.s`	The transport: begin, append, launch and wait, check.
`fujidisp.s`	The 40 × 24 text screen (Section 5).
`a78init.s`	Reset: the stack, cleared memory, the claim, the vectors.
`devdefs.inc`	The handbook's: the other devices, and every FUJI command by name.
`netdefs.inc`	The handbook's: the network device's commands, open modes and status layout.

The transport

Routine	In / out	What it does
`fn_chk`	Z set if present	Looks for `'F'`, `'N'` at `$0C09`.
`fn_beg`	`FN_DEV`, `FN_CMD`, `FN_NPR`	Writes the device, command and parameter count, then rewinds the TX stream. Sets `FN_TMO` to 16, about 11 seconds.
`fn_pb`	A	Appends a one-byte parameter.
`fn_pw`	A low, X high	Appends a two-byte parameter.
`fn_txb`	A	Appends one byte: payload.
`fn_path`	`FN_PTR`	Appends the NUL-terminated string at `FN_PTR`, padded with NULs to exactly 256 bytes, which is what OPEN_DIRECTORY and SET_DEVICE_FULLPATH read.
`fn_go`	A = result	Launches the transaction with `ACKSEQ`+1 and waits up to `FN_TMO` × 0.7 seconds. Returns the cartridge's error (0 is success), or `$FF` if it never answered.
`fn_ack`	A = result	After `fn_go` returned 0: 0 if the FujiNet said yes, `$EE` if it said no.

Routine	In / out	What it does
`fn_blk`	—	Arms a pushed image for the loader (BOOTLOCK).
`fn_boot`	—	Interrupts and DMA off, then into the loader. Does not return.
`fn_rw`	X register, A value	One register write. The others are built on it.

Every transaction in the handbook's programs has the same shape:

6502 ASSEMBLY (ca65) · clock.s

```
; ASK cmd, row -- a clock command with no parameters, its text on the row.
.macro  ASK     cmd, row
        lda     #CLKDEV
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
```

and every failure goes to one place, through a `go_ack` that each program carries:

6502 ASSEMBLY (ca65) · netget.s

```
; go_ack -- fn_go, then fn_ack; on any failure the code goes in aperr and
; the failure screen follows, so callers need no branch.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla                     ; drop our caller's return
        pla
        jmp     fail
.endproc
```

`go_ack` drops its caller's return address on failure and jumps to the program's `fail`, so a caller can treat each transaction as one line that either works or does not come back.

The wait

`fn_go` is the recipe of Section 4 with a timeout: three nested loops of about 0.7 seconds each pass, counted in `FN_CNT`.

6502 ASSEMBLY (ca65) · fujilib.s

```
.proc fn_go
        lda     FN_ACKSEQ
        clc
        adc     #1
        bne     have
        lda     #1              ; 0 is reserved as "never used"
have:   sta     FN_SEQ
        ldx     #FN_REG_SEQ
        jsr     fn_rw           ; launching it is this single store
        lda     FN_TMO
        sta     FN_CNT
outer:  ldy     #0
mid:    ldx     #0
inner:  lda     FN_ACKSEQ       ; published LAST, after the whole reply
        cmp     FN_SEQ
        beq     got
        dex
        bne     inner
        dey
        bne     mid
        dec     FN_CNT
        bne     outer
        lda     #FNEWAIT
        rts
got:    lda     FN_ERR
        rts
.endproc
```

The loops are counted, not timed, and MARIA's DMA steals cycles from the 6502 while the display is on, so the real wait is longer than the count says. That errs the right way. Raise `FN_TMO` after `fn_beg` for a command the cartridge allows more time: 96 (about a minute) for MOUNT_IMAGE, 130 or more for the network device's OPEN, STATUS, READ and CLOSE.

The zero page

Address	Name	What
`$E0`	`FN_SEQ`	The sequence number being waited for.
`$E1`	`FN_CNT`	Timeout passes left.
`$E2`–`$E4`	`FN_DEV, FN_CMD, FN_NPR`	The transaction `fn_beg` begins.
`$E5`	`FN_TMO`	The timeout, in about 0.7-second passes.
`$E6`–`$E7`	`FN_PTR`	A string pointer, for `fn_path` and `disp_str`.
`$E8`–`$EB`	`DISP_PTR, DISP_TMP`	The text cursor and scratch.

The linker gives the program's own `ZEROPAGE` segment `$40`–`$DF`.

Start-up

`a78init.s` is where the console arrives from the loader. It never stores below `$40`:

6502 ASSEMBLY (ca65) · a78init.s

```
reset:  sei
        cld
        lda     #$7F            ; DMA off until the display is built
        sta     CTRL
        ldx     #$FF
        txs
        lda     #0
        ldx     #$40            ; zero page $40-$FF: never the TIA below it
@zp:    sta     $00,x
        inx
        bne     @zp
        sta     DISP_PTR        ; the cart's RAM, $4000-$7FFF
        tay
        ldx     #$40
@pg:    stx     DISP_PTR+1
@b:     sta     (DISP_PTR),y
        iny
        bne     @b
        inx
        cpx     #$80
        bne     @pg
        jsr     disp_init
        jmp     main
```

Then the claim and the vectors: the claim with the RAM flag set (and, for a banked program, kind 5), the cartridge flags `$FF`, `$87` at `$FFF8`, and the NMI, reset and IRQ vectors. Every program supplies `main` and `nmi`; an `nmi` that is only `RTI` is enough while nothing turns display-list interrupts on.

Building

`a78.cfg` lays out a 32K program: code from `$8000`, the claim at `$FF70`, the signature area left empty, the vectors at `$FFF8`; the zero page from `$40`; `BSS` in the cartridge's RAM at `$4000`. `listings/asm/build.sh` assembles a program with the three shared files, links it, checks it with `checkrom.py`, and writes the `.a78`:

```
cd listings/asm
./build.sh netget               # build/netget.bin and build/netget.a78
```

The script finds a program in `listings/asm/` first and in `listings/common/` second, so the bring-up's own `hello`, `fujitest`, `fujiboot` and `fujibank` build the same way.

8 C and fujinet-lib

The library every FujiNet C program uses, on every computer the FujiNet serves, is fujinet-lib: the same `fuji_get_adapter_config_extended()`, the same `network_open()`, whatever is underneath. Its `atari7800` target puts the mailbox underneath, and brings the start-up code, linker configuration and checker a 7800 program needs. A C program for the FujiNet cartridge is an ordinary cc65 program with three things swapped in.

What the target brings

File	What it does
`bus/atari7800/crt0.s`	Start-up, in place of cc65's. It never stores to `$00`–`$1F` (cc65's locks INPTCTRL), turns MARIA's DMA off, sets both stacks, clears `BSS`, copies `DATA` into RAM, and points the NMI vector at a trampoline in RAM. It also carries the claim, with the RAM flag set.
`makefiles/atari7800-fujinet.cfg`	The linker configuration: 32K of ROM at `$8000`, the cartridge's 16K of RAM at `$4000`.
`makefiles/atari7800-romstamp.py`	The checker: claim, vectors, no read-modify-write on the mailbox, no store to the TIA (Section 3).
`bus/atari7800/fujinet-bus-atari7800.c`	The transport: register writes, the TX stream, the wait.
`include/fujinet-atari7800.h`	What only a 7800 has: the cartridge's mode, the boot, the TV standard, INPTCTRL, the High Score Cart.

Memory	Where	What
`ZP`	`$40`–`$FF`	cc65's zero-page registers and the program's `ZEROPAGE`.
`CONSOLE`	`$1800`–`$203F`	The console's RAM, for anything placed there by name (`#pragma bss-name`).
`CRAM`	`$4000`–`$77FF`	`VIDEO` (page-aligned: the tile engine's map and lists), `DATA`, `BSS`, the heap.
`STACK`	`$7800`–`$7FFF`	cc65's parameter stack, 2K.
`ROM`	`$8000`–`$FF6F`	`TILES` (page-aligned, for `CHARBASE`), code, read-only data.
`CLAIM, VEC`	`$FF70, $FFF8`	Filled by `crt0.s`.

First Contact, built this way, uses 2K of tiles, 3K of code and 4.6K of the cartridge's RAM for the tile engine. Most of the 32K is still free.

Calling the FujiNet

The calls are fujinet-lib's usual ones, and they behave the usual way: `true` for success, or `FN_ERR_OK` for the network and clock calls. On the 7800, most `fuji_*` calls are macros over one function, `fuji_bus_call()`, which writes the registers and TX stream, waits, and copies the reply into the buffer you gave it.

Header	Calls
`fujinet-fuji.h`	The adapter, WiFi, host and device slots, directories, mounting, appkeys, hashing, Base64, GUIDs.
`fujinet-network.h`	The N: device: open, status, read, write, close, JSON, HTTP headers and POST, UDP.
`fujinet-clock.h`	The clock, in seven formats, and the time zone.
`fujinet-atari7800.h`	The cartridge itself (below).

The library waits as long as the cartridge allows each call, plus a second: 15 seconds by default (`fn_default_timeout`), 61 for MOUNT_IMAGE and COPY_FILE, 91 for the network device's OPEN, STATUS, READ and CLOSE. It counts vertical blanks, so the display does not change the timing. When a call fails, `fn_device_error` says why: the cartridge's error (Section 4), `0xFF` if the cartridge never answered, or 144 if the FujiNet said no.

The 7800's own calls

Call	What it does
`fuji_a7800_present()`	True if a FujiNet cartridge answers (the magic bytes).
`fuji_a7800_mode()`	`FUJI_A7800_MODE_BOOT`, `_LOAD`, `_GAME` or `_APP`.
`fuji_a7800_boot_state()`, `_percent()`, `_error()`, `_got()`, `_total()`	A pushed image's progress (Section 12).
`fuji_a7800_staged()`	What the push left: ready, claimed, BIOS-OK, an HSC ROM.
`fuji_a7800_boot()`	Interrupts, DMA and POKEY off, joystick ports back to inputs,

Call	What it does
	then into the loader to start the pushed image. Does not return.
`fuji_a7800_config()`	The same, back to CONFIG.
`fuji_a7800_handover()`	How the last image was started: by the BIOS or by the loader.
`fuji_a7800_tv()`, `_set_tv()`, `_detect_tv()`	The TV standard the cartridge believes; set it; or count MARIA's lines (two frames) and tell it.
`fuji_a7800_inptctrl()`, `_inpt_locked()`	The cartridge's model of INPTCTRL.
`fuji_a7800_hsc()`, `_hsc_op()`	The High Score Cart (Section 13).
`fuji_a7800_nmi`	Three bytes of RAM, `JMP` to the NMI handler. Store your handler's address in bytes 1–2.

Underneath: the bus layer

The bus header, `fujinet-bus-atari7800.h`, is also a public interface; CONFIG uses it. It names every cell of the status page (`FN_ACKSEQ`, `FN_ERRCODE`, `FN_REPLYCMD`, `FN_BOOTSTAT`…), the reply (`FN_REPLY`), and the registers (`FNR_DEVICE`…), and it has three functions:

- `fn_regwr(reg, value)` — one register write, one `STA`;
- `fn_tx(byte)` — one byte into the TX stream;
- `fn_commit()` — launch with `ACKSEQ`+1 and wait; returns the cartridge's error or `FN_EWAIT`.

With these, a C program can send a command fujinet-lib has no call for, or read a reply in place instead of copying it. Section 4's recipe is written this way,

and the command reference shows the lower road wherever the library has no wrapper. For any command, `fuji_bus_call()` is the general form:

```
fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_GET_ADAPTERCONFIG_EXTENDED,
              FUJI_FIELD_REPLY, 0, 0, 0, 0, &ac, sizeof ac);
```

The third argument says what the call carries: `FUJI_FIELD_A1` up to `_A1_A2_A3_A4` for one to four one-byte parameters, `_B12` for one two-byte parameter (aux 1 and 2), `_B12_B34` for two, and `FUJI_FIELD_DATA` for a payload or `FUJI_FIELD_REPLY` for a reply in the buffer.

Rules

A. **No TIA.** Leave out cc65's TIA sound, and do not call `clock()` or use conio: they drag in cc65's own start-up code, which locks INPTCTRL, and the link fails. Make sound with the cartridge's POKEY at `$0450`.

B. **No read-modify-write on the mailbox.** Never `++`, `--`, `|=` or `&=` a mailbox cell; the romstamp refuses the program if you do.

C. **Paths are 256 bytes.** `fuji_open_directory()` and `fuji_set_device_filename()` send exactly 256 bytes from the pointer you give them. Give them a 256-byte buffer, not a string literal.

D. **A reply is the size of its buffer.** `fuji_read_appkey()` needs two bytes more than the key; read the header comment of any call you use.

Building

`listings/c/build.sh` compiles a program with `-t atari7800 -O`, the library's two include directories and the tile engine, links it with the library's configuration, then runs the romstamp, `checkrom.py` and `mka78.py`:

```
cd listings/c
FNLIB=~/fujinet-lib-experimental ./build.sh acfg
```

SHELL · build.sh

```
for p in "$@"; do
    objs=()
    for src in "$p".c ${EXTRA:-}; do
        o="build/$(basename "${src%.*}").o"
        cl65 "${CFLAGS[@]}" -c -o "$o" "$src"
        objs+=("$o")
```

A program bigger than 32K needs its own linker configuration, and its own claim; Section 14 builds one.

9 7800basic on the Mailbox

7800basic compiles a BASIC program into 6502 assembly, assembles it with dasm, and produces an ordinary 7800 cartridge image. Nothing in it knows about the FujiNet. Three things make one of its programs a FujiNet program: names for the mailbox, a small library of subroutines, and ten bytes stamped into the finished image.

The names

In 7800basic, `dim name = $address` makes a variable that ***is*** that address. So the mailbox needs no special statements at all: `fujinet.bas` names every cell and register a program uses, and a store to the name is a store to the mailbox.

7800basic · fujinet.bas

```
rem cells the cartridge paints: read them
dim FN_REPLY = $0800
dim FN_ACKSEQ = $0C00
dim FN_STATUS = $0C01
dim FN_ERR = $0C02
dim FN_REPLYCMD = $0C03
dim FN_RXLEN = $0C04
dim FN_RXLENHI = $0C05
dim FN_BOOTSTATE = $0C06
dim FN_BOOTPCT = $0C07
dim FN_BOOTERR = $0C08
dim FN_MAGIC0 = $0C09
dim FN_MAGIC1 = $0C0A
dim FN_MODE = $0C12
dim FN_INPTLOCK = $0C1D

rem registers it watches: write them, one store each
dim FN_DEVICE = $0D00
dim FN_CMD = $0D01
dim FN_NPARAM = $0D02
dim FN_DATA_RST = $0D05
dim FN_SEQ = $0D10
dim FN_BOOTLOCK = $0D11
dim FN_TX = $0F00
```

`FN_CMD = fn_cmd` compiles to exactly what Section 4 asks for:

```
  lda fn_cmd
  sta FN_CMD
```

Values that never change are `const`: `FN_FUJI`, `FN_NET`, `FN_OK` and the rest. A program brings the names in first, with `incbasic`, which pastes another BASIC file in where it stands:

```
incbasic fujinet.bas
```

The library

`fujilib.bas` is `fujilib.s` again, subroutine for subroutine, written in BASIC. Its subroutines take their arguments in the library's own variables, `var90`–`var99`:

Subroutine	Takes	Does
`fn_chk`	—	`fn_res` = 0 if a FujiNet cartridge answers, else `FN_ENOCART` (`$FD`).
`fn_beg`	`fn_dev`, `fn_cmd`, `fn_npr`	Writes the registers and rewinds the TX stream. Sets `fn_tmo` to 3.
`fn_pb`	`fn_val`	Appends a one-byte parameter.
`fn_pw`	`fn_val`, `fn_valhi`	Appends a two-byte parameter.
`fn_pad`	`fn_i`	Appends NULs until `fn_i` wraps: after a path of `fn_i` bytes, exactly 256.
`fn_go`	`fn_tmo`	Launches and waits, calling `drawscreen` each frame so the picture stays up. `fn_res` = 0, the cartridge's error, `FN_ENAK` (`$EE`) if the FujiNet said no, or `FN_EWAIT` (`$FF`) after `fn_tmo` × 256 frames.
`fn_hex`	`fn_val`	Two hex digits into `fn_hexbuf`, for `plotchars`.

The default timeout of 3 is about 13 seconds on NTSC and 15 on PAL. Set `fn_tmo` after `fn_beg` for a slow command: 16 for MOUNT_IMAGE, 22 for the network device. The library goes at the end of the program, after the main loop, where nothing falls into it:

```
main
 drawscreen
 goto main

 incbasic fujilib.bas
```

A transaction then reads like this:

7800basic · acfg.bas

```
 fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 if fn_res <> FN_OK then goto failed
```

The claim

7800basic fills the last 4K of the program upward from `$F000` with its own runtime and keeps `$FF7E`–`$FFFF` for a checksum, the signature and the vectors. The FujiNet claim goes at `$FF70`, so it fits whenever the runtime ends below that — which, in every program in this handbook, it does by more than 2K. `tools/basfix.py` checks and stamps it after dasm has built the image:

- it reads dasm's own count of bytes left in the reserved area, and refuses if the code reaches `$FF70`;
- it stamps `"FUJI"`, version 1, kind 0 (let the cartridge decide), no flags;
- it reads every instruction in dasm's listing, so graphics are never mistaken for code, and refuses a read-modify-write aimed at `$0D00`–`$0FFF`.

Then 7800basic's own `7800header` writes the `.a78` header, with the bank scheme the program asked for.

Rules

Every one of these was found the hard way, by a program in this handbook that compiled without a word and then did the wrong thing.

A. Do not name a variable with one letter. `a` to `z` are 7800basic's own variables, in zero page. `dim i = var2` fails the build with dasm's "EQU: Value mismatch".

B. No arithmetic on a mailbox name. `FN_TX = FN_TX + 1` compiles to `LDA $0F00` / `ADC #1` / `STA $0F00`: it reads a write page, which holds nothing meaningful, and appends that, plus one. Use the names only on the left of a plain assignment, and the cells only on the right.

C. No expression inside an array index. `line1[ix] = FN_REPLY[ix + 2]` compiles, silently, into nonsense: the listing shows `LDX i` (the variable `i`, not `ix`) and `ADC #2]`. Give the address its own name instead: `dim FN_R_VALUE = $0802`, then `FN_R_VALUE[ix]`.

D. No expression in a `for` bound. `for ix = 0 to len - 1` compiles as if it said `to len`: the `- 1` disappears and the loop runs once too often. One extra byte in an appkey is how this was found. Compute the bound first: `last = len - 1`, then `for ix = 0 to last`.

E. Copy before you plot. MARIA cannot read the mailbox (Section 5). Copy a string from the reply into an array in `$2200`–`$27FF`, and plot the array.

F. Index at most 255. An array index is one byte. For a reply field past the first 256 bytes, `dim` a name at its address.

INPTCTRL

7800basic's start-up locks INPTCTRL, as Atari's guidelines asked every cartridge to. On the FujiNet cartridge that has one consequence: a game the program boots (Section 12) can no longer be handed to the console's own BIOS, and the loader starts it directly instead. All 126 games the cartridge was tested with run that way too, so for most programs the lock does no harm.

A program that boots games can leave INPTCTRL unlocked with two constants that 7800basic's start-up and sound code check but its documentation does not mention:

```
const NOTIALOCK = 1
const NOTIALOCKMUTE = 1
```

`NOTIALOCK` makes the start-up write `$06`, unlocked, instead of `$07`. `NOTIALOCKMUTE` silences every TIA sound statement, because with INPTCTRL unlocked any store to the TIA rewrites it. Sound then has to come from the POKEY. `boot.bas` does this; its status line shows INPTCTRL unlocked where every other BASIC program shows it locked.

Banks

A program bigger than 48K uses 7800basic's bank switching (`set romsize 128k`), and two things must live in the last bank, which never moves: the library, so every bank can call it, and the font, because MARIA reads it every line and a font in a switched bank turns to garbage while another bank is in. Section 14 shows both.

Building

`listings/bas/build.sh` runs 7800basic's steps directly — the preprocessor, the compiler, the post-processor and dasm — then `basfix.py` and `7800header`:

SHELL · build.sh

```
(
    cd "$B"
    7800preprocess < "$p.bas" > "$p.bas.pre"
    7800basic -i "$bas7800dir" -b "$p.bas" -p "$p.bas.pre" > "$p.log"
    7800postprocess -i "$bas7800dir" > "$p.bas.asm"
    dasm "$p.bas.asm" -I"$bas7800dir/includes" -f3 -l"$p.bas.list.txt" -p20 \
         -s"$p.bas.symbol.txt" -o"$p.bas.bin" | 7800filter | tee dasm.log
)
python3 "$BOOK/tools/basfix.py" "$B/$p.bas.bin" "$B/$p.bas.list.txt" "$B/dasm.log"
(cd "$B" && 7800header -o -f a78info.cfg "$p.bas.bin" > /dev/null)
cp "$B/$p.bas.a78" "build/$p.a78"
```

```
cd listings/bas
./build.sh acfg                  # build/acfg.a78
```

10 First Contact

The first program that talks to the FujiNet asks it one question, GET_ADAPTERCONFIG_EXTENDED, and shows three parts of the answer: the WiFi network it is on, its IP address, and its firmware version. It is the smallest complete transaction there is: no parameters, no payload, a 240-byte reply.

Build and run

A. Start the FujiNet adapter (`fujinet-pc`), listening on bus-over-IP.

B. Build the program: `listings/asm/build.sh fujitest`, `listings/c/build.sh acfg` or `listings/bas/build.sh acfg`.

C. Run it: `emu/run.sh listings/c/build/acfg.a78`, or the same file in FujiNet Go Atari 7800.

D. The cartridge sees the claim, starts the program in APP mode, and the program's first transaction comes back in well under a second.

E. The SSID, IP address and firmware version appear. On the bench in this handbook the SSID is `Dummy Cafe`, fujinet-pc's stand-in for a real network, and the address is `127.0.0.1`.





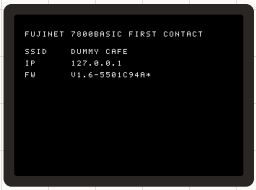


Figure 5. First Contact in assembly, C and 7800basic. The assembly one also shows ACKSEQ and ERR.

The reply

The reply is the adapter's configuration, both as raw bytes and as text ready to print. Everything is at an offset under 256, so one index register reaches all of it.

Offset	Size	Field
0	33	SSID, NUL-terminated
33	64	hostname
97, 101, 105, 109	4 each	IP address, gateway, netmask, DNS server: raw bytes

Offset	Size	Field
113, 119	6 each	MAC address, BSSID: raw bytes
125	15	firmware version
140, 156, 172, 188	16 each	IP address, gateway, netmask, DNS server: as text
204, 222	18 each	MAC address, BSSID: as text

In assembly

`fujitest.s` is the bring-up's own first contact, unchanged. It checks for the cartridge, makes the one transaction through the library, and puts the three fields on the screen straight from the reply window with `disp_rpl`, which copies as it draws.

6502 ASSEMBLY (ca65) · fujitest.s

```
have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     show
        jsr     fn_ack
        sta     aperr
```

6502 ASSEMBLY (ca65) · fujitest.s

```
ok:     PUTS    5, 2, tssid
        lda     #5
        ldx     #8
        jsr     disp_at
        ldx     #AC_SSID
        ldy     #30
        jsr     disp_rpl

        PUTS    7, 2, tip
        lda     #7
        ldx     #8
        jsr     disp_at
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl
```

It also shows ACKSEQ and ERR from the status page. Power the console off and on and run it again: ACKSEQ reads 02, not 01, because the cartridge was not reset and the program took its number from the cartridge (Section 4).

In C

fujinet-lib fills a struct with the reply, so the C version reads like any FujiNet C program on any computer. Only the screen is the 7800's.

C (cc65 + fujinet-lib) · acfg.c

```
    if (!fuji_a7800_present()) {
        mt_at(2, 5);
        mt_puts("NO FUJINET CARTRIDGE");
    } else if (!fuji_get_adapter_config_extended(&ac)) {
        mt_at(2, 5);
        mt_puts("FAILED: E");
        hex(fn_device_error);
    } else {
        field(5, "SSID", ac.ssid, 24);
        field(7, "IP", ac.sLocalIP, 16);
        field(9, "FW", ac.fn_version, 15);
    }
```

`fuji_get_adapter_config_extended()` is a macro over `fuji_bus_call()`: no parameters, a reply the size of `AdapterConfigExtended`. `mt_text` copies each field into the tile map and stops at its NUL.

In 7800basic

The BASIC version makes the same transaction with the library, then copies the three fields into console RAM, where `plotchars` can show them. The firmware and IP fields have names of their own at their addresses, `FN_R_FW` and `FN_R_IP`, so each copy is a plain indexed loop.

7800basic · acfg.bas

```
rem two of the reply's fields, by offset: firmware 125, IP text 140
dim FN_R_FW = $087D
dim FN_R_IP = $088C
rem console RAM for the strings, where MARIA can read them
dim ssid = $2200
dim ipaddr = $2220
dim fwver = $2230
dim ix = var0
```

7800basic · acfg.bas

```
fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
gosub fn_beg
gosub fn_go
```

Note: The `plotchars` statements can come before the copies: MARIA reads the arrays every frame, so the text appears the moment the bytes land.

11 The Network Device

The FujiNet's network device is how a 7800 reaches the rest of the world: HTTP and HTTPS, TCP and UDP, FTP, TNFS, and more, all behind one small set of commands. There are eight of them, N1: to N8:, devices `$71` to `$78`, each a connection of its own. A program names the far end with a **devicespec** such as `N:http://example.com/file.txt` and then opens, reads, writes and closes it like a file.

The lifecycle

Command	Code	Parameters	Payload and reply
OPEN	`$4F 'O'`	mode, translation (one byte each)	Payload: the devicespec. Reply: none.
STATUS	`$53 'S'`	two one-byte zeros	Reply: bytes waiting (two bytes, low first), connected (1 byte), status (1 byte; 1 is good).
READ	`$52 'R'`	the count (two bytes)	Reply: the bytes.
WRITE	`$57 'W'`	the count (two bytes)	Payload: the bytes.
CLOSE	`$43 'C'`	none	—

The modes are the protocol's: 4 to read (an HTTP GET), 8 to write (PUT), 12 to read and write (a GET that may also set headers), 13 for an HTTP POST. Translation 0 leaves the bytes alone; 1, 2 and 3 turn the far end's line endings into CR, LF or CR LF.

Five rules make the lifecycle behave:

A. **STATUS until it settles.** The adapter answers STATUS as soon as some of the response has arrived, so one reading can be a fraction of it. Read STATUS until two readings agree and are not zero.
B. **READ at most what fits.** The reply window holds 1024 bytes, and a READ's answer must fit in it. Ask for no more than STATUS says is waiting, and no more than 1024.
C. **Wait long enough.** The cartridge gives OPEN, STATUS, READ and CLOSE 90 seconds each. An HTTP request is not even sent until the first STATUS or READ, so that is where a slow server shows.

D. **Close what you open**, every time, including on the way out of a failure. The adapter keeps a connection open until it is told otherwise, even across a power cycle of the console.
E. **Use the devicespec on every call** in C: fujinet-lib takes the unit number from it.

netget

`netget` fetches `http://127.0.0.1:8765/hello.txt` — on the bench, a five-line file served by `python3 -m http.server` from `listings/fixtures` — and shows it a line at a time: LF ends a row, CR is skipped, a row stops at the edge.



Figure 6. netget in assembly, C and 7800basic.

In assembly

OPEN takes its two parameters, then the devicespec streamed into the TX page a byte at a time:

6502 ASSEMBLY (ca65) · netget.s

```
open:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCOPEN
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldy     #0
@url:   lda     url,y
        beq     @urle
        jsr     fn_txb
        iny
        bne     @url
@urle:  jsr     go_ack
```

STATUS until two readings agree, three frames apart:

6502 ASSEMBLY (ca65) · netget.s

```
stat:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCSTAT
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY+NSAVLO
        sta     navail
        lda     FN_REPLY+NSAVHI
        sta     navail+1
        cmp     nprev+1
        bne     again
        lda     navail
        cmp     nprev
        bne     again
        ora     navail+1
        bne     read
again:  lda     navail
        sta     nprev
        lda     navail+1
        sta     nprev+1
        inc     ntry
        lda     ntry
        cmp     #NTRIES
        bcc     @wait
        lda     #FNEWAIT
        sta     aperr
        jmp     fail
@wait:  ldx     #3              ; three frames between readings
@w:     jsr     disp_vbl
        dex
        bne     @w
        jmp     stat
```

READ takes one two-byte parameter. The text is then walked straight out of the reply window with a pointer, since it can be longer than 256 bytes:

6502 ASSEMBLY (ca65) · netget.s

```
@len:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCREAD
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     navail
        ldx     navail+1
        jsr     fn_pw
        jsr     go_ack
```

Note: The library's default wait is about 11 seconds. A real server can take longer than that to answer the first STATUS; set `FN_TMO` to 130 after `fn_beg` for each network command, and the cartridge's 90 seconds will run out first.

In C

fujinet-lib does the STATUS loop inside `network_read()`, which returns how many bytes it read, or a negative number on failure:

C (cc65 + fujinet-lib) · netget.c

```
    network_init();
    err = network_open(URL, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
    if (err)
        fail("OPEN", err);
    n = network_read(URL, buf, sizeof buf);
    if (n < 0)
        fail("READ", fn_device_error);
```

`network_init()` does nothing on the 7800, but some platforms need it, so a portable program calls it before the first `network_open()`. The library waits 91 seconds for each network call.

In 7800basic

The BASIC version follows the assembly step for step. The devicespec is an `alphadata` table, whose length 7800basic provides as `url_length`; the text goes into a 32 × 7 map in console RAM that `plotmap` put on the screen before anything arrived.

7800basic · netget.bas

```
 fn_dev = FN_NET : fn_cmd = $4F : fn_npr = 2
 gosub fn_beg
 fn_val = 12 : gosub fn_pb
 fn_val = 0 : gosub fn_pb
 ix = 0
open_url
 FN_TX = url[ix]
 ix = ix + 1
 if ix < url_length then goto open_url
 fn_tmo = 22
 gosub fn_go
 if fn_res <> FN_OK then goto failed
```

7800basic · netget.bas

```
  rem into the map, a row per line
  src = 0 : pos = 0
 show
  if src = FN_RXLEN then goto close
  ch = FN_REPLY[src]
  src = src + 1
  if ch = 13 then goto show
  if ch <> 10 then goto show_char
  pos = pos & $E0
  pos = pos + 32
  if pos = 224 then goto close
  goto show
 show_char
  ix = pos & 31
  if ix = 31 then goto show
  scr[pos] = ch
  pos = pos + 1
  goto show
```

The map is 224 bytes, so a one-byte position reaches all of it, and `pos & $E0` is the start of the current row.

JSON

An open connection can parse what it fetched as JSON, so a program never has to. Switch the channel to JSON mode (`$FC`, mode 1 as the second of two parameters), send PARSE (`$50 'P'`), then for each value send QUERY (`$51 'Q'`) with the path — `/0/name`, say — as the payload, and STATUS and READ the value's text. In C that is `network_json_parse()` and `network_json_query()`. The command cards in Section 17 have the details. FujiNet's game servers also answer in plain binary when asked (`?bin=1`), which is what the games do.

12 Directories, Mount and Boot

A FujiNet keeps up to eight **host slots**: places it can find files, such as its own SD card or a TNFS server on the network. This section lists a directory on one of them, and then does what CONFIG does with the file you pick: pushes it into the cartridge and runs it.

Hosts and directories

Command	Code	Parameters, payload, reply
MOUNT_HOST	$F9	The host slot (one byte). Connects to the host.
OPEN_DIRECTORY	$F7	The host slot; then exactly 256 bytes of payload: the path, a NUL, and optionally a filter pattern after the NUL, padded with NULs.
READ_DIR_ENTRY	$F6	The longest name wanted, and flags (one byte each). Reply: the next name, or $7F,$7F when there are no more.
CLOSE_DIRECTORY	$F5	None.
SET_DIRECTORY_POSITION	$E4	A position (two bytes): where the next READ_DIR_ENTRY starts, for paging back and forth.
READ_HOST_SLOTS	$F4	None. Reply: the eight hosts' names, 32 bytes each.

Warning: Do not ask READ_DIR_ENTRY for names of exactly 31 bytes. At that length the firmware puts icon bytes in front of the name. Every program here asks for 30 or 36.



Figure 7. The directory of `SD:/a7800book/`, three ways.

In assembly

`fn_path` streams the path and pads it to 256 bytes; READ_DIR_ENTRY's reply is drawn straight from the reply window.

6502 ASSEMBLY (ca65) · fujidir.s

```
        ; OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCODIR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     go_ack

        ; READ_DIR_ENTRY: two one-byte parameters, the longest name wanted
        ; and the flags; one name a call
        lda     #ROW0
        sta     row
entry:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCRDIR
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NAMELN
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY        ; $7F,$7F: the end of the directory
        cmp     #$7F
        bne     @show
        lda     FN_REPLY+1
        cmp     #$7F
        beq     close
@show:  lda     row
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #NAMELN
        jsr     disp_rpl
        inc     row
        lda     row
        cmp     #ROW0+NROWS
        bne     entry
```

In C

`fuji_open_directory()` sends 256 bytes from its pointer, so the path lives in a 256-byte buffer:

C (cc65 + fujinet-lib) · dir.c

```
    strcpy(path, "/a7800book/");
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_open_directory(HOSTSL, path))
        fail("OPEN_DIRECTORY");
    for (row = ROW0; row < ROW0 + ROWS; ++row) {
        if (!fuji_read_directory(NAMELN, 0, name))
            fail("READ_DIR_ENTRY");
        if (name[0] == 0x7F && name[1] == 0x7F)
            break;                      /* the end of the directory */
        mt_at(2, row);
        mt_text(name, NAMELN);
    }
```

In 7800basic

The path is copied byte by byte from an `alphadata` table, and `fn_pad` makes up the rest of the 256:

7800basic · dir.bas

```
 rem OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
 fn_dev = FN_FUJI : fn_cmd = $F7 : fn_npr = 1
 gosub fn_beg
 fn_val = HOSTSL : gosub fn_pb
 fn_i = 0
open_path
 FN_TX = path[fn_i]
 fn_i = fn_i + 1
 if fn_i < path_length then goto open_path
 gosub fn_pad
 gosub fn_go
 if fn_res <> FN_OK then goto failed

 rem READ_DIR_ENTRY: the longest name wanted, and the flags
 pos = 0
entry
 fn_dev = FN_FUJI : fn_cmd = $F6 : fn_npr = 2
 gosub fn_beg
 fn_val = NAMELN : gosub fn_pb
 fn_val = 0 : gosub fn_pb
 gosub fn_go
 if fn_res <> FN_OK then goto failed
 if FN_REPLY[0] <> $7F then goto entry_show
 if FN_REPLY[1] = $7F then goto close
entry_show
 for ix = 0 to NAMELAST
   if FN_REPLY[ix] = 0 then goto entry_next
   names[pos] = FN_REPLY[ix]
   pos = pos + 1
 next
entry_next
 pos = pos & $E0
 pos = pos + 32
 if pos <> 0 then goto entry
```

Mounting and booting

To run a file, a program asks the FujiNet to **mount** it on one of its device slots. On the 7800 cartridge, mounting means more: the FujiNet reads the whole file and pushes it, while the MOUNT_IMAGE request is still outstanding, into the cartridge, which keeps it until the program says go.

Command	Code	Parameters
MOUNT_HOST	$F9	The host slot.
SET_DEVICE_FULLPATH	$E2	Device slot, host slot, mode (1 for read), one byte each; then 256 bytes of path.
MOUNT_IMAGE	$F8	Device slot, mode. The cartridge allows 60 seconds.

Then the cartridge's status page takes over:

Address	Name	Meaning
$0C06	BOOT_STATE	0 idle, 1 receiving, 2 ready, $80 failed.
$0C07	BOOT_PCT	0–100, for a progress bar.
$0C08	BOOT_ERR	Why it failed: 1 too big, 2 truncated, 3 no board layout fits it, 4 the only store that fits it is in use.
$0C16–$0C1B	BOOT_GOT, BOOT_TOT	Bytes received, and the size (24 bits each, low first). Read each twice and keep a matching pair: the bytes can change between reads.
$0C1F	STAGED	What arrived: bit 0 ready, bit 1 it is a FujiNet app, bit 2 this console's BIOS will accept it, bit 3 it is a High Score Cart ROM.

The push can finish before MOUNT_IMAGE's answer comes back, or after it. Wait for BOOT_STATE to read 2 (or $80, and then BOOT_ERR says why), then:

A. Store $B5 to BOOTLOCK ($0D11).
B. Turn interrupts and MARIA's DMA off: SEI, then $7F to CTRL.
C. JMP $0600. The loader takes the image in, a kilobyte at a time, and starts it. Nothing comes back.

The cartridge keeps a pushed image in its own RAM (160K) or, if bigger, in its flash (512K), so a 448K image can be pushed into a 7800.

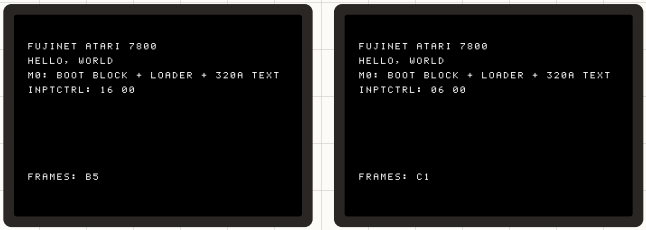


Figure 8. After `boot.c` and `boot.bas`: the pushed program, the bring-up's `hello`, running. The BASIC booter left INPTCTRL unlocked (`06 00`), as Section 9 explains.

In assembly

`fujiboot.s` is the bring-up's, unchanged; the handbook's build points it at `/a7800book/hello.bin`.

6502 ASSEMBLY (ca65) · fujiboot.s

```
; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ----
; Three one-byte parameters and then a FULL 256-byte NUL-padded path.
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCSDFP
        sta     FN_CMD
        lda     #3
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #BOOTHST
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     fn_path
```

6502 ASSEMBLY (ca65) · fujiboot.s

```
; ---- wait for the staged image ----
        lda     #4
        sta     apstep
        ldy     #0
wait1:  ldx     #0
wait2:  lda     FN_BOOTSTATE
        cmp     #FN_BOOT_READY
        bne     :+
        jmp     boot
:       cmp     #FN_BOOT_FAILED
        bne     :+
        jmp     bfail
:
```

```
        dex
        bne     wait2
        dey
        bne     wait1
        lda     #FNEWAIT
        sta     aperr
        jmp     show
bfail:  lda     FN_BOOTERR
        sta     aperr
        lda     #5
        sta     apstep
        jmp     show

; ---- boot it ----
boot:   jsr     fn_blk          ; arm the load
.if ::BOOTDIRECT
        lda     #$07            ; lock INPTCTRL: no BIOS hand-over now
        sta     INPTCTRL
.endif
        jmp     fn_boot         ; the loader; does not return
```

In C

C (cc65 + fujinet-lib) · boot.c

```
    strcpy(path, BOOTPATH);
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_set_device_filename(FMREAD, HOSTSL, DEVSL, path))
        fail("SET_DEVICE_FULLPATH");
    if (!fuji_mount_disk_image(DEVSL, FMREAD))
        fail("MOUNT_IMAGE");

    /* the push may finish before or after MOUNT_IMAGE returns */
    mt_at(2, 4);
    for (;;) {
        state = fuji_a7800_boot_state();
        pct = fuji_a7800_boot_percent();
        while (shown < pct / 4) {       /* 25 cells for 100 per cent */
            mt_put('#');
            ++shown;
```

`fuji_a7800_boot()` does the last three steps: BOOTLOCK, interrupts and DMA off, POKEY quiet, the joystick ports back to inputs, and the jump.

In 7800basic

7800basic · boot.bas

```
  rem the push may finish before or after MOUNT_IMAGE answers; a cell of
  rem the bar for every 4 per cent
  shown = 0
push
  ix = FN_BOOTPCT / 4
push_bar
  if shown >= ix then goto push_state
  bar[shown] = 35
  shown = shown + 1
  goto push_bar
push_state
  if FN_BOOTSTATE = FN_BOOT_READY then goto go
```

7800basic has no statement for "jump to the loader", so the last three instructions are inline assembly, between `asm` and `end`.

How the image is started

The loader starts an image in one of two ways, and the status page says which (`HANDOVER`, `$0C25`):

- **By the console's own BIOS** (1): the cartridge maps the BIOS back in and lets it check and start the game exactly as at power-on. It does this only for an image that is not a FujiNet app, that this console's BIOS is known to accept (the cartridge carries a verdict for every known game, NTSC and PAL), and only while INPTCTRL is still unlocked.
- **Directly** (2): the loader starts it, leaving what the BIOS leaves: X and S at `$16`, decimal and interrupts set, MARIA's DMA off. A FujiNet app is always started this way.

A game that runs either way does not care. A game that reads something the BIOS leaves behind is better started by the BIOS, which is why a program that boots games should leave INPTCTRL alone.

Board layouts

The cartridge serves a pushed image the way its own board would have, and says which layout it chose at `$0C14`:

Id	Layout	How it banks
0	plain ROM	Up to 48K, ending at `$FFFF`; `$FF` below it.
1	ROM + POKEY	As 0, with a POKEY at `$4000`.
2	SuperGame	16K banks: any store to `$8000`–`$BFFF` picks the bank shown there; the last bank is fixed at `$C000`.

Id	Layout	How it banks
3	SuperGame + POKEY	As 2, with a POKEY.
4	SuperGame + RAM	As 2, with 16K of RAM at `$4000` (8K boards see it twice).
5	SuperGame, 9 banks	144K: the bank picked is the value stored plus one.
6	RAM board	RAM at `$4000`, address line 8 ignored.
7	Absolute	A store to `$8000`: bit 0 or 1 picks the bank shown at `$4000`.
8	Activision	A store at `$E000` or above: the address's low 3 bits pick the bank; its halves appear at `$A000` and `$C000`.
9	High Score Cart	The HSC itself (Section 13).

The cartridge chooses in order: its database of every known game by CRC, a `mapper=` line in a `.cfg` file beside the image, the `.a78` header's type, and the image's size. A FujiNet app's claim names its layout with the kind byte (Section 3), which counts unless the database or a `.cfg` file has spoken.

13 *Keys, Clock and Utilities*

A FujiNet can remember things for a program, tell it the time, and do the arithmetic a 6502 should not have to. This section covers the three a program reaches for first — appkeys, the clock, and the utilities — and the one the 7800 cartridge adds: a High Score Cart whose scores live on the FujiNet.

Appkeys

An **appkey** is a small file on the FujiNet's SD card, named by three numbers: a **creator** (two bytes), an **application** and a **key** (one byte each). A program reads and writes its own keys to remember settings, names and scores between sessions; every FujiNet game reads the same key, creator 1, app 1, key 0, for the player's name.

Command	Code	Parameters, payload, reply
OPEN_APPKEY	`$DC`	No parameters. Payload: six bytes — creator (low, high), app, key, mode (0 read, 1 write), and a reserved 0 that must be sent.
READ_APPKEY	`$DD`	None. Reply: the length (two bytes, low first), then the value.
WRITE_APPKEY	`$DE`	The length (one byte); then the value as the payload.
CLOSE_APPKEY	`$DB`	None.

A key holds up to 64 bytes. Register a creator id of your own on the FujiNet wiki's AppKey Registry before you ship a program that writes keys. The 7800 cartridge's own creator is `$7800`; its app 1 is the High Score Cart (below). This handbook's programs use app `$FF` of the same creator as scratch.

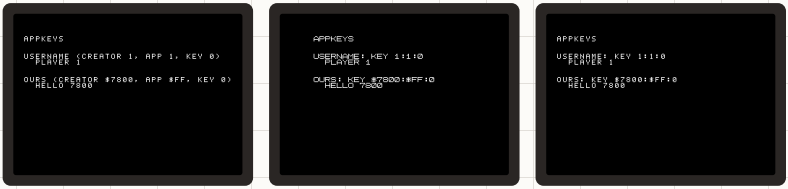


Figure 9. The shared player name, then a key of our own written and read back.

In assembly

6502 ASSEMBLY (ca65) · appkey.s

```
; akopen -- OPEN_APPKEY for the key in akcre/akapp/akkey, mode akmode: no
; parameters, the six-byte struct as the payload.
.proc akopen
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKOP
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        lda     akcre
        jsr     fn_txb
        lda     akcre+1
        jsr     fn_txb
        lda     akapp
        jsr     fn_txb
        lda     akkey
        jsr     fn_txb
        lda     akmode
        jsr     fn_txb
        lda     #0              ; reserved, and not optional
        jsr     fn_txb
        jmp     go_ack
.endproc

; akread -- READ_APPKEY: the value is left in the reply window, two bytes of
; length, then the bytes.
.proc akread
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKRD
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jmp     go_ack
.endproc

; akwrite -- WRITE_APPKEY: one parameter, the length, then the value.
.proc akwrite
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKWR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #valend-val
        jsr     fn_pb
        ldy     #0
@w:     lda     val,y
        jsr     fn_txb
        iny
        cpy     #valend-val
        bne     @w
        jmp     go_ack
.endproc
```

In C

fujinet-lib keeps the creator and app in `fuji_set_appkey_details()`*, and its read and write each do the OPEN themselves:*

C (cc65 + fujinet-lib) · appkey.c

```
static void show(uint8_t row)
{
    uint16_t n;

    mt_at(4, row);
    if (!fuji_read_appkey(0, &n, value))
        mt_puts("(READ FAILED)");
    else if (n == 0)
        mt_puts("(EMPTY)");
    else
        mt_text((char *)value, n);
}
```

C (cc65 + fujinet-lib) · appkey.c

```
    fuji_set_appkey_details(0x7800, 0xFF, DEFAULT);
    if (!fuji_write_appkey(0, sizeof ours - 1, (uint8_t *)ours)) {
        mt_at(4, 10);
        mt_puts("(WRITE FAILED)");
    } else
        show(10);
```

The read's buffer must be two bytes bigger than the key: the library reads the length into it too.

In 7800basic

7800basic · appkey.bas

```
 rem akopen -- OPEN_APPKEY: no parameters, the struct as the payload
akopen
 fn_dev = FN_FUJI : fn_cmd = $DC : fn_npr = 0
 gosub fn_beg
 FN_TX = akcrelo
 FN_TX = akcrehi
 FN_TX = akapp
 FN_TX = 0
 FN_TX = akmode
 FN_TX = 0
 gosub fn_go
 return

 rem akread -- READ_APPKEY: the reply is the length (2 bytes), then the value
akread
 fn_dev = FN_FUJI : fn_cmd = $DD : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 return
```

7800basic · appkey.bas

```
 rem name; ours is creator $7800, app $FF, key 0. OPEN_APPKEY takes ONE
 rem six-byte struct as its payload: creator (low, high), app, key, mode,
 rem and a reserved zero.
 incbasic fujinet.bas
```

```
displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

rem READ_APPKEY's value starts after its two-byte length
dim FN_R_VALUE = $0802
dim line1 = $2200
dim line2 = $2220
dim akcrelo = var0
dim akcrehi = var1
dim akapp = var2
dim akmode = var3
dim ix = var4
dim len = var5
dim last = var6

for ix = 0 to 63 : line1[ix] = 32 : next
clearscreen
plotchars 'APPKEYS' 0 8 2
plotchars 'USERNAME: KEY 1:1:0' 0 8 5
plotchars line1 0 16 6 30
plotchars 'OURS: KEY $7800:$FF:0' 0 8 9
plotchars line2 0 16 10 30
gosub fn_chk
```

`last = value_length - 1` before the loop is Section 9's rule about `for` bounds; without it, the key gains a stray byte.

The clock

The clock is a device of its own, `$45`. Its commands take no parameters and answer with the time in one of several forms:

Command	Code	Reply
GET_ISO_LOCAL	`$49 'I'`	`2026-10-20T15:46:08+0000` and a NUL, in the adapter's time zone.
GET_ISO_UTC	`$5A 'Z'`	The same in UTC.
GETTIME	`$93`	Six bytes: day, month, year (two digits), hour, minute, second.
(simple time)	`$54 'T'`	Seven bytes: century, year, month, day, hour, minute, second.
SETTZ_ALT	`$74 't'`	Payload: a POSIX time-zone string, such as `EST5EDT`: the zone the local reads use.



Figure 10. The clock. fujinet-pc on the bench runs in UTC, so both lines agree.

6502 ASSEMBLY (ca65) · clock.s

```
.proc main
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     @go
        lda     #FNENOC
        sta     aperr
        jmp     fail
@go:    PUTS    5, 2, tlocal
        ASK     CKISOL, 5
        PUTS    7, 2, tutc
        ASK     CKISOU, 7
loop:   jmp     loop
```

C (cc65 + fujinet-lib) · clock.c

```
static void line(uint8_t row, const char *label, TimeFormat f)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(10, row);
    if (clock_get_time(when, f))
        mt_puts("FAILED");
    else
        mt_text((char *)when, 25);
}
```

7800basic · clock.bas

```
 fn_dev = FN_CLOCK : fn_cmd = $49 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 if fn_res <> FN_OK then goto failed
 for ix = 0 to 24 : local[ix] = FN_REPLY[ix] : next

 fn_dev = FN_CLOCK : fn_cmd = $5A : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 if fn_res <> FN_OK then goto failed
 for ix = 0 to 24 : utc[ix] = FN_REPLY[ix] : next
```

Entropy, hashing, encoding

The FujiNet does the work a 6502 would take seconds over:

- **GENERATE_GUID** (`$BB`) answers with a fresh 36-character GUID.
- **HASH_INPUT** (`$C8`), **HASH_COMPUTE** (`$C7`), **HASH_LENGTH** (`$C6`) and **HASH_OUTPUT** (`$C5`) hash data: feed it in pieces, compute with an algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512), ask the length, read the result. In C, `fuji_hash_data()` does all four.
- The **Base64** commands (`$D0`–`$C9`, encode and decode) work the same way.
- The **QR code** commands (`$BC`–`$BF`) turn text into a QR code's modules.

The cards in Section 16 show each one in all three languages.

The High Score Cart

Atari's High Score Cart sat between the console and a game and kept the scores of every game that knew about it, in 2K of battery-backed RAM at `$1000`–`$17FF`, with its own ROM at `$3000`–`$3FFF`. The FujiNet cartridge can be one. It serves the HSC ROM — your own copy, installed once from CONFIG and kept in the cartridge's flash; none is supplied — and keeps the 2K of RAM as 32 appkeys of 64 bytes, creator `$7800`, app 1, keys 0–31, on the FujiNet's SD card. While a game runs, the cartridge saves each 64-byte chunk the game changed, two seconds after the last change. The scores follow the SD card, not the cartridge.

A program sees and controls it through the status page and one register:

Where	What
`$0C26` HSC	Bit 0 a ROM is installed; bit 1 games get the HSC; bit 2 the last save reached the SD card; bit 3 scores are waiting to be saved.
`$0D16` HSC	0 turn it off, 1 on, 2 forget the installed ROM, 3 install the pushed image (STAGED bit 3) as the HSC ROM.

In C, `fuji_a7800_hsc()` reads the bits and `fuji_a7800_hsc_op()` writes the register. CONFIG's information screen has the switch.

Note: The HSC is for games: the cartridge saves its RAM only while a game (GAME mode) runs. A FujiNet program keeps its own data in its own appkeys, as above.

14 Banks and SuperGame

A 7800 sees 48K of cartridge at most. Atari's answer for bigger games was **SuperGame** banking: the image is cut into 16K banks, the last one sits at `$C000`–`$FFFF` for good, and any of the others can be shown at `$8000`–`$BFFF`. A store anywhere in `$8000`–`$BFFF` picks which. The FujiNet cartridge does SuperGame, with or without 16K of RAM at `$4000`, and the mailbox stays put through every switch.

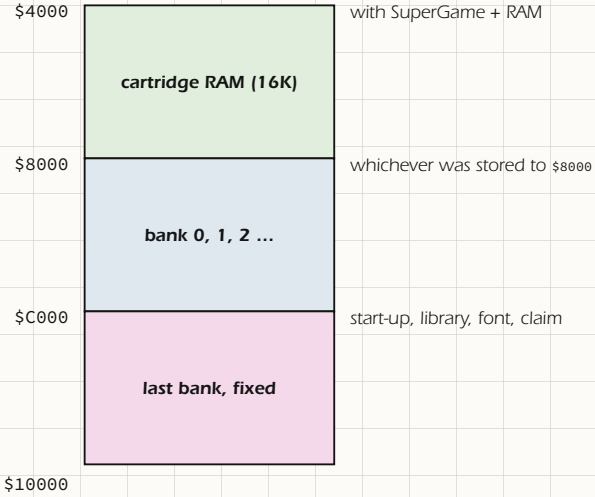


Figure 11. A SuperGame image. The mailbox, below `$1000`, never moves.

What goes where

Three rules decide what lives in the fixed bank:

A. **The code that switches banks.** Code that stores to `$8000` from a switched bank switches itself out from under the 6502. Switch from the fixed bank, and call into the switched one.

B. **Everything MARIA reads.** MARIA reads the font or tiles every line. If they live in a switched bank, the picture turns to garbage whenever another bank is in — and a program waiting for the FujiNet keeps the display running.

C. **The mailbox library, and anything every bank calls.** A switched bank can always call the fixed one.

And one to watch for: a **data** store into `$8000`–`$BFFF` switches banks too. A pointer that strays there does not write ROM; it changes what the 6502 is about to execute.

The cartridge has to know the image is SuperGame. A plain 64K or 128K image with no other clue would be taken for something else, so a FujiNet app says so in its claim's kind byte: 3 for SuperGame (the layout id 2, plus one), 5 for SuperGame with RAM. An `.a78` header that says SuperGame works too, if the claim leaves the kind at 0.



Figure 12. A transaction from each of three banks, three ways.

In assembly

`fujibank.s` is the bring-up's banked app: a 64K image, kind 5, three switched banks. Each bank's code at `$8000` makes a transaction and returns its own number, so a wrong bank shows on the screen:

6502 ASSEMBLY (ca65) · fujibank.s

```
.macro  BANKCODE n
        .segment .sprintf("BANK%d", n)
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        lda     #n
        rts
.endmacro
```

From the fixed bank, the program shows each bank and calls into it:

6502 ASSEMBLY (ca65) · fujibank.s

```
@b:     lda     bank
        sta     $8000           ; SuperGame: any write to $8000-$BFFF picks the bank
        jsr     $8000           ; the transaction, from the bank's own code
        cmp     bank
        beq     @right
        jmp     wrong
```

`a78bank.cfg` places the three banks at `$8000`, each its own 16K of the file, and everything else — start-up, the library, the font, the claim — in the fixed bank; `a78init.s` writes kind 5 when the build defines `BANKED`. After the three transactions the program boots `hello`, as `fujiboot` does, to show a banked app can.

In C

A banked C program needs two things fujinet-lib does not provide. `bank.cfg` is the library's linker configuration with the ROM split SuperGame's way:

LD65 CONFIGURATION · bank.cfg

```
    BANK0:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    BANK1:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    BANK2:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    ROM:     file = %O, define = yes, start = $C000, size = $3F70, type = ro, fill = yes, fillval = $FF;
```

and `crt0bank.s` is the library's start-up with one byte changed, the claim's kind:

6502 ASSEMBLY (ca65) · crt0bank.s

```
        .segment "CLAIM"
        .byte   "FUJI", 1, 5, 1, 0, 0, 0    ; version 1, SuperGame + RAM, cart RAM
```

Because the program's own `crt0bank.o` provides the start-up symbols, the linker never takes the library's. In the program, `#pragma code-name` puts each bank's function in its bank, and a small function in the fixed bank switches and calls:

C (cc65 + fujinet-lib) · bank.c

```
/* SuperGame: a store anywhere in $8000-$BFFF picks the bank shown there. */
#define BANK_SELECT (*(volatile uint8_t *)0x8000)

/* One function per bank, each at $8000 in its own bank. Each returns its
 * bank's number, so a wrong bank shows. */
#pragma code-name(push, "BANK0")
uint8_t bank0(void) { return fuji_get_adapter_config_extended(&ac) ? 0 : 0xFF; }
#pragma code-name(pop)
#pragma code-name(push, "BANK1")
uint8_t bank1(void) { return fuji_get_adapter_config_extended(&ac) ? 1 : 0xFF; }
#pragma code-name(pop)
#pragma code-name(push, "BANK2")
uint8_t bank2(void) { return fuji_get_adapter_config_extended(&ac) ? 2 : 0xFF; }
#pragma code-name(pop)

typedef uint8_t (*bankfn)(void);
static const bankfn entry[3] = { bank0, bank1, bank2 };

/* The call lives in the fixed bank, so it is still there after the switch. */
static uint8_t bank_call(uint8_t b)
{
    BANK_SELECT = b;
    return entry[b]();
}
```

The C runtime, fujinet-lib and the tile engine's tiles are all in the fixed bank by default, so a bank's function can call any of them. Build it with the configuration and the extra file:

```
cd listings/c
CFG=bank.cfg EXTRA=crt0bank.s ./build.sh bank
```

fujinet-lib's romstamp takes images of 48K at most, so for a bigger one `build.sh` runs `tools/checkbanks.py` instead: the same checks, with each bank scanned at its own place in the file.

In 7800basic

7800basic does the banking itself. `set romsize 128k` makes eight 16K banks, bank 8 fixed; `bank` starts each bank's code; `gosub label bankN` switches, calls and switches back when the subroutine ends with `return otherbank`. The library and the font go in bank 8:

7800basic · bank.bas

```
  gosub acfg
```

7800basic · bank.bas

```
 bank 2
fetch2
 gosub acfg
 if fn_res <> FN_OK then return otherbank
 for ix = 0 to 19 : ssid2[ix] = FN_REPLY[ix] : next
 return otherbank

 bank 3
fetch3
 gosub acfg
 if fn_res <> FN_OK then return otherbank
 for ix = 0 to 19 : ssid3[ix] = FN_REPLY[ix] : next
 return otherbank

 bank 8
 rem the font lives here too: MARIA reads it every line, and a font in
 rem bank 1 would turn to garbage while bank 2 or 3 is switched in
 incgraphic fujifont.png 320A

 rem acfg -- one GET_ADAPTERCONFIG_EXTENDED, from whichever bank calls
acfg
 fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 return
```

7800basic writes a SuperGame `.a78` header for a 128K program, and the claim's kind stays 0, so the cartridge goes by the header. Without its own RAM flag the program has no cartridge RAM, and needs none.

15 The Tile Engine

CONFIG and all five FujiNet games for the 7800 draw their screens with the same small engine, `maria.s` and `maria.h`, which each project copies into its own tree. The C programs in this handbook use it for their text. It is worth understanding on its own, because it is the shape of the problem every 7800 program with a lot on the screen has to solve: how much MARIA can read in a line, and how to change the picture without MARIA ever reading half a change.

The screen

32 × 24 cells of 8 × 8 pixels: 256 × 192, centred. The mode is 320B, where every pixel is two bits, so a tile has three colours and the background, the way NES tiles do. A cell holds a tile number (0–127) and a palette select (0 or 1). Pixel value 0 is `BACKGRND` in either palette; 1–3 are the selected palette's three colours. MARIA has eight palettes, but 320B honours only their top bit, so the two selects are palettes 0 and 4.

How it is drawn

Each row of cells is one 8-line zone. Its display list reads the row's 32 map bytes as **indirect two-byte characters**: each map byte is a tile number times two, and MARIA fetches two bytes of pixels for it from the tile set at `CHARBASE`. A row needs one object header per **run** of cells that share a palette, five bytes each:

map low	$E0	map high	pal · 32−width	hpos

`$E0` says extended header, indirect, write mode 1. So writing a tile costs nothing but the store: MARIA reads the map every line, and the next frame shows it. Changing a palette is different, because it can split or join runs and so change the list. `mt_setpal` marks the row; `mt_sync`, once a frame, rebuilds each marked row's list into the row's **spare** slot and then publishes it with one store to the row's DLL entry. MARIA reads either the old list or the new one, never half of each.

6502 ASSEMBLY (ca65) · maria.s

```
; Row X's list into its spare slot: one header per run of cells sharing a
; palette, then publish it with one store to the row's DLL entry.
.proc build
        stx     brow
        lda     rowlo,x
        sta     ptr1            ; the row's palette selects
        sta     tmp1            ; ...and map, low byte
        lda     rowhi,x
        sta     tmp2            ; map, high byte
        clc
        adc     #>(_mt_attr - _mt_map)
        sta     ptr1+1
        lda     slot,x
        eor     #1
        sta     slot,x
        beq     @s0
        lda     #SLOT
@s0:    clc
        adc     dllo,x
        sta     ptr2
        lda     dlhi,x
        sta     ptr2+1
```

What it costs

MARIA has 426 clocks a line to read its lists and the graphics, and it takes them from the 6502. A zone line here costs 16 clocks, plus 10 per run, plus 9 per cell (3 for the map byte, 6 for the two bytes of pixels). A one-run row is 314 clocks, leaving the 6502 about a third of every displayed line and about half of the whole frame. Rows are capped at 12 runs, 434 clocks being past the line; cells after the twelfth run take its palette.

Runs in a row	Clocks a line	Left for the 6502
1	314	112
4	344	82
12	424	2

Memory

Segment	Size	What
VIDEO (cartridge RAM)	768	The map: tile × 2 per cell.
	768	The palette selects, three pages after the map.
	3072	Two display-list slots of 64 bytes per row, both in one page.
	—	The DLL, a null list, a dirty flag and a slot flag per row.

Segment	Size	What
`TILES` (ROM)	2048	The tile set: 8 pages of 256 bytes, page 7 the top pixel row (MARIA counts a zone's lines downward), tile **t** at bytes 2 × **t** and 2 × **t** + 1 of each page.

The linker configuration puts both on page boundaries. The engine uses MARIA's own `WSYNC` at `$24`, owns `CTRL`, `CHARBASE`, `DPPH`/`DPPL`, `OFFSET`, `BACKGRND` and the palette registers it is told to set, and never stores below `$20`. It uses no display-list interrupts, so the NMI hook (`fuji_a7800_nmi`) is free for a game.

Making tiles

`mktiles.py` builds the tile set from either or both of two sources:

- **`font.txt`**, CONFIG's format: a header line `= c` (a character) or `= $hh` (a tile number), optionally `plane 1`, then up to eight rows of `#` and `.` `#` is colour 1 (or 3 with `plane 1`), `.` is colour 2. CONFIG uses colour 2 as the paper, which palette 1 turns into a selection bar.
- **NES CHR** (`--chr file`), 16 bytes a tile, plane 0 then plane 1: the FujiNet games' NES art converts unchanged, which is how most of the games got their 7800 graphics.

```
python3 mktiles.py font.txt build/tiles.s        # then assemble build/tiles.s
python3 mktiles.py --chr game.chr --chr-at 128 font.txt build/tiles.s
```

The calls

Call	What it does
`mt_init(blank)`	Measures NTSC or PAL, clears to tile `blank` in palette 0, builds the lists, display on at the next vertical blank.
`mt_off()`, `mt_on()`	MARIA's DMA off (the 6502 runs flat out) and on.
`mt_sync()`	Once a frame: waits for the vertical blank, counts it in `mt_frames`, rebuilds marked rows.
`mt_background(c)`, `mt_palette(p, c1, c2, c3)`	Colours: hue × 16 + luminance.
`mt_clear(blank)`	Every cell to `blank`, palette 0.
`mt_at(col, row)`	The write cursor. Writes move it along the row and stop at the end.
`mt_put(t)`, `mt_fill(t, n)`	One tile, or `n` of them.

Call	What it does
`mt_puts(s)`, `mt_text(s, max)`	Tiles from a string; `mt_text` stops at `max` and shows anything outside 32–126 as a space.
`mt_setpal(p, n)`	`n` cells from the cursor to palette `p`, cursor unmoved.
`mt_get(col, row)`	The tile at a cell.

The map and palette selects are `mt_map` and `mt_attr`, for a program that wants to write them directly. A map byte is the tile number times two.

16 Command Reference: The Fuji Device

Every command device `$70` answers, checked against the firmware the cartridge's ESP32-S3 runs: the RS-232 build's `rs232Fuji.cpp` and the four mixins every FujiNet shares (appkeys, Base64, hashing, QR codes). A command not on a card is answered with a NAK; the last section names them.

On the cards, **nparam** is the value for the NPARAM register, and each parameter goes into the TX stream as its size and then its value: a **byte** is 1 and the byte, a **word** 2 and two bytes, low first. The reply is in the reply window at `$0800`. The assembly examples use the library of Section 7 and its `FNBEGIN` macro (`devdefs.inc`), which sets `FN_DEV`, `FN_CMD` and `FN_NPR` and calls `fn_beg`. The C examples use fujinet-lib where it has a call, and its bus layer where it does not. The 7800basic examples use the library of Section 9. Every example is built by `tools/snipcheck.py`.

WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED / GET_ADAPTERCONFIG — $C4 / $E8

DEVICE	$70	**NPARAM**	0
PARAMS	none		
PAYLOAD	none		
REPLY	`$C4`: 240 bytes, laid out in Section 10. `$E8`: the first 140, addresses in binary only		

The first-contact command, and the one every status screen wants. It works before the WiFi is up, and its text fields save writing a routine to print an IP address.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCADPX, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+140    ; the IP address, as text
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_adapter_config_extended(&ac))
    failed();
mt_text(ac.sLocalIP, 16);
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
gosub fn_beg
gosub fn_go
if fn_res <> FN_OK then goto failed
ch = FN_REPLY[140] : rem the IP address, as text
```

SCAN_NETWORKS $FD

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: how many networks were found		

Starts a fresh scan and answers when it is done, which takes seconds; well inside the cartridge's five. Follow it with GET_SCAN_RESULT once for each network.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSCAN, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY        ; how many
        sta     cnt
```

C (cc65 + fujinet-lib)

```
if (!fuji_scan_for_networks(&count))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $FD : fn_npr = 0
gosub fn_beg
gosub fn_go
if fn_res <> FN_OK then goto failed
cnt = FN_REPLY[0]
```

GET_SCAN_RESULT $FC

DEVICE	$70	NPARAM	1
PARAMS	byte: which result, from 0		
PAYLOAD	none		
REPLY	34 bytes: the SSID (33, NUL-terminated), then the signal strength, a signed dBm		

The strength is negative: closer to zero is stronger.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSCNR, 1
        lda     idx             ; which network
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+33     ; signal strength, negative dBm
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_scan_result(idx, &si))
    failed();
mt_text(si.ssid, 24);           /* si.rssi: negative dBm */
```

7800basic

```
 fn_dev = FN_FUJI : fn_cmd = $FC : fn_npr = 1
 gosub fn_beg
 fn_val = idx : gosub fn_pb
 gosub fn_go
 if fn_res <> FN_OK then goto failed
 rssi = FN_REPLY[33]
```

SET_SSID $FB

DEVICE	$70	NPARAM	1
PARAMS	byte: any value; **required but ignored**		
PAYLOAD	**exactly 97 bytes**: the SSID (33) then the password (64), each NUL-padded		
REPLY	ACK		

Joins a network and saves it as the default. Two sharp edges: the parameter must be sent although nothing reads it, and the payload must be all 97 bytes; a short one fails the transaction. The BASIC example joins `Dummy Cafe` with no password: ten bytes of name, then 87 NULs.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSSSD, 1
        lda     #0
        jsr     fn_pb           ; the parameter that must be there
        ldx     #0
@pw:    lda     ssidpw,x        ; ssid[33] + password[64]
        jsr     fn_txb
        inx
        cpx     #97
        bne     @pw
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(nc.ssid, ac.ssid);      /* nc: a NetConfig, 33 + 64 */
nc.password[0] = 0;
if (!fuji_set_ssid(&nc))
    failed();
```

7800basic

```
 fn_dev = FN_FUJI : fn_cmd = $FB : fn_npr = 1
 gosub fn_beg
 fn_val = 0 : gosub fn_pb
 for ix = 0 to 9 : FN_TX = ssid[ix] : next
 for ix = 10 to 96 : FN_TX = 0 : next
 gosub fn_go
```

GET_SSID $FE

DEVICE	$70	**NPARAM**	0
PARAMS	none		
PAYLOAD	none		
REPLY	97 bytes: the SSID (33) then the password (64)		

The network the adapter is set to join, password in clear text.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCGSSD, 0
        jsr     fn_go
        bne     failed          ; SSID at FN_REPLY, password at +33
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_ssid(&nc))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $FE : fn_npr = 0
gosub fn_beg
gosub fn_go
```

GET_WIFISTATUS / GET_WIFI_ENABLED — $FA / $EA

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	$FA: 1 byte, 3 when connected; $EA: 1 byte, 1 if the radio is on		

What to poll while a join is in progress.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCWIFI, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY
        cmp     #3              ; connected?
        beq     done
```

C (cc65 + fujinet-lib)

```
if (fuji_get_wifi_status(&err) && err == 3)
    linked();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $FA : fn_npr = 0
gosub fn_beg
gosub fn_go
if FN_REPLY[0] = 3 then goto done
```

RESET — $FF

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK, then the adapter restarts		

Restarts the ESP32-S3. The cartridge, its mailbox and your program carry on; the link bit at $0C01 drops and comes back, and the cartridge waits up to three seconds for it before the next transaction.

STATUS / DEVICE_READY $53 / $00

DEVICE	$70	NPARAM	1 / 0
PARAMS	STATUS: byte, the kind of status wanted (1: mount times)		
PAYLOAD	none		
REPLY	STATUS 1: four bytes per device slot, a time, 0 if nothing is mounted; DEVICE_READY: the bus test's answer		

STATUS needs its parameter on this build. DEVICE_READY is a round trip with no other effect: it proves the mailbox, the cartridge, the USB link and the adapter at once.

Hosts, slots and mounting

READ_HOST_SLOTS $F4

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	256 bytes: eight host names of 32 bytes		

Slot 0 is the SD card on a standard `fujinet-pc`.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCRHST, 0
        jsr     fn_go
        bne     failed
        lda     #4              ; host 0's name on row 4
        ldx     #2
        jsr     disp_at
        ldx     #0              ; FN_REPLY + slot * 32
        ldy     #32
        jsr     disp_rpl
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_host_slots(hosts, 8))
    failed();
mt_text((char *)hosts[0], 32);
```

7800basic

```
 fn_dev = FN_FUJI : fn_cmd = $F4 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 for ix = 0 to 31 : buf[ix] = FN_REPLY[ix] : next
```

WRITE_HOST_SLOTS $F3

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	256 bytes: all eight slots, 32 bytes each		
REPLY	ACK		

All or nothing: there is no command to write one slot. Read the table with READ_HOST_SLOTS and stream it back with the one slot changed; the reply window holds still until the next launch, so the stream can be copied straight out of it.

MOUNT_HOST $F9

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot, from 0		
PAYLOAD	none		
REPLY	ACK; NAK if the host cannot be reached		

Connects the host: SD, TNFS, SMB and the rest. Needed before OPEN_DIRECTORY or a mount. UNMOUNT_HOST ($E6) is not answered on this build.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCMHST, 1
        lda     #0              ; host slot 0
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        jsr     fn_ack
        bne     refused
```

C (cc65 + fujinet-lib)

```
if (!fuji_mount_host_slot(0))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $F9 : fn_npr = 1
gosub fn_beg
fn_val = 0 : gosub fn_pb
gosub fn_go
if fn_res <> FN_OK then goto failed
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS $F2 / $F1

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	$F2: the device-slot table: host slot, mode and file name for each slot; $F1 takes the same table as its payload		

What is set up in each of the FujiNet's virtual drives. A 7800 boots images rather than reading drives, so CONFIG does not show them, but they answer.

SET_DEVICE_FULLPATH $E2

DEVICE	$70	NPARAM	3
PARAMS	bytes: device slot, host slot, mode (1 to read)		
PAYLOAD	**256 bytes**: the path, NUL-padded		
REPLY	ACK		

Names the file for a device slot: the first half of booting. fujinet-lib's call takes the mode first, the device slot last; the wire has the device slot first.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSDFP, 3
        lda     #0              ; device slot
        jsr     fn_pb
        lda     #0              ; host slot
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<file
        sta     FN_PTR
        lda     #>file
        sta     FN_PTR+1
        jsr     fn_path         ; padded to 256
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a7800book/hello.bin");
if (!fuji_set_device_filename(1, 0, 0, path))  /* mode, host, device */
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $E2 : fn_npr = 3
gosub fn_beg
fn_val = 0 : gosub fn_pb
fn_val = 0 : gosub fn_pb
fn_val = 1 : gosub fn_pb
last = file_length - 1
for fn_i = 0 to last : FN_TX = file[fn_i] : next
fn_i = file_length
gosub fn_pad
gosub fn_go
```

MOUNT_IMAGE / UNMOUNT_IMAGE — $F8 / $E9

DEVICE	$70	**NPARAM**	2 / 1
PARAMS	$F8: device slot, mode (1 to read); $E9: device slot		
PAYLOAD	none		
REPLY	ACK when the mount completes		

The second half of booting. While it is outstanding the FujiNet pushes the file into the cartridge, and BOOT_STATE and BOOT_PCT report the progress; the cartridge allows sixty seconds. Section 12 has the rest.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCMIMG, 2
        lda     #0              ; device slot
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #96             ; about a minute
        sta     FN_TMO
        jsr     fn_go
        bne     failed
@push:  lda     FN_BOOTSTATE    ; then watch the push
        cmp     #FN_BOOT_READY
        bne     @push
```

C (cc65 + fujinet-lib)

```
if (!fuji_mount_disk_image(0, 1))
    failed();
while (fuji_a7800_boot_state() != FUJI_A7800_BOOT_READY)
    mt_sync();
```

7800basic

```
 fn_dev = FN_FUJI : fn_cmd = $F8 : fn_npr = 2
 gosub fn_beg
 fn_val = 0 : gosub fn_pb
 fn_val = 1 : gosub fn_pb
 fn_tmo = 16
 gosub fn_go
wait_push
 if FN_BOOTSTATE <> FN_BOOT_READY then goto wait_push
```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT — $D7 / $D6 / $D9

DEVICE	$70	NPARAM	0 / 1 / 1
PARAMS	SET_BOOT_MODE: byte, the mode; CONFIG_BOOT: byte, on or off		
PAYLOAD	none		
REPLY	ACK		

MOUNT_ALL mounts every slot that has a file; the other two choose what the adapter serves at power-on on computers that boot from its drives. On the 7800 the cartridge decides what boots, and CONFIG does not send them.

Directories

OPEN_DIRECTORY — $F7

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot		
PAYLOAD	**256 bytes**: the path, a NUL, then an optional filter such as `*.A78`, the rest NULs		
REPLY	ACK		

Opens a directory on a mounted host. The adapter applies the filter, so a program never sees what it does not want. `fuji_open_directory2()` builds the path-and-filter payload for C.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCODIR, 1
        lda     #0              ; host slot
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     fn_go
        bne     failed
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a7800book/");
if (!fuji_open_directory(0, path))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $F7 : fn_npr = 1
gosub fn_beg
fn_val = 0 : gosub fn_pb
for fn_i = 0 to 10 : FN_TX = path[fn_i] : next
fn_i = path_length
gosub fn_pad
gosub fn_go
```

READ_DIR_ENTRY $F6

DEVICE	$70	NPARAM	2
PARAMS	bytes: the longest name wanted (**never 31**), flags (0)		
PAYLOAD	none		
REPLY	one name, NUL-terminated, a `/` at the end of a directory's; **$7F, $7F when there are no more**		

One name a call; the adapter moves on by itself. At a length of exactly 31 the firmware puts icon bytes in front of the name. Stop at the end marker: an SD host keeps answering `..` past it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCRDIR, 2
        lda     #36
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        lda     FN_REPLY
        cmp     #$7F            ; the end?
        beq     done
```

C (cc65 + fujinet-lib)

```
if (!fuji_read_directory(36, 0, buf))
    failed();
if (buf[0] == 0x7F && buf[1] == 0x7F)
    linked();                   /* the end */
```

7800basic

```
 fn_dev = FN_FUJI : fn_cmd = $F6 : fn_npr = 2
 gosub fn_beg
 fn_val = 30 : gosub fn_pb
 fn_val = 0 : gosub fn_pb
 gosub fn_go
 if FN_REPLY[0] = $7F then goto done
```

SET_DIRECTORY_POSITION / GET_DIRECTORY_POSITION — $E4 / $E5

DEVICE	$70	NPARAM	1 / 0
PARAMS	$E4: **one word**, the entry number		
PAYLOAD	none		
REPLY	$E5: a word, the current entry number; $E4: ACK		

Seek and tell for the open directory: paging in one command. The position is one two-byte parameter, not two one-byte ones.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSDPS, 1
        lda     #20             ; entry 20: the second page
        ldx     #0
        jsr     fn_pw
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (!fuji_set_directory_position(20))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $E4 : fn_npr = 1
gosub fn_beg
fn_val = 20 : fn_valhi = 0 : gosub fn_pw
gosub fn_go
```

CLOSE_DIRECTORY $F5

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Close what you open: the adapter has room for few open directories.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCCDIR, 0
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
fuji_close_directory();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $F5 : fn_npr = 0
gosub fn_beg
gosub fn_go
```

Files, prefixes and paths

COPY_FILE $D8

DEVICE	$70	NPARAM	2
PARAMS	bytes: source host, destination host, **counting from 1**		
PAYLOAD	`source|destination`; at its own length, or NUL-padded		
REPLY	ACK when the copy is done; the cartridge allows sixty seconds		

Host to host, without a byte passing through the console. Its hosts count from 1, unlike every other slot number; 0 is refused. When the destination ends in `/`,

the adapter adds the source's file name. fujinet-lib pads the payload to 256 bytes and CONFIG sends it at its own length; the firmware takes either.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCCOPY, 2
        lda     #1              ; host slot 0, counted from 1
        jsr     fn_pb
        lda     #1
        jsr     fn_pb
        ldy     #0
@spec:  lda     spec,y          ; source|destination
        beq     @go
        jsr     fn_txb
        iny
        bne     @spec
@go:    lda     #96
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a7800book/readme.txt|/a7800book/copy.txt");
if (!fuji_copy_file(1, 1, path))  /* hosts from 1 */
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $D8 : fn_npr = 2
gosub fn_beg
fn_val = 1 : gosub fn_pb : gosub fn_pb
last = spec_length - 1
for fn_i = 0 to last : FN_TX = spec[fn_i] : next
fn_tmo = 16
gosub fn_go
```

SET_HOST_PREFIX / GET_HOST_PREFIX — $E1 / $E0

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot; SET carries the prefix as its payload		
PAYLOAD	none		
REPLY	GET: the prefix		

A working directory per host, put in front of relative paths.

GET_DEVICE_FULLPATH $DA

DEVICE	$70	NPARAM	1
PARAMS	byte: the device slot		
PAYLOAD	none		
REPLY	the path SET_DEVICE_FULLPATH stored		

In C: `fuji_get_device_filename()`.

Appkeys

OPEN_APPKEY $DC

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	6 bytes: creator (word), app, key, mode (0 read, 1 write), and a reserved 0		
REPLY	ACK; NAK with no SD card, a creator of 0 or a bad mode		

Chooses the key the next read or write uses. The payload is one packed structure; leave off the reserved byte and the adapter waits for it until the transaction times out. On the SD card the key is the file `/FujiNet/CCCCAAKK.key`; it holds 64 bytes at most.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCAKOP, 0
        lda     #$00            ; creator $7800, low first
        jsr     fn_txb
        lda     #$78
        jsr     fn_txb
        lda     #$FF            ; app
        jsr     fn_txb
        lda     #0              ; key
        jsr     fn_txb
        lda     #AKMRD          ; mode
        jsr     fn_txb
        lda     #0              ; reserved, and required
        jsr     fn_txb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
fuji_set_appkey_details(0x7800, 0xFF, DEFAULT);
/* fuji_read_appkey() and fuji_write_appkey() each open the key */
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $DC : fn_npr = 0
gosub fn_beg
FN_TX = $00 : FN_TX = $78
FN_TX = $FF : FN_TX = 0
FN_TX = 0 : FN_TX = 0
gosub fn_go
```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY — $DD / $DE / $DB

DEVICE	$70	NPARAM	0 / 1 / 0
PARAMS	WRITE: byte, the length		
PAYLOAD	WRITE: the value, at most 64 bytes		
REPLY	READ: a word, the length, then the value; the others: ACK		

A write ends the selection, so each write needs its own OPEN in write mode. Section 13 has both directions in all three languages.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCAKRD, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY        ; the length; the value from FN_REPLY+2
        beq     done            ; empty
```

C (cc65 + fujinet-lib)

```
if (!fuji_read_appkey(0, &n, value))  /* value: the key + 2 */
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $DD : fn_npr = 0
gosub fn_beg
gosub fn_go
if FN_REPLY[0] = 0 then goto done
```

Utilities

GENERATE_GUID $BB

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	36 characters and a NUL		

A fresh, printable, unique id. RANDOM_NUMBER ($D3) is not answered on this build; the adapter's randomness reaches a program through a GUID or a hash.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCGUID, 0
        jsr     fn_go
        bne     failed
        lda     #6
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #36
        jsr     disp_rpl
```

C (cc65 + fujinet-lib)

```
if (!fuji_generate_guid(guid))
    failed();
```

7800basic

```
fn_dev = FN_FUJI : fn_cmd = $BB : fn_npr = 0
gosub fn_beg
gosub fn_go
for ix = 0 to 35 : buf[ix] = FN_REPLY[ix] : next
```

HASH: INPUT, COMPUTE, LENGTH, OUTPUT

$C8 / $C7 / $C6 / $C5

DEVICE	$70	NPARAM	1
PARAMS	INPUT: word, the byte count; COMPUTE: byte, the algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512); LENGTH and OUTPUT: byte, 1 for hex text, 0 for binary		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: the size; OUTPUT: the hash		

Four steps on the adapter: feed bytes with INPUT, as often as needed; COMPUTE; ask the LENGTH; read the OUTPUT. HASH_COMPUTE_NO_CLEAR ($C3) keeps the input for another algorithm, and HASH_CLEAR ($C2) empties it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCHSHI, 1   ; feed it
        lda     #dataend-data
        ldx     #0
        jsr     fn_pw
        ldy     #0
@in:    lda     data,y
        jsr     fn_txb
        iny
        cpy     #dataend-data
        bne     @in
        jsr     fn_go
        FNBEGIN FNDEVF, FNCHSHC, 1   ; SHA-256
        lda     #HASHA256
        jsr     fn_pb
        jsr     fn_go
        FNBEGIN FNDEVF, FNCHSHO, 1   ; read it, as hex
        lda     #1
        jsr     fn_pb
        jsr     fn_go           ; 64 characters at FN_REPLY
```

C (cc65 + fujinet-lib)

```
if (!fuji_hash_data(SHA256, (uint8_t *)data, 5, true, buf))
    failed();                    /* 64 hex characters */
```

```
fn_dev = FN_FUJI : fn_cmd = $C8 : fn_npr = 1
gosub fn_beg
fn_val = 10 : fn_valhi = 0 : gosub fn_pw
for ix = 0 to 9 : FN_TX = value[ix] : next
gosub fn_go
fn_cmd = $C7 : gosub fn_beg
fn_val = 3 : gosub fn_pb
gosub fn_go
fn_cmd = $C5 : gosub fn_beg
fn_val = 1 : gosub fn_pb
gosub fn_go
```

BASE64: ENCODE and DECODE — $D0--$CD / $CC--$C9

DEVICE	$70	NPARAM	varies
PARAMS	INPUT: word, the byte count; OUTPUT: word, how many bytes to read		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the result		

The same four steps as the hash. Encoding is INPUT `$D0`, COMPUTE `$CF`, LENGTH `$CE`, OUTPUT `$CD`; decoding is `$CC`, `$CB`, `$CA`, `$C9`. In C, `fuji_base64_encode_input()` and its seven siblings.

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT — $BC / $BD / $BE / $BF

DEVICE	$70	NPARAM	varies
PARAMS	INPUT: word, the count; ENCODE: version (0 to choose), error correction (0–3), shorten; LENGTH: the output mode (0 bytes, 2 bitmap); OUTPUT: word, the count		
PAYLOAD	INPUT: the text		
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the modules		

The adapter draws a QR code's matrix; a program only has to plot it. A version 1 code is 21 × 21 modules.

Answered with a NAK on this build

These are in `fujiCommandID.h` but fall through every table the RS-232 build dispatches, so they earn a NAK: `ENABLE_UDPSTREAM` (`$F0`), `SET_BAUDRATE` (`$EB`), `NEW_DISK` (`$E7`), `UNMOUNT_HOST` (`$E6`), `SET_HSIO_INDEX` (`$E3`), `SET_SIO_EXTERNAL_CLOCK` (`$DF`), `ENABLE_DEVICE` and `DISABLE_DEVICE` (`$D5`, `$D4`), `RANDOM_NUMBER` (`$D3`), `GET_TIME` (`$D2`; use the clock device), `DEVICE_ENABLE_STATUS` (`$D1`), `GET_HEAP` (`$C1`),

`QRCODE_CLEAR` (`$BA`), `GET_DEVICE1..10_FULLPATH` (`$A0`–`$A9`), `UPDATE_FIRMWARE` (`$90`) and `HSIO_INDEX` (`$3F`). Most tune other computers' buses.

17 Command Reference: The Network Device

Every command devices `$71`–`$78` answer, checked against the network device every FujiNet shares, `NDevice.cpp`. Most of the codes are the ASCII letter of their name — `'O'`pen, `'R'`ead, `'W'`rite — a habit from the Atari 8-bits. What a devicespec can say, and how each protocol behaves, belongs to The FujiNet Network Protocol Handbook; these are the cards. A READ's answer must fit the 1K reply window, and a WRITE's bytes the 320-byte TX stream with the parameter in front of them.

The assembly examples use `netdefs.inc`'s names; the C examples, fujinet-lib's network calls, which take the devicespec every time; the BASIC examples, `FN_NET` (`$71`). Each network command may take the cartridge 90 seconds, so the examples raise the timeout.

The lifecycle

NET_OPEN — $4F 'O'

DEVICE	$71	NPARAM	2
PARAMS	bytes: mode (4 read, 8 write, 12 read and write, 13 POST), translation (0 none, 1 CR, 2 LF, 3 CR LF)		
PAYLOAD	the devicespec, `N:` and the URL		
REPLY	ACK once the connection is set up		

On HTTP, modes 4 and 12 are both GET; 12 also lets a program set headers (NET_SET_HTTP_MODE). The request itself waits for the first STATUS or READ. Other HTTP modes: 14 PUT with headers, 5 and 9 DELETE. Opening a channel that is already open closes the old connection first.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCOPEN, 2
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldy     #0
@url:   lda     url,y           ; the devicespec, as long as it is
        beq     @go
        jsr     fn_txb
        iny
        bne     @url
@go:    lda     #130
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
    failed();
```

7800basic

```
fn_dev = FN_NET : fn_cmd = $4F : fn_npr = 2
gosub fn_beg
fn_val = 12 : gosub fn_pb
fn_val = 0 : gosub fn_pb
last = url_length - 1
for ix = 0 to last : FN_TX = url[ix] : next
fn_tmo = 22
gosub fn_go
```

NET_STATUS

$53 'S'

DEVICE	$71	NPARAM	2
PARAMS	two bytes of 0		
PAYLOAD	none		
REPLY	4 bytes: bytes waiting (word), connected, status (1 good, 136 end of file)		

The heartbeat. A terminal watches **connected***; a program that fetches and shows watches the status for end of file; every program reads the count, until two readings agree (Section 11).*

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCSTAT, 2
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        lda     #130
        sta     FN_TMO
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+NSAVLO ; bytes waiting
        ora     FN_REPLY+NSAVHI
        beq     done            ; nothing yet
```

C (cc65 + fujinet-lib)

```
if (network_status(url, &n, &count, &err) != FN_ERR_OK)
    failed();                    /* n waiting, count connected */
```

7800basic

```
fn_dev = FN_NET : fn_cmd = $53 : fn_npr = 2
gosub fn_beg
fn_val = 0 : gosub fn_pb : gosub fn_pb
fn_tmo = 22
gosub fn_go
avail = FN_REPLY[0]
```

NET_READ $52 'R'

DEVICE	$71	NPARAM	1
PARAMS	**one word**: how many bytes, no more than are waiting and no more than 1024		
PAYLOAD	none		
REPLY	that many bytes		

A short or failed read is a NAK. Use the bytes, or copy them, before the next transaction paints over the window. fujinet-lib's `network_read()` does the STATUS loop itself and returns how many it read; `network_read_nb()` reads only what is waiting.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCREAD, 1
        lda     #<256           ; 256 bytes
        ldx     #>256
        jsr     fn_pw
        lda     #130
        sta     FN_TMO
        jsr     fn_go           ; the bytes at FN_REPLY, FN_RXLEN of them
```

C (cc65 + fujinet-lib)

```
got = network_read(url, buf, 256);
if (got < 0)
    failed();
```

7800basic

```
 fn_dev = FN_NET : fn_cmd = $52 : fn_npr = 1
 gosub fn_beg
 fn_val = avail : fn_valhi = 0 : gosub fn_pw
 fn_tmo = 22
 gosub fn_go
```

NET_WRITE

$57 'W'

DEVICE	$71	**NPARAM**	1
PARAMS	word: how many bytes		
PAYLOAD	that many bytes		
REPLY	ACK		

The count goes twice: as the parameter, and as the length of the payload. The two parameter bytes and the count leave 317 bytes for the data in one transaction.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCWRITE, 1
        lda     #dataend-data
        ldx     #0
        jsr     fn_pw
        ldy     #0
@w:     lda     data,y
        jsr     fn_txb
        iny
        cpy     #dataend-data
        bne     @w
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (network_write(url, data, 5) != FN_ERR_OK)
    failed();
```

7800basic

```
fn_dev = FN_NET : fn_cmd = $57 : fn_npr = 1
gosub fn_beg
fn_val = value_length : fn_valhi = 0 : gosub fn_pw
last = value_length - 1
for ix = 0 to last : FN_TX = value[ix] : next
gosub fn_go
```

NET_CLOSE $43 'C'

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Close every connection you open, including on the way out of a failure; the adapter keeps it open otherwise, even across a power cycle of the console.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCCLOSE, 0
        lda     #130
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
network_close(url);
```

7800basic

```
fn_dev = FN_NET : fn_cmd = $43 : fn_npr = 0
gosub fn_beg
fn_tmo = 22
gosub fn_go
```

Parsers, parameters and line endings

NET_SET_PARSER / NET_PARSE / NET_QUERY — $FC / $50 'P' / $51 'Q'

DEVICE	$71	NPARAM	2 / 0 / 0
PARAMS	SET_PARSER: byte (any), then the parser: 0 none, 1 JSON, 2 HTML, 3 XML		
PAYLOAD	QUERY: the path, such as `/0/name`		
REPLY	QUERY: ACK; then STATUS and READ the value as text		

Switch an open connection to one of the adapter's parsers (the parser is the **second** parameter), parse the body, then ask for one value by its path. fujinet-lib's `network_json_parse()` sets the parser and parses in one call, and `network_json_query()` does the query, STATUS and READ.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCPRSR, 2   ; JSON
        lda     #0
        jsr     fn_pb
        lda     #NPJSON
        jsr     fn_pb
        jsr     fn_go
        FNBEGIN NETDEV, NCPARSE, 0  ; parse what was fetched
        jsr     fn_go
        FNBEGIN NETDEV, NCQUERY, 0  ; ask for one value
        ldy     #0
@q:     lda     query,y
        beq     @go
        jsr     fn_txb
        iny
        bne     @q
@go:    jsr     fn_go           ; then STATUS and READ it
```

C (cc65 + fujinet-lib)

```
if (network_json_parse(url) != FN_ERR_OK)
    failed();
got = network_json_query(url, query, (char *)buf);
if (got < 0)
    failed();
```

7800basic

```
fn_dev = FN_NET : fn_cmd = $FC : fn_npr = 2
gosub fn_beg
fn_val = 0 : gosub fn_pb
fn_val = 1 : gosub fn_pb
gosub fn_go
fn_cmd = $50 : fn_npr = 0 : gosub fn_beg
gosub fn_go
fn_cmd = $51 : gosub fn_beg
last = query_length - 1
for ix = 0 to last : FN_TX = query[ix] : next
gosub fn_go
```

NET_SET_PARAMETER $FB

DEVICE	$71	NPARAM	2
PARAMS	bytes: which (0 the query flags, 1 the line ending), the value		
PAYLOAD	none		
REPLY	ACK		

How the parser answers a query: flags remap characters and choose the output form; the line ending is the byte that ends each value. In C: `network_set_query_param()` and `network_set_line_ending()`.

NET_SET_HTTP_MODE $4D 'M'

DEVICE	$71	NPARAM	2
PARAMS	byte (any), then the mode: 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 POST data		
PAYLOAD	none		
REPLY	ACK; NAK on a connection that is not HTTP		

HTTP only. In mode 3 each WRITE sets a request header; in mode 4 a WRITE supplies the body of a POST; in mode 2 a READ returns the headers collected in mode 1. Mode 0 is the page itself. Open with mode 12 to use it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCHTTPM, 2  ; from now, WRITE sets headers
        lda     #0
        jsr     fn_pb
        lda     #3
        jsr     fn_pb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
network_http_start_add_headers(url);
network_http_add_header(url, "Accept: text/plain");
network_http_end_add_headers(url);
```

7800basic

```
 fn_dev = FN_NET : fn_cmd = $4D : fn_npr = 2
 gosub fn_beg
 fn_val = 0 : gosub fn_pb
 fn_val = 3 : gosub fn_pb
 gosub fn_go
```

NET_SEEK / NET_TELL — $25 '%' / $26 '&'

DEVICE	$71	NPARAM	1 / 0
PARAMS	SEEK: **one four-byte parameter**, the offset		
PAYLOAD	none		
REPLY	TELL: 4 bytes, the position		

Random access where the protocol has it: TNFS and other files, HTTP ranges. In C: `network_seek()` and `network_tell()`.

NET_TRANSLATION / NET_SET_EOL / NET_SET_INT_RATE — $54 'T' / $4C 'L' / $5A 'Z'

DEVICE	$71	NPARAM	2 / 0 / 1
PARAMS	TRANSLATION: byte (any), then the code; SET_INT_RATE: the rate		
PAYLOAD	SET_EOL: the line ending, as a string; empty restores the default		
REPLY	ACK		

Line-ending translation and terminators after OPEN. SET_INT_RATE paces an interrupt line on computers that have one; the cartridge has no such line, so the command is accepted and does nothing. Polling ACKSEQ is the only way a 7800 program learns a transaction is done.

RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR — $20 / $21 / $23 / $24 / $2A / $2B

DEVICE	$71	NPARAM	2
PARAMS	bytes: mode (4), and 0		
PAYLOAD	the devicespec; RENAME: `N:…/old,new`		
REPLY	ACK		

File operations on the protocols that have files: TNFS, FTP, SMB and the SD card. Each one opens a connection just for itself and closes it again. In C: `network_fs_rename()` and the rest.

NET_CHDIR / NET_GETCWD — $2C / $30

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	CHDIR: the path: absolute, relative, `..`, or a whole new `SCHEME://host/`		
REPLY	GETCWD: the prefix		

A working directory for the channel, put in front of every devicespec that does not name its own host.

NET_ACCEPT / NET_CLOSE_CLIENT — $41 'A' / $63 'c'

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

For a TCP server: a channel opened on `N:TCP://:port/` listens; ACCEPT takes the waiting client, and CLOSE_CLIENT hangs up on that client while the channel keeps listening. A 7800 can be the bulletin board.

NET_GET_REMOTE / NET_SET_DESTINATION / NET_USERNAME / NET_PASSWORD — $72 'r' / $44 'D' / $FD / $FE

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	SET_DESTINATION: `host:port`; USERNAME and PASSWORD: the credential		
REPLY	GET_REMOTE: who sent the last UDP datagram		

UDP's "who said that" and "talk to them instead", and credentials for protocols that log in (FTP, SMB and others). Send the credentials before the OPEN.

Answered with a NAK on this build

In `fujiCommandID.h` but not in the network device's table: `NET_GET_DSTATS_VALUE` (`$FF`), `NET_SET_UNIT` (`$FA`), `NET_SET_HSIO_INDEX` (`$E3`), `NET_QUERY_ALT` and `NET_PARSE_ALT` (`$81`, `$80`), `NET_GET_ERROR` (`$45`; STATUS's fourth byte says what went wrong), and `NET_HSIO_INDEX` (`$3F`).

18 Command Reference: Disk, Printer, Clock, Modem

The mailbox is a transport, not a translator: whatever byte goes into the DEVICE register goes onto the wire. Beyond the FUJI and network devices, the FujiNet answers four more for a 7800: disk drives, a printer, a clock and a Hayes-style modem. The clock is exercised by a program in this handbook (Section 13), and its replies below were measured on the bench. The others are reachable, transcribed from the firmware's handlers, and offered as such.

Disk: devices $31–$38

Eight virtual drives, $31 plus the drive number, each serving the image mounted in its device slot. The sector size is the image's. On the 7800 a mount also pushes the image into the cartridge (Section 12); reading it as a disk as well is possible, and rarely useful.

DISK_READ / DISK_WRITE / DISK_PUT $52 / $57 / $50

DEVICE	$31	NPARAM	1
PARAMS	the sector number, as one parameter (two or four bytes)		
PAYLOAD	WRITE and PUT: one sector		
REPLY	READ: one sector; WRITE and PUT: ACK		

With nothing mounted, READ answers NAK. WRITE verifies what it wrote; PUT does not.

6502 ASSEMBLY (ca65)

```
        FNBEGIN DSKDEV, $52, 1  ; DISK_READ, drive 1
        lda     #1              ; sector 1
        ldx     #0
        jsr     fn_pw
        jsr     fn_go           ; the sector at FN_REPLY
```

C (cc65 + fujinet-lib)

```
if (!fuji_bus_call(0x31, 0x52, FUJI_FIELD_B12 | FUJI_FIELD_REPLY,
                   1, 0, 0, 0, buf, 512))
    failed();
```

7800basic

```
fn_dev = $31 : fn_cmd = $52 : fn_npr = 1
gosub fn_beg
fn_val = 1 : fn_valhi = 0 : gosub fn_pw
gosub fn_go
```

DISK_STATUS $53

DEVICE	$31	NPARAM	0 or 1
PARAMS	optional byte: the kind of status		
PAYLOAD	none		
REPLY	4 bytes: drive status, controller status, format timeout, 0		

The Atari-style status block, from whatever image is mounted.

DISK_FORMAT / FORMAT_MEDIUM / PERCOM_READ / PERCOM_WRITE $21 / $22 / $4E / $4F

DEVICE	$31	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	FORMAT: a map of bad sectors; PERCOM_READ: a 12-byte geometry block		

Atari-drive commands the RS-232 build keeps; on an image they do little.

Printer: device $40

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS $57 / $50 / $53

DEVICE	$40	NPARAM	0 / 0 / 1
PARAMS	STATUS: byte, required but ignored		
PAYLOAD	WRITE and PUT: the bytes to print, fewer than 255		
REPLY	WRITE and PUT: ACK; STATUS: 4 bytes		

The page goes to a file on the adapter, in the printer it is set to imitate, and comes out through its web interface. A printer switched off in the adapter's settings does not answer at all, and the cartridge reports a timeout.

6502 ASSEMBLY (ca65)

```
        FNBEGIN PRNDEV, $57, 0  ; PRINTER_WRITE
        ldy     #0
@p:     lda     data,y
        jsr     fn_txb
        iny
        cpy     #dataend-data
        bne     @p
        lda     #13             ; a carriage return
        jsr     fn_txb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (!fuji_bus_call(0x40, 0x57, FUJI_FIELD_DATA,
                   0, 0, 0, 0, data, 5))
    failed();
```

7800basic

```
fn_dev = $40 : fn_cmd = $57 : fn_npr = 0
gosub fn_beg
last = value_length - 1
for ix = 0 to last : FN_TX = value[ix] : next
FN_TX = 13
gosub fn_go
```

Clock: device $45

On by default. A read with a parameter of 1 uses the alternate time zone instead of the adapter's own. The lengths below are what the bench's `fujinet-pc` answered.

GETTIME / GET_ISO_LOCAL / GET_ISO_UTC — $93 / $49 'I' / $5A 'Z'

DEVICE	$45	NPARAM	0 or 1
PARAMS	optional byte: 1 for the alternate zone		
PAYLOAD	none		
REPLY	GETTIME: 6 bytes: day, month, year (two digits), hour, minute, second. The ISO forms: `2026-10-20T16:14:12+0000` and a NUL, 25 bytes		

Section 13 has the ISO forms in all three languages. GET_ISO_UTC ignores the alternate zone.

6502 ASSEMBLY (ca65)

```
        FNBEGIN CLKDEV, CKGTIME, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+3      ; the hour
        sta     cnt
```

C (cc65 + fujinet-lib)

```
if (clock_get_time(when, APETIME_BINARY))
    failed();                   /* when[3]: the hour */
```

7800basic

```
 fn_dev = FN_CLOCK : fn_cmd = $93 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 cnt = FN_REPLY[3] : rem the hour
```

The other formats — `$54 $50 $53 $4D $47 $4C $9A`

DEVICE	$45	NPARAM	0 or 1
PARAMS	none		
PAYLOAD	none		
REPLY	`$54 'T'`: 7 bytes, century, year, month, day, hour, minute, second; `$50 'P'`: 4 bytes, ProDOS; `$53 'S'`: 19 characters, Apple III SOS; `$4D 'M'`: the 7 bytes and hundredths; `$47 'G'`: the zone's name, NUL-terminated; `$4C 'L'`: one byte, its length; `$9A`: GETTIME in the alternate zone		

Every format fujinet-lib's `clock_get_time()` offers, plus the zone. `$54` doubles as a write command on buses that have one; on this one it only reads.

SETTZ_ALT / SETTZ — `$74 't' / $99`

DEVICE	$45	NPARAM	0
PARAMS	none		
PAYLOAD	a POSIX time-zone string, such as `CST6CDT`		
REPLY	ACK		

`$74` sets the zone the adapter uses for local time; `$99` sets the **alternate** zone that a read with a parameter of 1 uses. In C, `clock_set_tz()` sets the adapter's, and `clock_get_time_tz()` reads in another zone without changing it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN CLKDEV, $74, 0  ; the adapter's own zone
        ldy     #0
@z:     lda     tz,y
        beq     @go
        jsr     fn_txb
        iny
        bne     @z
@go:    jsr     fn_go
        jmp     done
tz:     .byte   "CST6CDT", 0
```

C (cc65 + fujinet-lib)

```
if (clock_set_tz("CST6CDT"))
    failed();
```

7800basic

```
fn_dev = FN_CLOCK : fn_cmd = $74 : fn_npr = 0
gosub fn_beg
FN_TX = $43 : FN_TX = $53 : FN_TX = $54 : FN_TX = $36
gosub fn_go
rem CST6: Central time, no daylight saving
```

Modem: device $50

A Hayes-style modem that dials TCP instead of telephone lines: `ATDT host:port` opens a connection, and `ATNET1` runs it through telnet negotiation. Ten packet commands set it up and poll it; the characters themselves flow through WRITE and STATUS.

MODEM_STATUS / MODEM_WRITE / MODEM_STREAM — $53 / $57 / $58

DEVICE	$50	NPARAM	1 / 0 / 0
PARAMS	STATUS: byte, required but ignored		
PAYLOAD	WRITE: an AT command line, or data once connected		
REPLY	STATUS: 2 bytes, the second the handshake lines (`$CD` connected with data waiting, `$CC` connected, `$00` idle); WRITE: ACK; STREAM: a 9-byte baud table		

The other seven set the modem up: CONTROL `$41`, CONFIGURE `$42`, SET_DUMP `$44`, LISTEN `$4C`, UNLISTEN `$4D`, BAUDRATELOCK `$4E`, AUTOANSWER `$4F`. Switched off in the adapter's settings, the modem does not answer.

19 Command Reference: The Cartridge Itself

Some of what a FujiNet program can ask for never leaves the cartridge: its registers, its one-shot operations, the boot block and loader, the POKEY, the High Score Cart. This section is all of it, from the cartridge's own `fuji_mailbox.h` and the shared `fuji_mailbox_proto.h`.

Registers

A store of V to `$0D00`+n sets register n. Registers `$00`–`$13` are the protocol's, shared by every FujiNet cartridge; `$14` and up are the 7800's.

Register	Address	Meaning
`$00` DEVICE	`$0D00`	FujiBus device of the next transaction.
`$01` CMD	`$0D01`	Its command.
`$02` NPARAM	`$0D02`	How many parameters the TX stream holds.
`$05` DATA_RST	`$0D05`	Any value: rewind the TX stream.
`$06` RXSLICE	`$0D06`	The reply slice shown; the 7800 has one, so 0.
`$10` SEQ	`$0D10`	Non-zero and not the last one: launch.
`$11` BOOTLOCK	`$0D11`	`$B5`: arm the staged image for the loader. Ignored until BOOT_STATE says READY.
`$12`, `$13` BOOTSEL	`$0D12`, `$0D13`	`$B5` then `$4A`: restart the RP2354B in its USB boot mode, to load new firmware. Two stores, so no single stray write can do it.
`$14` SLICE_ACK	`$0D14`	The loader's: the slice numbered LOAD_SEQ is copied.
`$15` TV	`$0D15`	0 NTSC, anything else PAL: a program's own measurement, for the cartridge to keep.
`$16` HSC	`$0D16`	The High Score Cart: 0 off, 1 on, 2 forget the ROM, 3 install the staged image as the ROM.

One-shot operations

Stores to `$0D80`–`$0DFF` are not registers; the address is the operation. Four do something, and only the loader should use them:

Address	Name	What it does
`$0DFE`	SWAP	Into LOAD with the armed image, or CONFIG if none is armed. The loader's entry at `$0600` does this.

Address	Name	What it does
$0DFD	GO	Switch to the loaded image now: the loader's last store, from zero page, before JMP ($FFFC).
$0DFC	CONFIG	Into LOAD with CONFIG, whatever is armed. The loader's entry at $0603.
$0DFB	GO_BIOS	Switch to the image when the BIOS is next mapped in: the BIOS hand-over.

A program leaves by jumping to the loader, never by storing to these itself:

6502 ASSEMBLY (ca65)

```
        sei                     ; back to CONFIG
        lda     #$7F            ; MARIA's DMA off
        sta     CTRL
        jmp     FN_LOADER_CONFIG
```

C (cc65 + fujinet-lib)

```
fuji_a7800_config();    /* back to CONFIG; does not return */
```

7800basic

```
 rem back to CONFIG
 asm
   sei
   lda #$7F
   sta CTRL
   jmp $0603
end
```

The status page

Everything the cartridge paints, $0C00–$0CFB. Section 4 introduced the first eleven.

Address	Name	Meaning
$0C00	ACKSEQ	The last sequence number answered. Painted last.
$0C01	STATUS	Bit 0: the link to the FujiNet is up.
$0C02	ERR	0 OK, 1 no link, 2 timed out, 3 bad frame, 4 too big.
$0C03	REPLY_CMD	$06 ACK, $15 NAK.
$0C04–$0C05	RXLEN	The reply's length.
$0C06	BOOT_STATE	0 idle, 1 receiving, 2 ready, $80 failed.
$0C07	BOOT_PCT	0–100.
$0C08	BOOT_ERR	1 too big, 2 truncated, 3 no layout fits, 4 store busy.

Address	Name	Meaning
$0C09–$0C0A	MAGIC	'F', 'N'.
$0C0B	PROTO_VER	1.
$0C0C	SLICE_ECHO	The reply slice shown.
$0C0D	LOAD_STATE	0 idle, 1 a slice is waiting, 2 all loaded.
$0C0E	LOAD_DST	High byte of where this slice goes.
$0C0F	LOAD_N	Slices in this load, divided by 4.
$0C10	LOAD_SEQ	Counts slices, 1–255.
$0C11	LOAD_PCT	0–100 across the whole load.
$0C12	MODE	0 BOOT, 1 LOAD, 2 GAME, 3 APP.
$0C14	MAPPER	The running image's board layout (Section 12).
$0C15	LINK	1 once the link to the FujiNet has been seen.
$0C16–$0C18	BOOT_GOT	Bytes of a push received.
$0C19–$0C1B	BOOT_TOT	Bytes in the push.
$0C1C	INPTCTRL	The cartridge's model of INPTCTRL.
$0C1D	INPT_LOCK	1 once something has locked it. Only a power cycle clears it.
$0C1E	TV	0 NTSC, 1 PAL.
$0C1F	STAGED	The pushed image: bit 0 ready, bit 1 a FujiNet app, bit 2 this console's BIOS accepts it, bit 3 an HSC ROM.
$0C20	STAGED_KIND	Its board layout.
$0C21–$0C24	STAGED_CRC	Its CRC-32, low byte first.
$0C25	HANDOVER	0 none yet, 1 by the BIOS, 2 by the loader.
$0C26	HSC	Bit 0 ROM installed, bit 1 on, bit 2 last save reached the SD card, bit 3 scores unsaved.
$0C27	DIAG_RMW	How many read-modify-write dummy writes the cartridge has thrown away. Should be 0.

Power-on and the boot block

At power-on the cartridge serves a 4K block at $F000–$FFFF, signed so the console's BIOS will start it. The block checks for 'F', 'N' at $0C09 and jumps to the loader at $0600; if the magic is missing — the console cannot see the cartridge's pages below $1000 — it turns the screen red instead of crashing. The block is only ever served in BOOT mode.

The loader

The loader is 512 bytes of code the cartridge serves at `$0600`. Its two entries are `JMP`s, and neither returns:

- `$0600`: load the armed image, or CONFIG if nothing is armed, and start it.
- `$0603`: load CONFIG and start it.

It turns MARIA off, sets its own stack, and stores to SWAP or CONFIG. Then, for every 1K slice the cartridge offers, it copies the reply window to the page LOAD_DST names, in the 8K window at `$4000` the cartridge points at the right part of its RAM, and acknowledges with LOAD_SEQ into SLICE_ACK. When LOAD_STATE reads 2 it starts the image, by the BIOS or directly (Section 12), from a six-byte trampoline in zero page, so the arena can vanish beneath it. If the cartridge does not answer its first store within about two seconds, it turns the background yellow and keeps waiting: the console's stores below `$1000` are not reaching the cartridge.

The push

During MOUNT_IMAGE the FujiNet sends the image to the cartridge on device `$FF` as an OPEN (the stream — 0 the image, 1 a `.cfg` file whose `mapper=` line overrides the layout — and the size), WRITEs, and a CLOSE; a CLOSE carrying `$01` abandons it. The cartridge keeps the image in RAM (160K) or flash (512K), refuses one bigger than 448K, and reports through BOOT_STATE. A program never sends these; it only watches.

The POKEY

Every FujiNet app may play sound through a POKEY the cartridge serves at `$0450`–`$045F`, the address 7800 sound cartridges have always used. Its registers are the usual ones; the cartridge synthesises the sound and sends it to the console on the cartridge edge's audio line. Two differences from the chip: of the registers that read back, only RANDOM (`$045A`) is live (the rest read `$FF`), and it never raises an interrupt.

6502 ASSEMBLY (ca65)

```
POKEY   = $0450
        lda     #0              ; SKCTL: reset, then run
        sta     POKEY+$0F
        lda     #3
        sta     POKEY+$0F
        lda     #0
        sta     POKEY+$08       ; AUDCTL: 64 kHz clock
        lda     #$50
        sta     POKEY+$00       ; AUDF1: the pitch
        lda     #$A8
        sta     POKEY+$01       ; AUDC1: pure tone, volume 8
```

C (cc65 + fujinet-lib)

```
#define POKEY ((volatile uint8_t *)0x0450)

POKEY[0x0F] = 0;                /* SKCTL: reset, then run */
POKEY[0x0F] = 3;
POKEY[0x08] = 0;                /* AUDCTL: 64 kHz clock */
POKEY[0x00] = 0x50;             /* AUDF1: the pitch */
POKEY[0x01] = 0xA8;             /* AUDC1: pure tone, volume 8 */
```

7800basic

```
 rem AUDF1 and AUDC1 are already the TIA's names
 dim PKSKCTL = $045F
 dim PKAUDCTL = $0458
 dim PKAUDF1 = $0450
 dim PKAUDC1 = $0451
 PKSKCTL = 0
 PKSKCTL = 3
 PKAUDCTL = 0
 PKAUDF1 = $50
 PKAUDC1 = $A8
```

7800basic's own POKEY sound engine works too, with `set pokeysupport $450`; its TIA statements must not be used by a program that leaves INPTCTRL unlocked (Section 9).

The High Score Cart

Section 13 describes it; its controls are the HSC register above and the HSC byte of the status page. In C, `fuji_a7800_hsc()` and `fuji_a7800_hsc_op()`.

20 Battleship

Fuji Battleship is a networked game for up to four players, with bots to fill the empty seats, and it runs on every computer FujiNet serves. Its 7800 port is merged into the game's repository, and it is the best example there is of a C program on the FujiNet cartridge: one shared game, written once, and a thin 7800 layer underneath that draws, reads the stick, makes the noises, and talks to the cartridge through fujinet-lib.

This section walks through the port at the game's commit `87b8cc7`. Appendix D prints its 7800 files.

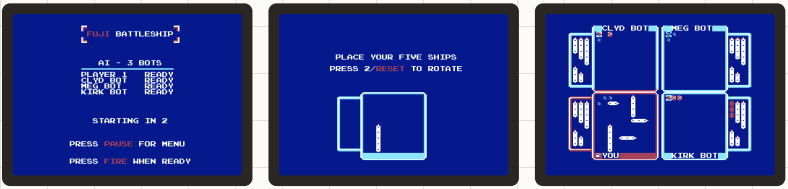


Figure 13. At a table with three bots, all ready; placing the five ships; a game under way, four boards, attacks marked in red.

Controls

Input	Does
Joystick	Moves the cursor.
Button 1	Ready, place, attack: the RETURN of the other ports.
Button 2 or RESET	Rotates a ship while placing.
PAUSE	The in-game menu.
SELECT with button 1, button 2, PAUSE	Help, sound on or off, quit to the lobby.

A 2600 joystick works too: its one button is button 1, and RESET stands in for button 2. The buttons reach the shared code as key codes behind `kbhit()` and `cgetc()`, exactly as a keyboard's would.

What is shared and what is the 7800's

File	Lines	What it does
`src/main.c`, `gamelogic.c`, `screens.c`, `misc.c`, `stateclient.c`	1846	The game: every port's. Screens, rules, the polling loop, the server calls.
`src/atari7800/graphics.c`	675	The screen, on the tile engine.
`src/atari7800/osk.c`	226	The on-screen keyboard for the player's name.
`src/atari7800/sound.c`	243	The sound effects, on the POKEY.
`src/atari7800/fujinet.c`	170	Every call to the cartridge.
`src/atari7800/input.c`, `util.c`	193	The stick, the switches, timing.
`src/atari7800/maria.s`, `maria.h`	607	The tile engine (Section 15), verbatim.
`src/atari7800/mkchr.py`	—	The tiles, from the MS-DOS client's art.

Talking to the cartridge

Everything that reaches the cartridge is in `fujinet.c`. On the 7800, most `fuji_*` calls are macros over the library's bus call, so the shared code's own declarations of them would link against nothing; the game is built with `CUSTOM_FUJINET_CALLS`, and the shared code calls three hooks this file supplies. The network hook is one HTTP request, start to finish:

C (cc65 + fujinet-lib) · fujinet.c

```c
int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    // Show what has been drawn (palette changes wait for a frame) before
    // the transaction holds the 6502.
    mt_sync();

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}
```

`mt_sync()` first shows whatever was just drawn, because the transaction holds the 6502 for as long as the server takes, and a palette change waits for a frame.

The URL and the state

The game never parses anything. The shared `apiCall()` builds the request — `n:`, the server, a path such as `state`, the player and table, and `bin=1` — and the server answers with the whole game as a C structure in binary:

C (cc65 + fujinet-lib) · stateclient.c

```
    strcpy(url, "n:");
    strcat(url, serverEndpoint);
    strcat(url, path);
    strcat(url, query);
    strcat(url, query[0] ? "&bin=1&v=" API_CLIENT_VERSION : "?bin=1&v=" API_CLIENT_VERSION);
```

The `Game` structure is 509 bytes: the prompt, whose turn it is, and each of four players' name, status, 100-square field and ships left. One READ brings all of it into the reply window and fujinet-lib copies it into `clientState`; a 1K window holds it with room to spare. The game polls this way every second or so and redraws what changed.

Which server

The FujiNet Lobby leaves the game a server and a table in appkey 1:1:5 (`http://server/?table=t`), and the player's name in 1:1:0. Without the Lobby, the game uses the public server. The game's own preferences live in appkey `$E41C`:5:0, and a first byte of `$FF` there points it at `http://127.0.0.1:8080/`, a server on the same computer. The screens above came from the server in `fujinet-game-system/battleship`, run locally with `go run .`, with the Lobby key set to its `ai3` table.

The board

Four players' boards fill the 32 × 24 tiles, a quadrant each, with a legend of ships at the sides. The art is the MS-DOS client's, converted by `mkchr.py`, which fits it into 128 tiles by letting the tile engine's two palettes do what the NES port did with extra sets of glyphs:

PYTHON · mkchr.py

```
   palette 0  1 white  2 red   3 white     text, active board chrome
   palette 1  1 red    2 cyan  3 white     red capitals, inactive chrome, art

The font draws in value 1, so the same glyph is white text in palette 0 and
red alternate text in palette 1. Chrome draws red as 2 and white as 3, so the
active (palette 0) and inactive (palette 1, red -> cyan) boards share tiles.
Other art is recoded to palette 1. A tile that is only white or blank looks
the same in both and is marked AGNOSTIC: drawing it leaves the cell's palette
alone. Name plates have no glyph set of their own: names are font text over
the plate tile.
```

The cost of a palette change is a run in the row's display list, and a row holds at most twelve (Section 15). So `graphics.c` keeps runs long: text sets its whole string to one palette, white-only art leaves a cell's palette alone, and `drawBoard()` turns the fields to the art palette before anything is drawn in them. The busiest rows in a four-player game have seven runs.

Sound

`sound.c` plays the same cues as the other ports through the cartridge's POKEY, never the TIA. Tones join channels 1 and 2 into one 16-bit channel clocked at 1.79 MHz, which reaches every note of the table exactly; explosions sweep channels 3 and 4 joined the same way, with a noise burst on channel 1.

Back to the Lobby

Quitting a game boots the FujiNet Lobby, the way CONFIG would: find the lobby's host among the host slots, mount `/atari7800/lobby.a78`, watch the push, and hand the console to the loader:

C (cc65 + fujinet-lib) · fujinet.c

```
    if (!fuji_mount_host_slot(slot)
        || !fuji_set_device_filename(LOBBY_MODE_READ, slot, LOBBY_DEVICE_SLOT, (char *)lobbyPath)
        || !fuji_mount_disk_image(LOBBY_DEVICE_SLOT, LOBBY_MODE_READ))
    {
        quitStatus("LOBBY NOT AVAILABLE");
        return;
    }

    // Some FujiNets push the image after answering MOUNT_IMAGE, so watch the
    // cart's own progress rather than trust the reply.
    for (;;)
    {
        state = fuji_a7800_boot_state();
        if (state == FUJI_A7800_BOOT_READY)
            break;
        if (state == FUJI_A7800_BOOT_FAILED || ++frames > LOBBY_FRAMES)
        {
            quitStatus("LOBBY LOAD FAILED");
            return;
        }
        if (fuji_a7800_boot_percent() != pct)
        {
            pct = fuji_a7800_boot_percent();
            utoa(pct, pctText, 10);
            strcat(pctText, "%");
            drawText(25, HEIGHT - 1, pctText);
        }
        mt_sync();
    }

    // Hand the console to the cartridge's loader. It does not return.
    fuji_a7800_boot();
}
```

Note: At the time of writing the Lobby has no 7800 client on its host, so "LOBBY NOT AVAILABLE" is what a player sees. The code is ready for the day it does.

Building and testing it

```
make PLATFORMS=atari7800 atari7800 FUJINET_LIB=~/fujinet-lib-experimental
```

writes `r2r/atari7800/fbs.a78`, running `mkchr.py` first. The game's `support/atari7800/smoke.lua` reads the screen out of the tile engine's map and drives the game with a script of button presses; `emu/bssmoke.lua` is a copy. A game server counts real seconds, so a run against one must not be sped up: the handbook's `emu/run.sh` takes `THROTTLE=1` for that.

```
THROTTLE=1 FBS_TILEMAP=support/atari7800/tilemap.lua \
  FBS_MAPFILE=r2r/atari7800/fbs.map \
  FBS_SCRIPT=wait2,fire,wait12,show,fire,fire,fire,fire,fire,wait6,show \
  emu/run.sh r2r/atari7800/fbs.a78 bssmoke
```

21 CONFIG

CONFIG is the program every FujiNet cartridge runs at power-on: the host list, the file browser, WiFi setup, and the boot. The 7800's is in fujinet-config's `atari7800/` directory, a 32K C program built exactly as Section 8 describes, and baked into the cartridge's firmware so it is there before any network is. It is the largest FujiNet program on the 7800, and every technique in this handbook is in it.



Figure 14. CONFIG: the host slots, the SD card's root, and the adapter information with the TV standard, INPTCTRL and the High Score Cart.

The screens

File	Lines	Screen
`st_hosts.c`	158	The eight host slots, renaming one, PLAY GAME LOBBY; also where a copy's destination is picked.
`st_files.c`	304	The file browser: pages, into and out of directories, the filter.
`st_copy.c`	124	Host-to-host copy: mark a file, pick a host and directory, copy.
`st_wifi.c`	206	Checking, scanning, choosing a network, a password, a hidden SSID.
`st_info.c`	106	The adapter, the TV standard, INPTCTRL's lock, the High Score Cart switch.
`st_boot.c`	212	Mounting into device slot 0, the progress bar, the hand-over; installing an HSC ROM.
`st_lobby.c`	80	Finding or adding the Lobby's host, mounting it, booting it.
`config.c`	152	The state machine that moves between them.

The support files are the same pieces this handbook has used: the tile engine (`maria.s`), a layer of frames and bars over it (`fujidisp.c`), the joystick and

switches with auto-repeat (`fujiin.c`), an on-screen keyboard (`fujiedit.c`), and POKEY sounds (`sfx.c`).

Screen	Button 1	Button 2 / RESET	SELECT / PAUSE
Hosts	open the host, or boot the Lobby	WiFi setup	rename / information
Files	open the directory, or boot the file	up a directory	filter / copy this file
WiFi	choose the network	back	— / scan again
Information	back	back	High Score Cart on or off

Drawn straight out of the reply

CONFIG keeps no copy of a host list or a directory page. It asks, and draws what comes back straight from the reply window into the tile map, `mt_text` taking each name from `$0800`. A screen of 20 names is 20 READ_DIR_ENTRY transactions; on the cartridge that is a fraction of a second. The habit comes from the CONFIGs for consoles with no RAM to spare, and it is why CONFIG fits.

fujiraw: when the library will not do

`fuji_bus_call()` takes its payload as one buffer. A few of CONFIG's commands want one assembled from pieces, and `fujiraw.c` sends those with the bus layer — `fn_regwr, fn_tx, fn_commit` — streaming each piece into the TX page and padding as it goes, so nothing is assembled in RAM first:

- OPEN_DIRECTORY with a filter: the path, a NUL, the pattern, padded to 256;
- WRITE_HOST_SLOTS: all eight slots, streamed back out of the reply window of a READ_HOST_SLOTS with the renamed one replaced;
- SET_DEVICE_FULLPATH: the directory from RAM, then the file name straight out of the reply window;
- SET_SSID's 33 and 64 bytes, and COPY_FILE's `source|destination`;
- MOUNT_IMAGE without waiting for its answer, so the progress bar can be drawn while the push arrives and the answer is checked when `ACKSEQ` comes round.

Everything else goes through fujinet-lib.

Building it into the cartridge

```
cd fujinet-config/atari7800
make FNLIB=~/fujinet-lib-experimental     # build/config.bin, build/config.a78
make install PICO_7800=~/fujinet-firmware/pico/atari-7800
cd ~/fujinet-firmware/pico/atari-7800
./build-cart.sh --bake                    # into the firmware, and into firmware/baked/
```

`build-cart.sh` turns `config.bin`, the loader and the signed boot block into headers the firmware compiles in, and builds the RP2354B firmware. `firmware/baked/` keeps the last baked copies, so a build with no cc65 at hand (the ESP32's, or CI's) still has a CONFIG.

The cartridge's own `emu/cfgtest.lua` drives this CONFIG in MAME through the hosts, a directory and a boot, and checks each screen. The screens above came from the same build, driven by the handbook's copy of Battleship's smoke script with an ASCII tile map.

22 Things That Cost Real Time

Every fact in this handbook was bought with time, and some of them were expensive. Here are the expensive ones, so you do not have to pay again.

The milestones

The cartridge was brought up in the FujiNet MAME model, one milestone at a time, each with a test that had to pass before the next began:

Milestone	What it proved	Exit test
M0	The boot block passes the BIOS's signature check, on NTSC and PAL; the loader runs; MARIA shows text.	`sigtest.lua`: the BIOS leaves by its success exit; a deliberately broken block fails.
M1	One real transaction, and the cartridge outliving a power cycle of the console.	`fujitest` shows the live SSID; `powertest.lua` requires the next sequence number to be 2, not 1.
M2	Booting over the network, every board layout.	`fujiboot` and `fujibank`; every image the database knows pushed and started.
Tier A	The cartridge serves each of 126 known games exactly as MAME's own cartridge does.	Frame-for-frame screenshots: 126 of 126.
Tier C	Each of the 126 pushed over the network, byte-exact in the RAM, started the right way.	126 of 126, and again with every hand-over forced direct.
POKEY	The synthesiser sounds like MAME's POKEY.	`pokeycmp.py`: 13 of 13 test steps; Ballblazer and Commando within 0.97 of MAME's loudness.
HSC	High scores survive a power cycle through the FujiNet's SD card.	`hscram.lua`: two MAME runs print the same HSC RAM.

The BIOS verdicts the cartridge carries came from the same bench: of the 126 images, 72 pass both consoles' signature checks, 51 only the PAL one, and 3 neither. Those three run only by the direct hand-over.

In the cartridge

- **MARIA reads the cartridge too fast for a microcontroller.** Its DMA windows are 280 to 420 nanoseconds. That ruled out serving game ROM

from the RP2354B and put a real SRAM on the edge, with the RP2354B only steering its upper address lines. It is also why nothing MARIA reads may live in the mailbox.

- **Only the first INPTCTRL lock counts.** A game writes TIA values with bit 0 set long after it locked the console; the cartridge's model of INPTCTRL must stop at the first lock, or it believes every game turned 2600.
- **MAME hides the BIOS's view of the cartridge.** Every mode change had to be triggered by a write, because MAME does not show the cartridge the BIOS's reads of `$FFFC`.
- **MAME does not emulate the NTSC console's fall back to 2600 mode,** so a broken signature can look fine in MAME. The negative test exists for that reason.
- **MAME runs an autoboot script again on every reset.** Harness state lives in `_G`, or it starts over halfway through.
- **Activision images** in the No-Intro set have their 8K halves the other way round from MAME's. The database carries a flag for them.

In this handbook

- **7800basic miscompiles two things without a word.** An expression in an array index (`A[ix + 2]`) and an expression in a `for` bound (`to n - 1`) both compile and both do the wrong thing. One extra byte in an appkey gave the second away. Section 9.
- **The vendored ITC Eras fonts report half their real cap height.** Typst set lines on top of one another until `tools/fixeras.py` wrote the true values in.
- **A smart quote after a digit is a prime**, which Eras does not have: `7800's` must be typed with its apostrophe already curly, or a fallback font creeps into the book.
- **A game server counts real seconds.** Battleship sat at "STARTING IN 2" for sixteen seconds of emulated time, because MAME was running fifty times faster than the server's clock. `THROTTLE=1`.
- **A comment can be out of date.** CONFIG's notes say COPY_FILE must go at its exact length; on today's firmware fujinet-lib's padded payload copies the file perfectly well. The test took a minute; the claim would have stood for years.
- **The clock's commands are not what their names say.** GETTIME answers six bytes; SETTZ sets the **alternate** zone. Measured on the bench, and Section 18 says so.

- **fujinet-lib's romstamp checks only 32K and 48K images.** A banked C program needs `tools/checkbanks.py`, which scans each bank where it lies in the file.
- **A font in a switched bank** turns to garbage while another bank is in. MAME will not show it, because a transaction there finishes inside one store; on a real cartridge the display runs while the program waits. Section 14.

23 Troubleshooting Checklist

The following list is supplied to help you troubleshoot any possible problems in getting a program to talk to the FujiNet. Always rebuild, and run the program on the bench, before checking anything else.

Building

Symptom

- `checkrom: … a store to $01 rewrites INPTCTRL.`

Probable Cause and Remedy

- The program stores to the TIA: its sound registers, its `WSYNC`, or INPTCTRL itself. Use the cartridge's POKEY at `$0450` and MARIA's `WSYNC` at `$24`, and leave INPTCTRL alone (Section 3).

Symptom

- A C program fails to link once it calls `clock()`, `cputs()` or another conio call.

Probable Cause and Remedy

- Those drag in cc65's own start-up code, which collides with fujinet-lib's. Use the tile engine for text and MARIA's frame count for time.

Symptom

- `romstamp: image is 65536 bytes: not whole KiB of at most 48K.`

Probable Cause and Remedy

- The program is banked. fujinet-lib's checker takes 48K at most; check a banked image with `tools/checkbanks.py` (Section 14).

Symptom

- dasm stops with `EQU: Value mismatch` on a 7800basic program.

Probable Cause and Remedy

- A variable has a one-letter name, which is one of 7800basic's own (`a`–`z`).
- A `dim` reuses a name 7800basic already has: `AUDF1` and `AUDC1` are the TIA's. Prefix the POKEY's names.

Symptom

- `basfix: … no room for the claim at $FF70.`

Probable Cause and Remedy

- 7800basic's runtime has grown past `$FF6F`, into the ten bytes the claim needs. Move code into banks, or out of the last 4K.

Symptom

- `basfix: … writes the old value first.`

Probable Cause and Remedy

- A read-modify-write on the mailbox: arithmetic on a mailbox name in BASIC, or `INC`, `DEC` or a shift on `$0D00`–`$0FFF` in assembly.

The mailbox

Symptom

- `fn_go` returns `$FD`, or `fuji_a7800_present()` is false.

Probable Cause and Remedy

- The magic `'F','N'` is missing at `$0C09`. The program is not on a FujiNet cartridge, or MAME was started without `-cartslot fujinet`.
- The image has no claim at `$FF70`, so the cartridge started it as a GAME and turned the mailbox off.

Symptom

- Every transaction after the first comes back `$FF`: no answer.

Probable Cause and Remedy

- The program keeps its own sequence number. Take the next one from `ACKSEQ` at `$0C00` every time (Section 4).

Symptom

- ERR is 2, timed out, on a network command.

Probable Cause and Remedy

- The server is slow to answer. The cartridge allows 90 seconds for OPEN, STATUS, READ and CLOSE; the program's own wait must be longer (Sections 4, 7, 9).
- The adapter cannot reach the server. Check the devicespec, and that the server is up.

Symptom

- ERR is 3, bad frame, before anything is sent.

Probable Cause and Remedy

- A parameter has a size other than 1, 2 or 4, or there are more than eight.

Symptom

- REPLY_CMD is `$15`, NAK.

Probable Cause and Remedy

- The command is not answered on this build (the lists at the ends of Sections 16 and 17).
- A parameter has the wrong shape: two one-byte parameters where one two-byte parameter belongs, as SET_DIRECTORY_POSITION's.
- A required parameter is missing: SET_SSID's, STATUS's.

Symptom

- OPEN_APPKEY times out.

Probable Cause and Remedy

- The payload is missing its sixth, reserved byte, and the adapter is still waiting for it.

Symptom

- `DIAG_RMW` at `$0C27` is not 0.

Probable Cause and Remedy

- Something performed a read-modify-write on a write page. Find it: `checkrom.py` and `basfix.py` name the address.

The display

Symptom

- Text from a reply shows as garbage.

Probable Cause and Remedy

- MARIA is reading the mailbox. Copy the reply into RAM first (Section 5).

Symptom

- A 7800basic program copies a reply and shows nothing, or the wrong bytes.

Probable Cause and Remedy

- An expression inside an array index, such as `FN_REPLY[ix + 2]`. Give the field its own `dim` at its address (Section 9).

Symptom

- A 7800basic loop copies one byte too many.

Probable Cause and Remedy

- An expression in the `for` bound. Compute the bound into a variable first.

Symptom

- The picture breaks up only while a banked program waits for the FujiNet.

Probable Cause and Remedy

- The font or tiles are in a switched bank. Put everything MARIA reads in the fixed bank (Section 14).

Symptom

- Colours go wrong part of the way across a row on the tile engine.

Probable Cause and Remedy

- The row has more than twelve palette runs. Keep a string, and the cells around it, in one palette (Section 15).

The network

Symptom

- STATUS says 0 bytes waiting, forever.

Probable Cause and Remedy

- The server never answered. Check the devicespec and the server.
- On the bench: the fixture server is not running on port 8765.

Symptom

- READ comes back NAK.

Probable Cause and Remedy

- It asked for more than STATUS said was waiting, or for more than 1024 bytes.

Symptom

- A second OPEN on the same channel gets the first one's data.

Probable Cause and Remedy

- The first was never closed. Close every connection, including on the way out of a failure.

Booting

Symptom

- BOOT_STATE reads `$80`, failed.

Probable Cause and Remedy

- BOOT_ERR says why: 1 the image is bigger than 448K; 2 it arrived short; 3 no board layout fits it (add a `.cfg` beside it with a `mapper=` line); 4 the only store big enough is the one running the program.

Symptom

- The screen turns yellow after a boot and stays yellow.

Probable Cause and Remedy

- The loader's stores below `$1000` are not reaching the cartridge. On real hardware this is the first thing to check (Section 1).

Symptom

- The screen is red at power-on.

Probable Cause and Remedy

- The boot block cannot see the cartridge's magic at `$0C09`: the same fault, earlier.

Symptom

- A game boots, but not through the console's BIOS.

Probable Cause and Remedy

- INPTCTRL is locked: 7800basic locks it at start-up unless told not to (Section 9).
- The image is a FujiNet app, which always starts directly.
- This console's BIOS is not known to accept the image (STAGED bit 2).

Symptom

- A banked FujiNet app runs as garbage.

Probable Cause and Remedy

- The cartridge took it for a plain 64K image. Name the bank scheme in the claim's kind byte, or in an `.a78` header.

The bench

Symptom

- MAME ignores `-autoboot_script` and nothing is checked.

Probable Cause and Remedy

- MAME was not run from its own tree. `emu/run.sh` changes there first.

Symptom

- The cartridge model never connects to the FujiNet.

Probable Cause and Remedy

- `fujinet-pc` is not listening on the address in `FUJINET_TCP` (127.0.0.1:9995 by default).
- Another MAME is still connected; a listener takes one client. `emu/run.sh` stops a stale one first.

Symptom

- A networked game stops at a countdown.

Probable Cause and Remedy

- The server counts real seconds and MAME is running unthrottled. Use `THROTTLE=1`.

Symptom

- MAME will not start headless.

Probable Cause and Remedy

- Set `SDL_VIDEODRIVER=dummy`.

24 Parts List

Every address, register, command and listing in this handbook was transcribed from these, at these revisions, and every program was built and run against them. When ordering repair parts, always include the part number.

The cartridge

Mailbox, loader, boot block, mappers, POKEY, HSC.	`fujinet-firmware pico/atari-7800 @ 07e9bbaa9`
Shared mailbox protocol and fujimail.	`fujinet-firmware pico/common @ 07e9bbaa9`
Client library, start-up, text, test programs.	`pico/atari-7800/testrom @ 07e9bbaa9`
Checkers: checkrom.py, mka78.py, mkfont.py.	`pico/atari-7800/tools @ 07e9bbaa9`
MAME cartridge model.	`pico/atari-7800/emu @ 07e9bbaa9`
Board.	`fujinet-hardware ATARI-7800/FujiNet-7800-Rev0`

The adapter

Command dispatch: FUJI device and mixins.	`lib/device/rs232, fujiDevice @ 07e9bbaa9`
Network device.	`lib/device/NDevice @ 07e9bbaa9`
Clock.	`lib/device/fujiClock @ 07e9bbaa9`
fujinet-pc on the bench.	`v1.6-5501c94a, RS232 build`
Bus-over-IP.	`127.0.0.1:9986`

The programs

fujinet-lib, atari7800 target.	`fujinet-lib-experimental add-atari7800 @ 1ebf9d0 (PR 152)`
Tile engine, CONFIG.	`fujinet-config atari7800 @ f38ed1d`
Battleship.	`fujinet-battleship @ 87b8cc7`
Battleship server.	`fujinet-game-system/battleship @ 19770c3`
This handbook's programs.	`listings/asm, listings/c, listings/bas`

The tools

cc65: ca65, ld65, cc65, cl65.	`V2.19, 92fa7cb23`
7800basic.	`v0.41, e7c18d2`

dasm	`2.20.16`
MAME, a7800 driver with the cartridge	`533800328a2`
Typst	`0.14.2`
ITC Eras Std, Source Code Pro	`fonts/`

Further reading

Every devicespec and protocol	`The FujiNet Network Protocol Handbook`
MARIA, the DLL and display lists	`Atari 7800 Software Guide`
7800basic's statements	`randomterrain.com/7800basic.html`
Style reference	`ATARI 7800 ProSystem Owner's Manual, 1986`

A Mailbox Quick Reference

The map

Address	Access	What
$0450–$045F	write; RANDOM read	POKEY (APP mode, and games that have one)
$0600	JMP	Loader: load the armed image (or CONFIG) and start it
$0603	JMP	Loader: load CONFIG and start it
$0800–$0BFF	read	Reply window, 1K
$0C00–$0CFB	read	Status page
$0D00–$0D7F	write	Registers: store V at $0D00+n
$0D80–$0DFF	write	One-shot operations (the loader's)
$0F00–$0FFF	write	TX stream: each store appends its byte
$1000–$17FF	read, write	High Score Cart RAM, when on
$3000–$3FFF	read	High Score Cart ROM, when on
$FF70–$FF79	in the image	The claim: "FUJI", version 1, kind+1, flags, 0, 0, 0

Status page

Address	Name	Address	Name
$0C00	ACKSEQ	$0C12	MODE
$0C01	STATUS	$0C14	MAPPER
$0C02	ERR	$0C15	LINK
$0C03	REPLY_CMD	$0C16–18	BOOT_GOT
$0C04–05	RXLEN	$0C19–1B	BOOT_TOT
$0C06	BOOT_STATE	$0C1C	INPTCTRL
$0C07	BOOT_PCT	$0C1D	INPT_LOCK
$0C08	BOOT_ERR	$0C1E	TV
$0C09–0A	MAGIC 'F','N'	$0C1F	STAGED
$0C0B	PROTO_VER	$0C20	STAGED_KIND
$0C0C	SLICE_ECHO	$0C21–24	STAGED_CRC
$0C0D	LOAD_STATE	$0C25	HANDOVER
$0C0E	LOAD_DST	$0C26	HSC
$0C0F–11	LOAD_N, SEQ, PCT	$0C27	DIAG_RMW

Registers and one-shots

Store to	Register	Store to	Register
$0D00	DEVICE	$0D12, $0D13	BOOTSEL ($B5, $4A)
$0D01	CMD	$0D14	SLICE_ACK
$0D02	NPARAM	$0D15	TV
$0D05	DATA_RST	$0D16	HSC
$0D06	RXSLICE	$0DFB	GO_BIOS
$0D10	SEQ	$0DFC	CONFIG
$0D11	BOOTLOCK ($B5)	$0DFD, $0DFE	GO, SWAP

The transaction

1. DEVICE, CMD, NPARAM; DATA_RST.
2. Parameters into $0F00: size (1, 2, 4), then the value, low first. Then the payload. 320 bytes, eight parameters at most.
3. SEQ = ACKSEQ + 1; 0 becomes 1.
4. Wait for ACKSEQ = SEQ.
5. ERR 0? REPLY_CMD $06? The reply is at $0800, RXLEN bytes.

The names, in each language

What	Assembly	C	7800basic
Reply	FN_REPLY	FN_REPLY[i]	FN_REPLY[ix]
ACKSEQ	FN_ACKSEQ	FN_ACKSEQ	FN_ACKSEQ
ERR	FN_ERR	FN_ERRCODE	FN_ERR
REPLY_CMD	FN_REPLYCMD	FN_REPLYCMD	FN_REPLYCMD
A register	sta FN_REGSEL+FN_REG_CMD	fn_regwr(FNR_CMD, v)	FN_CMD = v
TX	sta FN_TXDATA, fn_txb	fn_tx(b)	FN_TX = b
Launch and wait	fn_go	fn_commit()	gosub fn_go

The assembly library's zero page

Address	Name	Address	Name
$E0	FN_SEQ	$E5	FN_TMO
$E1	FN_CNT	$E6–E7	FN_PTR
$E2–E4	FN_DEV, FN_CMD, FN_NPR	$E8–EB	DISP_PTR, DISP_TMP

B Error and Status Codes

Where	Value	Meaning
ERR `$0C02`	0	The transaction completed.
	1	No link to the FujiNet.
	2	The FujiNet did not answer in time.
	3	Bad frame: a malformed request or reply.
	4	Too big for the TX stream or the reply window.
REPLY_CMD `$0C03`	`$06`	ACK: the FujiNet said yes.
	`$15`	NAK: it said no, or does not know the command.
	0	Nothing came back.
Assembly library (A)	0	Success.
	`$FF`	No answer: `fn_go` gave up.
	`$EE`	NAK, from `fn_ack`.
	`$FD`	No cartridge: the magic is missing.
7800basic library (`fn_res`)	0, `$FF`, `$EE`, `$FD`	The same.
fujinet-lib (`fn_device_error`)	1–4	The cartridge's ERR.
	`$FF`	No answer.
	144	NAK.
BOOT_STATE `$0C06`	0, 1, 2, `$80`	Idle, receiving, ready, failed.
BOOT_ERR `$0C08`	1	The image is bigger than 448K.
	2	Truncated: it arrived short.
	3	No board layout fits it.
	4	The only store that fits is in use.
LOAD_STATE `$0C0D`	0, 1, 2	Idle, a slice is waiting, done.
MODE `$0C12`	0, 1, 2, 3	BOOT, LOAD, GAME, APP.
HANDOVER `$0C25`	0, 1, 2	None yet, by the BIOS, by the loader.
TV `$0C1E`	0, 1	NTSC, PAL.
STAGED `$0C1F`	bits 0–3	Ready, a FujiNet app, the BIOS accepts it, an HSC ROM.

Where	Value	Meaning
HSC $0C26	bits 0–3	ROM installed, on, saved to SD, unsaved changes.
N: STATUS reply	byte 3 = 1	Good.
	byte 3 = 136	End of file.
GET_WIFISTATUS	3	Connected.
READ_DIR_ENTRY	$7F,$7F	No more entries.

C Program Listings

Every program in this handbook, as built and run. The assembly library and its programs from `listings/common/` are the cartridge bring-up's, unchanged.

The assembly library

fujinet.inc

```
; fujinet.inc -- the cart's mailbox and the 7800's registers, for ca65.
; Hand-mirrored from firmware/include/fuji_mailbox.h; keep them in step.
;
; Write pages take plain STA/STX/STY only: a read-modify-write instruction
; writes the old value first, which the cart sees as a stray event.

FN_LOADER       = $0600         ; the loader (cart-served, 512 bytes)
FN_LOADER_BOOT  = $0600         ; load the armed image (or CONFIG) and start it
FN_LOADER_CONFIG = $0603        ; load CONFIG and start it
FN_REPLY        = $0800         ; 1K reply window
FN_STATUS       = $0C00         ; status page
FN_REGSEL       = $0D00         ; write V to +n: register n = V
FN_TXDATA       = $0F00         ; write anywhere: append one TX byte

; status page (FN_R_* - $400)
FN_ACKSEQ       = FN_STATUS + $00
FN_STAT         = FN_STATUS + $01
FN_ERR          = FN_STATUS + $02
FN_REPLYCMD     = FN_STATUS + $03
FN_RXLEN        = FN_STATUS + $04
FN_BOOTSTATE    = FN_STATUS + $06
FN_BOOTPCT      = FN_STATUS + $07
FN_BOOTERR      = FN_STATUS + $08
FN_MAGIC        = FN_STATUS + $09   ; 'F','N'
FN_PROTOVER     = FN_STATUS + $0B
FN_LOADSTATE    = FN_STATUS + $0D
FN_LOADDST      = FN_STATUS + $0E
FN_LOADN        = FN_STATUS + $0F
FN_LOADSEQ      = FN_STATUS + $10
FN_LOADPCT      = FN_STATUS + $11
FN_MODE         = FN_STATUS + $12
FN_MAPPER       = FN_STATUS + $14
FN_LINK         = FN_STATUS + $15
FN_BOOTGOT      = FN_STATUS + $16
FN_BOOTTOT      = FN_STATUS + $19
FN_INPTCTRL     = FN_STATUS + $1C
FN_INPTLOCK     = FN_STATUS + $1D
FN_TV           = FN_STATUS + $1E
FN_STAGED       = FN_STATUS + $1F
FN_STAGEDKIND   = FN_STATUS + $20
FN_STAGEDCRC    = FN_STATUS + $21
FN_HANDOVER     = FN_STATUS + $25
FN_HSC          = FN_STATUS + $26
FN_DIAGRMW      = FN_STATUS + $27

; registers (write to FN_REGSEL + n)
FN_REG_DEVICE   = $00
FN_REG_CMD      = $01
FN_REG_NPARAM   = $02
FN_REG_DATA_RST = $05
FN_REG_RXSLICE  = $06
FN_REG_SEQ      = $10
FN_REG_BOOTLOCK = $11
FN_REG_SLICE_ACK = $14
FN_REG_TV       = $15
FN_REG_HSC      = $16
FN_HOT_GO_BIOS  = $FB
FN_HOT_CONFIG   = $FC
FN_HOT_GO       = $FD
FN_HOT_SWAP     = $FE

FN_BOOTLOCK_MAGIC = $B5

FN_BOOT_READY   = 2
FN_BOOT_FAILED  = $80
FN_LOAD_IDLE    = 0
FN_LOAD_SLICE   = 1
FN_LOAD_DONE    = 2
FN_HO_BIOS      = 1
FN_HO_DIRECT    = 2

; the claim, at $FF70 in every FujiNet app
FN_CLAIM        = $FF70

; 7800 hardware
INPTCTRL        = $01           ; write-only; any TIA write reaches it until locked
INPT4           = $0C           ; left fire (one-button mode)
INPT5           = $0D
INPT0           = $08           ; left buttons (two-button mode)
INPT1           = $09
BACKGRND        = $20
P0C1            = $21
P0C2            = $22
P0C3            = $23
MWSYNC          = $24
P1C1            = $25
P1C2            = $26
P1C3            = $27
MSTAT           = $28           ; bit 7: VBLANK
P2C1            = $29
P2C2            = $2A
P2C3            = $2B
DPPH            = $2C
P3C1            = $2D
P3C2            = $2E
P3C3            = $2F
DPPL            = $30
P4C1            = $31
P4C2            = $32
P4C3            = $33
CHARBASE        = $34
P5C1            = $35
P5C2            = $36
P5C3            = $37
OFFSET          = $38
P6C1            = $39
P6C2            = $3A
P6C3            = $3B
CTRL            = $3C
P7C1            = $3D
P7C2            = $3E
P7C3            = $3F
SWCHA           = $280
CTLSWA          = $281
SWCHB           = $282
CTLSWB          = $283

; client error codes (fujilib.s)
FNEOK     = 0                   ; FN_ERR_OK
FNEWAIT   = $FF                 ; the cart never answered at all
FNENAK    = $EE                 ; the server said NAK
FNENOC    = $FD                 ; no cartridge answered ('F','N' missing)

; FujiNet devices and commands
FNDEVFC   = $70                 ; the FUJI device
FNCADPX   = $C4                 ; GET_ADAPTERCONFIG_EXTENDED
FNCMHST   = $F9                 ; MOUNT_HOST
```

```
FNCMIMG   = $F8                ; MOUNT_IMAGE
FNCSDFP   = $E2                ; SET_DEVICE_FULLPATH
FMREAD    = 1                  ; disk access mode: read

; the clients' zero page, clear of what the loader uses ($80-$89)
FN_SEQ    = $E0                ; the sequence we are waiting for
FN_CNT    = $E1                ; timeout quanta remaining
FN_DEV    = $E2                ; transaction setup, read by fn_beg
FN_CMD    = $E3
FN_NPR    = $E4
FN_TMO    = $E5                ; timeout, in ~0.7 s quanta
FN_PTR    = $E6                ; a string pointer (2 bytes)
DISP_PTR  = $E8                ; the text cursor (2 bytes)
DISP_TMP  = $EA
```

fujilib.s

```
; fujilib.s -- the 6502 side of the FujiNet cartridge mailbox: the code.
;
; A register write is ONE store: `sta FN_REGSEL,x` with the register in X and
; the value in A. The base's low byte is $00, so the index never carries and
; the store's extra cycle -- a READ of the same address -- is inert here.

        .include "fujinet.inc"

        .export fn_chk, fn_rw, fn_beg, fn_txb, fn_pb, fn_pw, fn_path
        .export fn_go, fn_ack, fn_blk, fn_boot

        .segment "CODE"

; fn_rw -- register X = A.
.proc fn_rw
        sta     FN_REGSEL,x
        rts
.endproc

; fn_chk -- Z set if a FujiNet cartridge is answering.
.proc fn_chk
        lda     FN_MAGIC
        cmp     #'F'
        bne     done
        lda     FN_MAGIC+1
        cmp     #'N'
done:   rts
.endproc

; fn_beg -- begin the transaction described by FN_DEV / FN_CMD / FN_NPR.
; Rewinds the TX stream last, so parameters may be appended at once. Resets
; FN_TMO, so a slow command raises it after this.
.proc fn_beg
        lda     #16             ; ~11 s: past the cart's 5 s window for ordinary commands
        sta     FN_TMO
        lda     FN_DEV
        ldx     #FN_REG_DEVICE
        jsr     fn_rw
        lda     FN_CMD
        ldx     #FN_REG_CMD
        jsr     fn_rw
        lda     FN_NPR
        ldx     #FN_REG_NPARAM
        jsr     fn_rw
        lda     #0
        ldx     #FN_REG_DATA_RST
        jmp     fn_rw
.endproc

; fn_txb -- append A to the TX stream.
.proc fn_txb
        sta     FN_TXDATA
        rts
.endproc

; fn_pb -- append a one-byte parameter with value A.
.proc fn_pb
        pha
        lda     #1
        sta     FN_TXDATA
        pla
        sta     FN_TXDATA
        rts
.endproc

; fn_pw -- append a two-byte little-endian parameter. A = low, X = high.
.proc fn_pw
        pha
        lda     #2
        sta     FN_TXDATA
        pla
        sta     FN_TXDATA
        txa
        sta     FN_TXDATA
        rts
.endproc

; fn_path -- append the NUL-terminated string at FN_PTR to the TX stream,
; padded with NULs to EXACTLY 256 bytes: OPEN_DIRECTORY and
; SET_DEVICE_FULLPATH both read exactly 256.
.proc fn_path
        ldy     #0
loop:   lda     (FN_PTR),y
        beq     pad
        sta     FN_TXDATA
        iny
        bne     loop
        rts
pad:    lda     #0
padl:   sta     FN_TXDATA
        iny
        bne     padl
        rts
.endproc

; fn_go -- commit and wait. Returns A = FNE* (0 = the transaction completed).
; The next sequence number comes from the CART's persisted ACKSEQ + 1, never
; a counter in RAM: a power cycle restarts the client, not the cart.
.proc fn_go
        lda     FN_ACKSEQ
        clc
        adc     #1
        bne     have
        lda     #1              ; 0 is reserved as "never used"
have:   sta     FN_SEQ
        ldx     #FN_REG_SEQ
        jsr     fn_rw           ; launching it is this single store
```

```
        lda     FN_TMO
        sta     FN_CNT
outer:  ldy     #0
mid:    ldx     #0
inner:  lda     FN_ACKSEQ       ; published LAST, after the whole reply
        cmp     FN_SEQ
        beq     got
        dex
        bne     inner
        dey
        bne     mid
        dec     FN_CNT
        bne     outer
        lda     #FNEWAIT
        rts
got:    lda     FN_ERR
        rts
.endproc

; fn_ack -- 0 on success, FNENAK if the server said no. Call after fn_go
; returned 0.
.proc fn_ack
        lda     FN_REPLYCMD
        cmp     #$06
        beq     ok
        lda     #FNENAK
        rts
ok:     lda     #FNEOK
        rts
.endproc

; fn_blk -- arm the image load (FN_REG_BOOTLOCK).
.proc fn_blk
        lda     #FN_BOOTLOCK_MAGIC
        ldx     #FN_REG_BOOTLOCK
        jmp     fn_rw
.endproc

; fn_boot -- hand the console to the loader. Does not return.
.proc fn_boot
        sei
        lda     #$7F
        sta     CTRL
        jmp     FN_LOADER_BOOT
.endproc
```

fujidisp.s

```
; fujidisp.s -- text for the bring-up clients: MARIA 320A, 40 x 24.
;
; One zone per text row; each zone's display list is two indirect headers of
; 20 characters, so MARIA reads character codes from the screen buffer and
; their pixels from the font at CHARBASE. Everything MARIA reads but the font
; lives in the cart's RAM. Character = ASCII code; +$80 is reverse video.

        .include "fujinet.inc"

        .export disp_init, disp_on, disp_off, disp_at, disp_str, disp_hex
        .export disp_rpl, disp_putc, disp_dec, disp_vbl

COLS    = 40
ROWS    = 24
SCR     = $4000                 ; ROWS x COLS character codes
DLS     = $4400                 ; ROWS display lists, 12 bytes each
DLL     = $4600                 ; the display list list
NULLDL  = $46F0                 ; an empty display list

        .segment "CODE"

; disp_init -- cleared screen, display lists, palette. DMA stays off.
.proc disp_init
        lda     #' '
        ldx     #0
@clr:   sta     SCR,x
        sta     SCR+$100,x
        sta     SCR+$200,x
        sta     SCR+$300,x
        inx
        bne     @clr
        lda     #0
        sta     NULLDL
        sta     NULLDL+1

        ; the display lists: [lo, $60, hi, pal 0 / 20 wide, hpos] twice, then
        ; 0, 0; ROWS x 12 bytes is more than Y reaches, so through a pointer
        ldx     #0              ; row
@dl:    lda     dllo,x
        sta     DISP_PTR
        lda     dlhi,x
        sta     DISP_PTR+1
        ldy     #0
        lda     rowlo,x
        sta     (DISP_PTR),y    ; +0
        ldy     #5
        clc
        adc     #20
        sta     (DISP_PTR),y    ; +5
        lda     rowhi,x
        adc     #0
        ldy     #7
        sta     (DISP_PTR),y    ; +7
        lda     rowhi,x
        ldy     #2
        sta     (DISP_PTR),y    ; +2
        lda     #$60            ; 320A, extended header, indirect
        ldy     #1
```

```
        sta     (DISP_PTR),y
        ldy     #6
        sta     (DISP_PTR),y
        lda     #$0C            ; palette 0, 20 bytes
        ldy     #3
        sta     (DISP_PTR),y
        ldy     #8
        sta     (DISP_PTR),y
        lda     #80
        ldy     #9
        sta     (DISP_PTR),y
        lda     #0
        ldy     #4
        sta     (DISP_PTR),y
        ldy     #10
        sta     (DISP_PTR),y
        iny
        sta     (DISP_PTR),y
        inx
        cpx     #ROWS
        bne     @dl

        ; the DLL: 25 blank lines, the rows, 64 blank lines
        ldx     #0
        ldy     #0
@top:   lda     top,x
        sta     DLL,y
        iny
        inx
        cpx     #6
        bne     @top
        ldx     #0
@rows:  lda     #7              ; 8 lines
        sta     DLL,y
        lda     dlhi,x
        sta     DLL+1,y
        lda     dllo,x
        sta     DLL+2,y
        iny
        iny
        iny
        inx
        cpx     #ROWS
        bne     @rows
        ldx     #4
@bot:   lda     #15
        sta     DLL,y
        lda     #>NULLDL
        sta     DLL+1,y
        lda     #<NULLDL
        sta     DLL+2,y
        iny
        iny
        iny
        dex
        bne     @bot

        lda     #>font
        sta     CHARBASE
        lda     #0
        sta     BACKGRND
        lda     #$0F
        sta     P0C1
        sta     P0C2
        sta     P0C3
        lda     #>DLL
        sta     DPPH
        lda     #<DLL
        sta     DPPL
        rts
.endproc

; disp_vbl -- wait for the start of the next vertical blank.
.proc disp_vbl
@in:    bit     MSTAT
        bmi     @in
@out:   bit     MSTAT
        bpl     @out
        rts
.endproc

; disp_on -- DMA on at a frame edge: 320A, one-byte characters.
.proc disp_on
        jsr     disp_vbl
        lda     #$43
        sta     CTRL
        rts
.endproc

; disp_off -- DMA off.
.proc disp_off
        lda     #$7F
        sta     CTRL
        rts
.endproc

; disp_at -- cursor to row A (0-23), column X (0-39).
.proc disp_at
        tay
        txa
        clc
        adc     rowlo,y
        sta     DISP_PTR
        lda     rowhi,y
        adc     #0
        sta     DISP_PTR+1
        rts
.endproc

; disp_putc -- one character at the cursor.
.proc disp_putc
        ldy     #0
        sta     (DISP_PTR),y
        inc     DISP_PTR
        bne     @ok
        inc     DISP_PTR+1
@ok:    rts
.endproc

; disp_str -- the NUL-terminated string at FN_PTR, at most 40 characters;
; the cursor ends after it.
.proc disp_str
        ldy     #0
@l:     lda     (FN_PTR),y
        beq     advance
        sta     (DISP_PTR),y
        iny
        cpy     #COLS
        bne     @l
        ; fall through
.endproc

; advance -- the cursor moves on by Y.
.proc advance
        tya
        clc
        adc     DISP_PTR
        sta     DISP_PTR
        bcc     @d
        inc     DISP_PTR+1
@d:     rts
.endproc

; disp_rpl -- up to Y bytes of the reply window from offset X, stopping at a
; NUL; the cursor ends after them.
.proc disp_rpl
        sty     DISP_TMP
        ldy     #0
@l:     lda     FN_REPLY,x
        beq     @d
        sta     (DISP_PTR),y
        inx
        iny
        cpy     DISP_TMP
        bne     @l
@d:     jmp     advance
.endproc

; disp_hex -- A as two hex digits.
.proc disp_hex
        pha
        lsr
        lsr
        lsr
        lsr
        tax
        lda     hex,x
        jsr     disp_putc
        pla
        and     #$0F
        tax
        lda     hex,x
        jmp     disp_putc
.endproc

; disp_dec -- A as up to three decimal digits, no leading zeros.
.proc disp_dec
        ldx     #0
@h:     cmp     #100
        bcc     @hd
```

```
239         sbc     #100
240         inx
241         bne     @h
242 @hd:     sta     DISP_TMP
243         txa
244         beq     @t0
245         ora     #'0'
246         jsr     disp_putc
247         ldx     #1              ; tens must
    show from here on
248         bne     @t1
249 @t0:     ldx     #0
250 @t1:     stx     DISP_TMP+1
251         lda     DISP_TMP
252         ldx     #0
253 @t:      cmp     #10
254         bcc     @td
255         sbc     #10
256         inx
257         bne     @t
258 @td:     sta     DISP_TMP
259         txa
260         bne     @tp
261         lda     DISP_TMP+1
262         beq     @o
263         txa
264 @tp:     ora     #'0'
265         jsr     disp_putc
266 @o:      lda     DISP_TMP
267         ora     #'0'
268         jmp     disp_putc
269 .endproc
270
271         .segment "RODATA"
272 hex:    .byte   "0123456789ABCDEF"
273 top:    .byte   15, >NULLDL, <NULLDL, 8,
    >NULLDL, <NULLDL
274 rowlo:
275         .repeat ROWS, r
276         .byte   <(SCR + r * COLS)
277         .endrepeat
278 rowhi:
279         .repeat ROWS, r
280         .byte   >(SCR + r * COLS)
281         .endrepeat
282 dllo:
283         .repeat ROWS, r
284         .byte   <(DLS + r * 12)
285         .endrepeat
286 dlhi:
287         .repeat ROWS, r
288         .byte   >(DLS + r * 12)
289         .endrepeat
290
291         .segment "FONT"
292 font:
293         .include "font.inc"
294
```

a78init.s

```
1  ; a78init.s -- reset, the claim and the
   vectors for the bring-up clients.
2  ;
3  ; Nothing here writes $00-$1F: INPTCTRL is
   left as the BIOS left it,
4  ; unlocked, so a game booted later can be
   handed to the console's own BIOS.
5
6          .include "fujinet.inc"
7
8          .import main, nmi, disp_init
9
10         .segment "STARTUP"
11 reset:  sei
12         cld
13         lda     #$7F            ; DMA off
   until the display is built
14         sta     CTRL
15         ldx     #$FF
16         txs
17         lda     #0
18         ldx     #$40            ; zero page
   $40-$FF: never the TIA below it
19 @zp:    sta     $00,x
20         inx
21         bne     @zp
22         sta     DISP_PTR        ; the cart's
   RAM, $4000-$7FFF
23         tay
24         ldx     #$40
25 @pg:    stx     DISP_PTR+1
26 @b:     sta     (DISP_PTR),y
27         iny
28         bne     @b
29         inx
30         cpx     #$80
31         bne     @pg
32         jsr     disp_init
33         jmp     main
34 irq:    rti
35
36         .segment "CLAIM"
37 .ifdef BANKED
38         .byte   "FUJI", 1, 5, 1, 0, 0, 0    ;
   version 1, SuperGame + RAM, cart RAM
39 .else
40         .byte   "FUJI", 1, 0, 1, 0, 0, 0    ;
   version 1, kind auto, cart RAM
41 .endif
42
43         .segment "VECTORS"
44         .byte   $FF, $87                    ;
   a 7800 cart, hashed from $8000
45         .word   nmi, reset, irq
46
```

a78.cfg

```
# a78.cfg -- a 32K FujiNet client: ROM at $8000, the cart's 16K of RAM at
# $4000 (the claim's RAM flag asks for it), the claim at $FF70.
MEMORY {
    ZP:     file = "", start = $0040, size = $00A0;
    RAM:    file = "", start = $4000, size = $4000;
    ROM:    file = %O, start = $8000, size = $7F70, fill = yes, fillval = $FF;
    CLAIM: file = %O, start = $FF70, size = $000A, fill = yes, fillval = $00;
    SIG:    file = %O, start = $FF7A, size = $007E, fill = yes, fillval = $FF;
    VEC:    file = %O, start = $FFF8, size = $0008;
}
SEGMENTS {
    ZEROPAGE: load = ZP,    type = zp,  optional = yes;
    BSS:      load = RAM,   type = bss, optional = yes;
    STARTUP:  load = ROM,   type = ro;
    CODE:     load = ROM,   type = ro;
    RODATA:   load = ROM,   type = ro,  optional = yes;
    FONT:     load = ROM,   type = ro,  align = $100;
    CLAIM:    load = CLAIM, type = ro;
    VECTORS:  load = VEC,   type = ro;
}

```

a78bank.cfg

```
# a78bank.cfg -- a 64K banked FujiNet client (SuperGame with cart RAM, the
# claim's kind byte says so): three 16K banks that take turns at $8000, the
# fixed last bank at $C000 with the claim, and the cart's RAM at $4000.
MEMORY {
    ZP:    file = "", start = $0040, size = $00A0;
    RAM:   file = "", start = $4000, size = $4000;
    BANK0: file = %O, start = $8000, size = $4000, fill = yes, fillval = $FF;
    BANK1: file = %O, start = $8000, size = $4000, fill = yes, fillval = $FF;
    BANK2: file = %O, start = $8000, size = $4000, fill = yes, fillval = $FF;
    ROM:   file = %O, start = $C000, size = $3F70, fill = yes, fillval = $FF;
    CLAIM: file = %O, start = $FF70, size = $000A, fill = yes, fillval = $00;
    SIG:   file = %O, start = $FF7A, size = $007E, fill = yes, fillval = $FF;
    VEC:   file = %O, start = $FFF8, size = $0008;
}
SEGMENTS {
    ZEROPAGE: load = ZP,    type = zp,  optional = yes;
    BSS:      load = RAM,   type = bss, optional = yes;
    BANK0:    load = BANK0, type = ro;
    BANK1:    load = BANK1, type = ro;
    BANK2:    load = BANK2, type = ro;
    STARTUP:  load = ROM,   type = ro;
    CODE:     load = ROM,   type = ro;
    RODATA:   load = ROM,   type = ro,  optional = yes;
    FONT:     load = ROM,   type = ro,  align = $100;
    CLAIM:    load = CLAIM, type = ro;
    VECTORS:  load = VEC,   type = ro;
}

```

```
; devdefs.inc -- the other devices, and the FUJI commands the bring-up's
; fujinet.inc does not name. Values from include/fujiDeviceID.h and
; include/fujiCommandID.h; the appkey struct from lib/device/fujiDevice/
; AppKeyMixin.h. Kept apart from fujinet.inc so that file stays verbatim.

; ---------------- device ids ----------------
DSKDEV  =       $31             ; D1: (D2:-D8: are $32-$38)
PRNDEV  =       $40             ; P1:
CLKDEV  =       $45             ; the clock (APETIME)
MDMDEV  =       $50             ; R: the modem
CPMDEV  =       $5A             ; CP/M

; ---------------- FUJI ($70) commands ----------------
FNCRST  =       $FF             ; RESET
FNCGSSD =       $FE             ; GET_SSID
FNCSCAN =       $FD             ; SCAN_NETWORKS
FNCSCNR =       $FC             ; GET_SCAN_RESULT
FNCSSSD =       $FB             ; SET_SSID
FNCWIFI =       $FA             ; GET_WIFISTATUS
FNCRHST =       $F4             ; READ_HOST_SLOTS
FNCWHST =       $F3             ; WRITE_HOST_SLOTS
FNCRDEV =       $F2             ; READ_DEVICE_SLOTS
FNCWDEV =       $F1             ; WRITE_DEVICE_SLOTS
FNCWIEN =       $EA             ; GET_WIFI_ENABLED
FNCODIR =       $F7             ; OPEN_DIRECTORY
FNCRDIR =       $F6             ; READ_DIR_ENTRY
FNCCDIR =       $F5             ; CLOSE_DIRECTORY
FNCSDPS =       $E4             ; SET_DIRECTORY_POSITION
FNCUIMG =       $E9             ; UNMOUNT_IMAGE
FNCADPC =       $E8             ; GET_ADAPTERCONFIG
FNCNEWD =       $E7             ; NEW_DISK
FNCUHST =       $E6             ; UNMOUNT_HOST
FNCGDPS =       $E5             ; GET_DIRECTORY_POSITION
FNCSHPX =       $E1             ; SET_HOST_PREFIX
FNCGHPX =       $E0             ; GET_HOST_PREFIX
FNCAKWR =       $DE             ; WRITE_APPKEY
FNCAKRD =       $DD             ; READ_APPKEY
FNCAKOP =       $DC             ; OPEN_APPKEY
FNCAKCL =       $DB             ; CLOSE_APPKEY
FNCGDFP =       $DA             ; GET_DEVICE_FULLPATH
FNCCBOT =       $D9             ; CONFIG_BOOT
FNCCOPY =       $D8             ; COPY_FILE
FNCMALL =       $D7             ; MOUNT_ALL
FNCSBM  =       $D6             ; SET_BOOT_MODE
FNCRAND =       $D3             ; RANDOM_NUMBER
FNCB64EI =      $D0             ; BASE64_ENCODE_INPUT
FNCB64EC =      $CF             ; BASE64_ENCODE_COMPUTE
FNCB64EL =      $CE             ; BASE64_ENCODE_LENGTH
FNCB64EO =      $CD             ; BASE64_ENCODE_OUTPUT
FNCB64DI =      $CC             ; BASE64_DECODE_INPUT
FNCB64DC =      $CB             ; BASE64_DECODE_COMPUTE
FNCB64DL =      $CA             ; BASE64_DECODE_LENGTH
FNCB64DO =      $C9             ; BASE64_DECODE_OUTPUT
FNCHSHI =       $C8             ; HASH_INPUT
FNCHSHC =       $C7             ; HASH_COMPUTE
FNCHSHL =       $C6             ; HASH_LENGTH
FNCHSHO =       $C5             ; HASH_OUTPUT
FNCHSHN =       $C3             ; HASH_COMPUTE_NO_CLEAR
FNCHSHX =       $C2             ; HASH_CLEAR
FNCQRO  =       $BF             ; QRCODE_OUTPUT
FNCQRL  =       $BE             ; QRCODE_LENGTH
FNCQRE  =       $BD             ; QRCODE_ENCODE
FNCQRI  =       $BC             ; QRCODE_INPUT
FNCGUID =       $BB             ; GENERATE_GUID
FNCSTAT =       $53             ; STATUS
FNCRDY  =       $00             ; DEVICE_READY

; ---------------- appkeys ----------------
; OPEN_APPKEY's payload is ONE six-byte packed struct: creator (u16, little-
; endian), app, key, mode, and a reserved byte that is not optional.
AKMRD   =       0               ; mode: read
AKMWR   =       1               ; mode: write
AKCREAT =       1               ; the shared FujiNet username slot...
AKAPP   =       1               ; ...creator 1, app 1, key 0, which every
AKKEY   =       0               ; game in the family reads

; ---------------- the clock ($45) ----------------
CKGTIME =       $93             ; GETTIME: 7 bytes, binary
CKISOL  =       $49             ; 'I' GET_ISO_LOCAL: "YYYY-MM-DDTHH:MM:SS"...
CKISOU  =       $5A             ; 'Z' GET_ISO_UTC
CKGEN   =       $47             ; 'G' GET_GENERAL
CKTZLEN =       $4C             ; 'L' GETTZ_LEN
CKSETTZ =       $99             ; SETTZ: payload = the POSIX TZ string
CKTZTIM =       $9A             ; GETTZTIME

; ---------------- hash algorithms (HASH_COMPUTE's parameter) ----------------
HAMD5   =       0
HASHA1  =       1
HASHA256 =      3
HASHA512 =      4


; ---------------- FNBEGIN: the four stores before fn_beg ----------------
; FNBEGIN dev, cmd, nparam -- set up and begin a transaction (fn_beg).
.macro  FNBEGIN dev, cmd, npr
        lda     #dev
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #npr
        sta     FN_NPR
        jsr     fn_beg
.endmacro

```

```
; netdefs.inc -- the N: (network) device, for the client library.
;
; Device ids from include/fujiDeviceID.h, command bytes from
; include/fujiCommandID.h, open modes from lib/network-protocol/HTTP.cpp and
; the dispatch table and STATUS reply from lib/device/NDevice/NDevice.cpp.
; The bring-up's fujinet.inc covers the FUJI device only; this is the rest of
; what a client that talks to the network needs, kept separate so that file
; stays verbatim.
NETDEV  =       $71             ; N1: (N2:-N8: are $72-$78)
NCOPEN  =       $4F             ; 'O' open: params mode, translation; payload = devicespec
NCCLOSE =       $43             ; 'C' close
NCREAD  =       $52             ; 'R' read: ONE two-byte param, the count
NCWRITE =       $57             ; 'W' write: ONE two-byte param, then the payload
NCSTAT  =       $53             ; 'S' status: two one-byte params, both 0
NCPARSE =       $50             ; 'P' parse the reply as JSON
NCQUERY =       $51             ; 'Q' query a JSON path; payload = the path
NCPRSR  =       $FC             ; SET_PARSER: second param 0 none, 1 JSON, 2 HTML, 3 XML
NCJSON  =       NCPRSR
NCSETP  =       $FB             ; SET_PARAMETER: params type (0 query flags, 1 EOL), value
NCTRANS =       $54             ; 'T' translation: second param the code
NCEOL   =       $4C             ; 'L' set the EOL: payload the string
NCRATE  =       $5A             ; 'Z' interrupt rate: param the rate
NCSEEK  =       $25             ; '%' seek: ONE four-byte param, the offset
NCTELL  =       $26             ; '&' tell: reply 4 bytes
NCREN   =       $20             ; rename: params mode, 0; payload "N:...old,new"
NCDEL   =       $21             ; '!' delete \
NCLOCK  =       $23             ; '#' lock |
NCUNLK  =       $24             ; '$' unlock | params mode, 0;
NCMKDIR =       $2A             ; '*' mkdir | payload the devicespec
NCRMDIR =       $2B             ; '+' rmdir       /
NCCHDIR =       $2C             ; ',' chdir: payload the path
NCGETCW =       $30             ; '0' getcwd: reply the prefix
NCACCPT =       $41             ; 'A' accept a TCP client
NCCCLI  =       $63             ; 'c' close the TCP client
NCRMT   =       $72             ; 'r' UDP: who sent the last datagram
NCDEST  =       $44             ; 'D' UDP: payload host:port
NCUSER  =       $FD             ; payload the user name
NCPASS  =       $FE             ; payload the password
NPNONE  =       0               ; parsers
NPJSON  =       1
NPHTML  =       2
NPXML   =       3
NCHTTPM =       $4D             ; 'M' HTTP channel mode: second param 0 body, 1 collect
                                ;   headers, 2 get headers, 3 set headers, 4 POST data
NMREAD  =       4               ; open mode: read (HTTP GET)
NMWRITE =       8               ; write (HTTP PUT)
NMHTTPG =       12              ; read/write (HTTP GET, headers allowed)
NMPOST  =       13              ; HTTP POST
NTRNONE =       0               ; translation: none
NTRCR   =       1               ; CR to/from the host's EOL
NTRLF   =       2               ; LF
NTRCRLF =       3               ; CR LF
NSAVLO  =       0               ; STATUS reply: bytes waiting, low
NSAVHI  =       1               ;   high
NSCONN  =       2               ;   connected
NSDEVST =       3               ;
nDevStatus_t: 1 = success

```

The assembly programs

hello.s

```
; hello.s -- M0: the boot block, the loader, MARIA text and a frame counter.
;
; No mailbox traffic; it carries the claim so the cart runs it as an app.
; Also shows what the cart's INPTCTRL model says, which must be unlocked:
; nothing on this path may have locked the console.

        .include "fujinet.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_vbl, disp_putc
        .export main, nmi

        .segment "ZEROPAGE"
frames: .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        PUTS    2, 2, title
        PUTS    4, 2, hello
        PUTS    6, 2, m0
        PUTS    8, 2, tinpt
        PUTS    20, 2, frames_lbl
        jsr     disp_on
loop:   jsr     disp_vbl
        inc     frames
        lda     #20
        ldx     #10
        jsr     disp_at
        lda     frames
        jsr     disp_hex
        lda     #8
        ldx     #12
        jsr     disp_at
        lda     FN_INPTCTRL
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        lda     FN_INPTLOCK
        jsr     disp_hex
        jmp     loop
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:      .byte "FUJINET ATARI 7800", 0
hello:      .byte "HELLO, WORLD", 0
m0:         .byte "M0: BOOT BLOCK + LOADER + 320A TEXT", 0
tinpt:      .byte "INPTCTRL: ", 0
frames_lbl: .byte "FRAMES: ", 0

```

fujitest.s

```
; fujitest.s -- M1: one real transaction, and proof the cart survives a
; power cycle.
;
; Asks the FujiNet for GET_ADAPTERCONFIG_EXTENDED and puts the live SSID, IP
; address and firmware version on screen. Nothing is faked: the bytes come
; off a socket to fujinet-pc, through the cartridge's own fujimail.c.
; emu/powertest.lua power-cycles the console and requires the NEXT sequence
; number to be ACKSEQ+1, not 1.

        .include "fujinet.inc"

        .import disp_at, disp_str, disp_hex, disp_rpl, disp_on, disp_putc
```

```
        .import fn_chk, fn_beg, fn_go, fn_ack
        .export main, nmi

; Field offsets inside GET_ADAPTERCONFIG_EXTENDED (lib/device/fujiDevice/
; fujiDevice.h, packed). All below 256, so one index reaches them.
AC_SSID = 0                     ; char[33]
AC_VER  = 125                   ; char[15]
AC_SIP  = 140                   ; char[16], the IP already formatted as text

        .segment "ZEROPAGE"
aperr:  .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        lda     #0
        sta     aperr
        jsr     fn_chk
        beq     have
        lda     #FNENOC
        sta     aperr
        jmp     show

have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     show
        jsr     fn_ack
        sta     aperr

show:   PUTS    2, 2, title
        lda     aperr
        beq     ok

        PUTS    5, 2, tfail
        lda     #5
        ldx     #12
        jsr     disp_at
        lda     aperr
        jsr     disp_hex
        jmp     seq

ok:     PUTS    5, 2, tssid
        lda     #5
        ldx     #8
        jsr     disp_at
        ldx     #AC_SSID
        ldy     #30
        jsr     disp_rpl

        PUTS    7, 2, tip
        lda     #7
        ldx     #8
        jsr     disp_at
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl

        PUTS    9, 2, tver
        lda     #9
        ldx     #8
        jsr     disp_at
        ldx     #AC_VER
        ldy     #15
        jsr     disp_rpl

        ; the sequence number, in hex: after a power cycle it must read 02
seq:    PUTS    12, 2, tseq
        lda     #12
        ldx     #10
        jsr     disp_at
        lda     FN_ACKSEQ
        jsr     disp_hex

        PUTS    14, 2, terr
        lda     #14
        ldx     #10
        jsr     disp_at
        lda     FN_ERR
        jsr     disp_hex

        jsr     disp_on
loop:   jmp     loop
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:  .byte "FUJINET ATARI 7800 M1", 0
tfail:  .byte "FAILED: E", 0
tssid:  .byte "SSID", 0
tip:    .byte "IP  ", 0
tver:   .byte "FW  ", 0
tseq:   .byte "ACKSEQ: ", 0
terr:   .byte "ERR:    ", 0
```

```
; netget.s -- fetch a URL through the N: device and show the text.
;
; OPEN the devicespec as an HTTP GET, STATUS until the byte count settles,
; READ what is waiting (at most 512 bytes) into the reply window, show it a
; line at a time -- LF ends a row, CR is skipped, a row is at most 36
; characters -- then CLOSE.

        .include "fujinet.inc"
        .include "netdefs.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_putc, disp_vbl
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_pw, fn_txb
        .export main, nmi

NMAX    = 512                   ; the most we read
NROW0   = 4                     ; the first text row of the reply
NROWS   = 18                    ; rows 4-21
NCOLS   = 36
NTRIES  = 30                    ; STATUS readings before giving up

        .segment "ZEROPAGE"
aperr:  .res 1                  ; what went wrong, if anything
navail: .res 2                  ; bytes waiting, from STATUS
nprev:  .res 2                  ; the previous STATUS reading
ntry:   .res 1
nsrc:   .res 2                  ; the reply cursor
nleft:  .res 2                  ; reply bytes still to show
nrow:   .res 1
ncol:   .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        PUTS    2, 2, title
        jsr     disp_on
        lda     #0
        sta     aperr
        jsr     fn_chk
        beq     open
        lda     #FNENOC
        sta     aperr
        jmp     fail

        ; OPEN: two one-byte parameters, the mode and the translation, then
        ; the devicespec as the payload
open:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCOPEN
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldy     #0
@url:   lda     url,y
        beq     @urle
        jsr     fn_txb
        iny
        bne     @url
@urle:  jsr     go_ack

        ; STATUS until two readings agree and are not zero: the adapter
        ; answers as soon as SOME of the response has arrived
        lda     #$FF
        sta     nprev
        sta     nprev+1
        lda     #0
        sta     ntry
stat:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCSTAT
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY+NSAVLO
        sta     navail
        lda     FN_REPLY+NSAVHI
        sta     navail+1
        cmp     nprev+1
        bne     again
        lda     navail
        cmp     nprev
        bne     again
        ora     navail+1
        bne     read
again:  lda     navail
        sta     nprev
        lda     navail+1
        sta     nprev+1
        inc     ntry
        lda     ntry
        cmp     #NTRIES
        bcc     @wait
        lda     #FNEWAIT
        sta     aperr
        jmp     fail
@wait:  ldx     #3              ; three frames between readings
@w:     jsr     disp_vbl
        dex
        bne     @w
        jmp     stat

        ; READ: ONE two-byte parameter, the count, capped at NMAX
read:   lda     navail+1
        cmp     #>NMAX
        bcc     @len
        lda     #<NMAX
        sta     navail
        lda     #>NMAX
        sta     navail+1
@len:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCREAD
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     navail
        ldx     navail+1
        jsr     fn_pw
        jsr     go_ack

        ; show it, a row per line
        lda     #<FN_REPLY
        sta     nsrc
        lda     #>FN_REPLY
        sta     nsrc+1
        lda     FN_RXLEN
        sta     nleft
        lda     FN_RXLEN+1
        sta     nleft+1
        lda     #NROW0
        sta     nrow
        jsr     newrow
next:   lda     nleft
        ora     nleft+1
        beq     close
        ldy     #0
        lda     (nsrc),y
        inc     nsrc
```

```
        bne     @s
        inc     nsrc+1
@s:     ldx     nleft
        bne     @n
        dec     nleft+1
@n:     dec     nleft
        cmp     #13             ; CR: skip
        beq     next
        cmp     #10             ; LF: next row
        bne     @c
        inc     nrow
        lda     nrow
        cmp     #NROW0+NROWS
        beq     close
        jsr     newrow
        jmp     next
@c:     ldx     ncol
        cpx     #NCOLS
        beq     next            ; past the edge: drop it
        inc     ncol
        jsr     disp_putc
        jmp     next

        ; CLOSE: no parameters
close:  lda     #NETDEV
        sta     FN_DEV
        lda     #NCCLOSE
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
        PUTS    22, 2, tdone
loop:   jmp     loop
.endproc

; newrow -- the cursor to column 2 of row nrow.
.proc newrow
        lda     #0
        sta     ncol
        lda     nrow
        ldx     #2
        jmp     disp_at
.endproc

; go_ack -- fn_go, then fn_ack; on any failure the code goes in aperr and
; the failure screen follows, so callers need no branch.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla                     ; drop our caller's return
        pla
        jmp     fail
.endproc

; fail -- FAILED and the code. Does not return.
.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
url:    .byte "N:http://127.0.0.1:8765/hello.txt", 0
title:  .byte "NETGET", 0
tdone:  .byte "CLOSED", 0
tfail:  .byte "FAILED: E", 0

```

fujidir.s

```
; fujidir.s -- list a directory on a FujiNet host.
;
```

```
; MOUNT_HOST, then OPEN_DIRECTORY with the path, then READ_DIR_ENTRY until
; the $7F,$7F marker says there are no more (or the screen is full), each
; name shown straight from the reply window, then CLOSE_DIRECTORY.

        .include "fujinet.inc"
        .include "devdefs.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_rpl
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_path
        .export main, nmi

HOSTSL  = 0                     ; host slot 0: the SD card
NAMELN  = 36                    ; READ_DIR_ENTRY's maxlen (never 31: at 31 the
                                ;   firmware puts icon bytes in front)
ROW0    = 4
NROWS   = 18

        .segment "ZEROPAGE"
aperr:  .res 1
row:    .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     mount
        lda     #FNENOC
        sta     aperr
        jmp     fail

        ; MOUNT_HOST: one parameter, the host slot
mount:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMHST
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        jsr     go_ack

        ; OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCODIR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     go_ack

        ; READ_DIR_ENTRY: two one-byte parameters, the longest name wanted
        ; and the flags; one name a call
        lda     #ROW0
        sta     row
entry:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCRDIR
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NAMELN
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY        ; $7F,$7F: the end of the directory
        cmp     #$7F
        bne     @show
        lda     FN_REPLY+1
        cmp     #$7F
        beq     close
@show:  lda     row
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #NAMELN
        jsr     disp_rpl
        inc     row
        lda     row
        cmp     #ROW0+NROWS
        bne     entry

        ; CLOSE_DIRECTORY: no parameters
close:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCCDIR
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
loop:   jmp     loop
.endproc

; go_ack -- fn_go, then fn_ack; on any failure the code goes in aperr and
; the failure screen follows, so callers need no branch.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
path:   .byte "/a7800book/", 0
title:  .byte "DIRECTORY OF SD:/A7800BOOK/", 0
tfail:  .byte "FAILED: E", 0
```

fujiboot.s

```
; fujiboot.s -- M2: mount a cartridge image over the network and boot it.
;
; MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE. The last one makes the
; ESP32 side stream the file back to the cartridge on the same link,
; addressed to the DBC device, while MOUNT_IMAGE's own reply is still
; outstanding; the cart stages it and reports BOOT_STATE. Then BOOTLOCK, and
; the loader at $0600 copies the image into the SRAM and starts it -- through
; the console's own BIOS if this console would start it itself.
;
; Target host slot and path come from bootcfg.inc, written by build.sh:
;   BOOT_HOST=0 BOOT_PATH=/game.bin ./build.sh fujiboot
; BOOT_DIRECT=1 locks INPTCTRL first, which forces the loader's own hand-over.

        .include "fujinet.inc"
        .include "bootcfg.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_dec, disp_putc
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_path, fn_blk, fn_boot
        .export main, nmi

DEVSLOT = 0

        .segment "ZEROPAGE"
aperr:  .res 1
apstep: .res 1
lastpct: .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        lda     #0
        sta     aperr
        sta     apstep
        PUTS    2, 2, title
        PUTS    4, 2, tpath
        lda     #4
        ldx     #8
        jsr     disp_at
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     disp_str
        PUTS    6, 2, tmount
        jsr     disp_on

        jsr     fn_chk
        beq     have
        lda     #FNENOC
        sta     aperr
        jmp     show

; ---- MOUNT_HOST(host) ----
have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMHST
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #BOOTHST
        jsr     fn_pb
        lda     #1
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ----
; Three one-byte parameters and then a FULL 256-byte NUL-padded path.
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCSDFP
        sta     FN_CMD
        lda     #3
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #BOOTHST
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     fn_path
        lda     #2
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- MOUNT_IMAGE(dev, mode) ----
; The one that takes real time: the ESP32 fetches the file and streams it to
; the cart as DBC frames while this reply is outstanding.
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMIMG
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #96                 ; ~67 s: past the cart's 60 s for MOUNT_IMAGE
        sta     FN_TMO
        lda     #3
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- wait for the staged image ----
        lda     #4
        sta     apstep
        ldy     #0
wait1:  ldx     #0
wait2:  lda     FN_BOOTSTATE
        cmp     #FN_BOOT_READY
        bne     :+
```

```
        jmp     boot
:       cmp     #FN_BOOT_FAILED
        bne     :+
        jmp     bfail
:
        dex
        bne     wait2
        dey
        bne     wait1
        lda     #FNEWAIT
        sta     aperr
        jmp     show
bfail:  lda     FN_BOOTERR
        sta     aperr
        lda     #5
        sta     apstep
        jmp     show

; ---- boot it ----
boot:   jsr     fn_blk          ; arm the load
.if ::BOOTDIRECT
        lda     #$07            ; lock INPTCTRL: no BIOS hand-over now
        sta     INPTCTRL
.endif
        jmp     fn_boot         ; the loader; does not return

; ---- the failure screen: WHICH step, with WHAT ----
show:   PUTS    8, 2, tfail
        lda     #8
        ldx     #14
        jsr     disp_at
        lda     apstep
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        lda     #'E'
        jsr     disp_putc
        lda     aperr
        jsr     disp_hex
        jsr     disp_on
hang:   jmp     hang
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:  .byte "FUJINET ATARI 7800 M2", 0
tpath:  .byte "BOOT: ", 0
tmount: .byte "MOUNTING...", 0
tfail:  .byte "FAILED STEP ", 0
bootpath: .byte BOOTPATH, 0
```

appkey.s

```
; appkey.s -- read the shared FujiNet username, then write a key of our own
; and read it back.
;
; An appkey is a small file on the adapter's SD card, named by a creator id,
; an application id and a key number. Creator 1, app 1, key 0 is the slot
; every FujiNet game reads the player's name from. This program shows it,
; then writes "HELLO 7800" into creator $7800, app $FF, key 0 and reads that
; back. ($7800 is the cartridge's own creator; its app 1 is the High Score
; Cart. App $FF is this handbook's scratch key.)
;
; OPEN_APPKEY's payload is ONE six-byte packed struct -- creator (u16, little-
; endian), app, key, mode, and a reserved byte that is not optional -- not
; four parameters. READ_APPKEY's reply is a two-byte length and then the
; value. WRITE_APPKEY takes one parameter, the length, then the bytes.

        .include "fujinet.inc"
        .include "devdefs.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_rpl
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_txb
        .export main, nmi

OURCRE  = $7800
OURAPP  = $FF
OURKEY  = 0

        .segment "ZEROPAGE"
aperr:  .res 1
akcre:  .res 2                  ; the key being opened: creator...
akapp:  .res 1                  ; ...app...
akkey:  .res 1                  ; ...key number...
akmode: .res 1                  ; ...and mode (AKMRD / AKMWR)

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
```

```
        jsr     disp_str
.endmacro

.macro  KEY     cre, app, key, mode
        lda     #<cre
        sta     akcre
        lda     #>cre
        sta     akcre+1
        lda     #app
        sta     akapp
        lda     #key
        sta     akkey
        lda     #mode
        sta     akmode
.endmacro

.proc main
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     @go
        lda     #FNENOC
        sta     aperr
        jmp     fail

@go:    PUTS    5, 2, tname     ; the shared username
        KEY     AKCREAT, AKAPP, AKKEY, AKMRD
        jsr     akopen
        jsr     akread
        lda     #6
        jsr     akshow

        PUTS    9, 2, tours     ; ours: write it, then read it back
        KEY     OURCRE, OURAPP, OURKEY, AKMWR
        jsr     akopen
        jsr     akwrite
        lda     #AKMRD
        sta     akmode
        jsr     akopen
        jsr     akread
        lda     #10
        jsr     akshow
loop:   jmp     loop
.endproc

; akopen -- OPEN_APPKEY for the key in akcre/akapp/akkey, mode akmode: no
; parameters, the six-byte struct as the payload.
.proc akopen
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKOP
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        lda     akcre
        jsr     fn_txb
        lda     akcre+1
        jsr     fn_txb
        lda     akapp
        jsr     fn_txb
        lda     akkey
        jsr     fn_txb
        lda     akmode
        jsr     fn_txb
        lda     #0              ; reserved, and not optional
        jsr     fn_txb
        jmp     go_ack
.endproc

; akread -- READ_APPKEY: the value is left in the reply window, two bytes of
; length, then the bytes.
.proc akread
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKRD
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jmp     go_ack
.endproc

; akwrite -- WRITE_APPKEY: one parameter, the length, then the value.
.proc akwrite
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKWR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #valend-val
        jsr     fn_pb
        ldy     #0
@w:     lda     val,y
        jsr     fn_txb
        iny
        cpy     #valend-val
        bne     @w
        jmp     go_ack
.endproc

; akshow -- the value just read, on row A; "(EMPTY)" if there is none.
.proc akshow
        ldx     #4
        jsr     disp_at
        lda     FN_REPLY
        ora     FN_REPLY+1
        bne     @val
        lda     #<tempty
        sta     FN_PTR
        lda     #>tempty
        sta     FN_PTR+1
        jmp     disp_str
@val:   ldx     #2              ; the value starts after the length word
        ldy     FN_REPLY        ; at most its length (all are under 64)
        jmp     disp_rpl
.endproc

; go_ack -- fn_go, then fn_ack; on any failure, the failure screen.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:  .byte "APPKEYS", 0
tname:  .byte "USERNAME (CREATOR 1, APP 1, KEY 0)", 0
tours:  .byte "OURS (CREATOR $7800, APP $FF, KEY 0)", 0
tempty: .byte "(EMPTY)", 0
tfail:  .byte "FAILED: E", 0
val:    .byte "HELLO 7800"
valend:

```

clock.s

```
; clock.s -- the time of day, from the adapter's clock.
;
; The clock is a device of its own ($45). GET_ISO_LOCAL answers with the
; local time as text -- "2026-10-20T10:45:31" and a NUL -- in the adapter's
; configured time zone, and GET_ISO_UTC the same in UTC. Each goes on the
; screen straight out of the reply window.

        .include "fujinet.inc"
        .include "devdefs.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_rpl
        .import fn_chk, fn_beg, fn_go, fn_ack
        .export main, nmi

        .segment "ZEROPAGE"
aperr:  .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

; ASK cmd, row -- a clock command with no parameters, its text on the row.
.macro  ASK     cmd, row
        lda     #CLKDEV
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
        lda     #row
        ldx     #10
        jsr     disp_at
        ldx     #0
        ldy     #25
        jsr     disp_rpl
.endmacro

.proc main
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     @go
        lda     #FNENOC
        sta     aperr
        jmp     fail
@go:    PUTS    5, 2, tlocal
        ASK     CKISOL, 5
        PUTS    7, 2, tutc
        ASK     CKISOU, 7
loop:   jmp     loop
.endproc

; go_ack -- fn_go, then fn_ack; on any failure, the failure screen.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:  .byte "CLOCK", 0
tlocal: .byte "LOCAL", 0
tutc:   .byte "UTC", 0
tfail:  .byte "FAILED: E", 0
```

fujibank.s

```
; fujibank.s -- M2: a FujiNet app bigger than the slot: SuperGame banking with
; the mailbox live, then it boots a game itself.
;
; Each of three 16K banks, switched into $8000, runs a real transaction from
; its own code and names itself on screen; then the app pushes and boots
; BOOT_PATH as CONFIG would. The claim in the fixed bank says "SuperGame with
; cart RAM" (kind byte = A78MAP_SG_RAM + 1).

        .include "fujinet.inc"
        .include "bootcfg.inc"

        .import disp_at, disp_str, disp_hex, disp_on, disp_rpl, disp_vbl
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_path, fn_blk, fn_boot
        .export main, nmi

AC_SSID = 0

        .segment "ZEROPAGE"
bank:   .res 1
aperr:  .res 1
apstep: .res 1

; In each bank, at $8000: A = the bank's number on return, and the
; transaction's result in aperr.
.macro  BANKCODE n
        .segment .sprintf("BANK%d", n)
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        lda     #n
        rts
.endmacro

        BANKCODE 0
        BANKCODE 1
        BANKCODE 2

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        PUTS    2, 2, title
        jsr     disp_on
        lda     #0
        sta     bank
@b:     lda     bank
        sta     $8000           ; SuperGame: any write to $8000-$BFFF picks the bank
        jsr     $8000           ; the transaction, from the bank's own code
        cmp     bank
        beq     @right
        jmp     wrong
@right: pha
        lda     bank
        asl
        clc
        adc     #5
        ldx     #2
        jsr     disp_at
        lda     #<tbank
        sta     FN_PTR
        lda     #>tbank
        sta     FN_PTR+1
        jsr     disp_str
        pla
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        lda     aperr
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        ldx     #AC_SSID
        ldy     #20
        jsr     disp_rpl
        inc     bank
        lda     bank
        cmp     #3
        beq     @pushed
        jmp     @b
@pushed:
        ldx     #120            ; two seconds to read the banks
@pause: jsr     disp_vbl
        dex
        bne     @pause

        ; push the game and boot it, as CONFIG does
        PUTS    12, 2, tboot
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMHST
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #BOOTHST
        jsr     fn_pb
        lda     #1
        jsr     go_ack
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCSDFP
        sta     FN_CMD
        lda     #3
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #BOOTHST
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     fn_path
        lda     #2
        jsr     go_ack
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMIMG
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #$96            ; ~67 s: past the cart's 60 s for MOUNT_IMAGE
        sta     FN_TMO
        lda     #3
        jsr     go_ack

        lda     #4
        sta     apstep
        ldy     #0
@w1:    ldx     #0
@w2:    lda     FN_BOOTSTATE
        cmp     #FN_BOOT_READY
        beq     @boot
        cmp     #FN_BOOT_FAILED
        beq     @bfail
        dex
        bne     @w2
        dey
        bne     @w1
        lda     #FNEWAIT
        sta     aperr
        jmp     fail
```

```
@bfail: lda     FN_BOOTERR
        sta     aperr
        lda     #5
        sta     apstep
        jmp     fail
@boot:  jsr     fn_blk
        jmp     fn_boot

wrong:  PUTS    12, 2, twrong
@h:     jmp     @h
.endproc

; go_ack -- fn_go and fn_ack as step A; the failure screen if either fails.
.proc go_ack
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     bad
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        bne     bad
        rts
bad:    jmp     fail
.endproc

; fail -- WHICH step, with WHAT. Does not return.
.proc fail
        PUTS    14, 2, tfail
        lda     #14
        ldx     #14
        jsr     disp_at
        lda     apstep
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        lda     #'E'
        jsr     disp_putc
        lda     aperr
        jsr     disp_hex
hang:   jmp     hang
.endproc

        .import disp_putc

.proc nmi
        rti
.endproc

        .segment "RODATA"
title:  .byte "FUJINET ATARI 7800 M2: BANKED APP", 0
tbank:  .byte "BANK ", 0
tboot:  .byte "BOOTING...", 0
twrong: .byte "WRONG BANK ANSWERED", 0
tfail:  .byte "FAILED STEP ", 0
bootpath: .byte BOOTPATH, 0

```

build.sh

```
#!/usr/bin/env bash
# build.sh -- build the assembly programs with cc65's ca65 and ld65, the way
# the bring-up's build.sh builds its clients: the program plus the shared
# start-up (a78init), text (fujidisp) and mailbox library (fujilib) from
# ../common, linked by a78.cfg (a78bank.cfg for a banked program), checked by
# checkrom.py and given the .a78 header MAME needs.
#
#   ./build.sh fujitest netget          -> build/fujitest.bin, build/fujitest.a78, ...
#   BOOT_PATH=/a7800book/hello.bin ./build.sh fujiboot
set -euo pipefail
cd "$(dirname "$0")"
BOOK="$(cd ../.. && pwd)"
mkdir -p build/gen build/obj

python3 "$BOOK/tools/mkfont.py" > build/gen/font.inc
cat > build/gen/bootcfg.inc <<EOC
; GENERATED by build.sh -- do not edit.
BOOTHST = ${BOOT_HOST:-0}
.define BOOTPATH "${BOOT_PATH:-/a7800book/hello.bin}"
BOOTDIRECT = ${BOOT_DIRECT:-0}
EOC

SHARED="a78init fujidisp fujilib"
for name in "$@"; do
    objs=() cfg=../common/a78.cfg defs=()
    src=$name.s
    [ -f "$src" ] || src=../common/$name.s
    if [ "$name" = fujibank ]; then
        cfg=../common/a78bank.cfg
        defs=(-D BANKED)
    fi
    for f in $SHARED; do
        ca65 -I . -I ../common -I build/gen "${defs[@]}" -o "build/obj/$f.o" "../common/$f.s"
        objs+=("build/obj/$f.o")
    done
    ca65 -I . -I ../common -I build/gen "${defs[@]}" -o "build/obj/$name.o" "$src"
    objs+=("build/obj/$name.o")
    ld65 -C "$cfg" -m "build/obj/$name.map" -o "build/$name.bin" "${objs[@]}"
```

```
    python3 "$BOOK/tools/checkrom.py" --claim
  --notia --map "build/obj/$name.map" "build/
  $name.bin"
    python3 "$BOOK/tools/mka78.py" "build/
  $name.bin" "build/$name.a78" "FUJINET $name"
done

```

The C programs

hello.c

```
/* hello.c -- the smallest FujiNet program in
  C: a screen, some text, and
 * proof the cartridge is there. No
  transaction yet. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-atari7800.h>
#include "maria.h"

void main(void)
{
    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);    /* ink
  white, paper = background */
    mt_at(2, 2);
    mt_puts("FUJINET ATARI 7800");
    mt_at(2, 4);
    mt_puts("HELLO, WORLD");
    mt_at(2, 6);
    mt_puts(fuji_a7800_present() ? "A FUJINET
  CARTRIDGE ANSWERS"
                                 : "NO FUJINET
  CARTRIDGE");
    mt_at(2, 7);
    mt_puts(fuji_a7800_mode() ==
  FUJI_A7800_MODE_APP ? "RUNNING AS AN APP" :
  "");
    for (;;)
        mt_sync();
}

```

acfg.c

```
/* acfg.c -- First Contact in C: one
  GET_ADAPTERCONFIG_EXTENDED, and the
 * SSID, IP address and firmware version on
  the screen. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-fuji.h>
#include <fujinet-atari7800.h>
#include "maria.h"

static AdapterConfigExtended ac;

static void hex(uint8_t v)
{
    static const char digit[] =
  "0123456789ABCDEF";
    mt_put(digit[v >> 4]);
    mt_put(digit[v & 15]);
}

static void field(uint8_t row, const char
  *label, const char *s, uint8_t max)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(8, row);
    mt_text(s, max);
}

void main(void)
{
    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);    /* ink
  white, paper = background */
    mt_at(2, 2);
    mt_puts("FUJINET 7800 C FIRST CONTACT");

    if (!fuji_a7800_present()) {
        mt_at(2, 5);
        mt_puts("NO FUJINET CARTRIDGE");
    } else if (!
  fuji_get_adapter_config_extended(&ac)) {
        mt_at(2, 5);
        mt_puts("FAILED: E");
        hex(fn_device_error);
    } else {
        field(5, "SSID", ac.ssid, 24);
        field(7, "IP", ac.sLocalIP, 16);
        field(9, "FW", ac.fn_version, 15);
    }
    for (;;)
        mt_sync();
}

```

netget.c

```
/* netget.c -- fetch a URL through the N: device and show the text.
 *
 * network_open() as an HTTP GET, network_read() what arrives, then
 * network_close(). The text goes up a line at a time: LF ends a row, CR is
 * skipped, a row is at most 30 characters. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-network.h>
#include <fujinet-atari7800.h>
#include "maria.h"

#define URL   "N:http://127.0.0.1:8765/hello.txt"
#define ROW0  4
#define ROWS  18
#define COLS  30

static char buf[512];

static void hex(uint8_t v)
{
    static const char digit[] = "0123456789ABCDEF";
    mt_put(digit[v >> 4]);
    mt_put(digit[v & 15]);
}

static void fail(const char *what, uint8_t code)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED: E");
    hex(code);
    for (;;)
        mt_sync();
}

void main(void)
{
    int16_t n, i;
    uint8_t row = ROW0, col = 0, err;

    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("NETGET");

    network_init();
    err = network_open(URL, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
    if (err)
        fail("OPEN", err);
    n = network_read(URL, buf, sizeof buf);
    if (n < 0)
        fail("READ", fn_device_error);

    mt_at(2, row);
    for (i = 0; i < n; ++i) {
        if (buf[i] == '\r')
            continue;
        if (buf[i] == '\n') {
            if (++row == ROW0 + ROWS)
                break;
            mt_at(2, row);
            col = 0;
        } else if (col < COLS) {
            mt_put(buf[i]);
            ++col;
        }
    }
    network_close(URL);
    mt_at(2, 22);
    mt_puts("CLOSED");
    for (;;)
        mt_sync();
}

```

dir.c

```
/* dir.c -- list a directory on a FujiNet            host.
```

```
 *
 * fuji_mount_host_slot(),
fuji_open_directory() with the path, then
 * fuji_read_directory() until the 0x7F,0x7F
marker (or the screen is
 * full), then fuji_close_directory(). */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari7800.h>
#include "maria.h"

#define HOSTSL  0                       /*
host slot 0: the SD card */
#define NAMELN  36                      /*
never 31: icon bytes come first */
#define ROW0    4
#define ROWS    18

static char path[256];                  /*
OPEN_DIRECTORY takes exactly 256 */
static char name[NAMELN + 1];

static void fail(const char *what)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED");
    for (;;)
        mt_sync();
}

void main(void)
{
    uint8_t row;

    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("DIRECTORY OF SD:/A7800BOOK/");

    strcpy(path, "/a7800book/");
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_open_directory(HOSTSL, path))
        fail("OPEN_DIRECTORY");
    for (row = ROW0; row < ROW0 + ROWS; ++row)
{
        if (!fuji_read_directory(NAMELN, 0,
name))
            fail("READ_DIR_ENTRY");
        if (name[0] == 0x7F && name[1] ==
0x7F)
            break;                      /* the
end of the directory */
        mt_at(2, row);
        mt_text(name, NAMELN);
    }
    fuji_close_directory();
    for (;;)
        mt_sync();
}

```

boot.c

```
/* boot.c -- push a program from the SD card
into the cartridge and run it.
 *
 * MOUNT_HOST, SET_DEVICE_FULLPATH, then
MOUNT_IMAGE, which makes the
 * FujiNet push the image into the cartridge's
RAM. When the cartridge says
 * the image is READY, fuji_a7800_boot() hands
the console to the loader,
 * which copies the image in and starts it. A
bar shows the push. */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari7800.h>
#include "maria.h"

#define HOSTSL   0                      /*
host slot 0: the SD card */
#define DEVSL    0                      /*
device slot 0 */
#define FMREAD   1                      /*
mount for reading */
#define BOOTPATH "/a7800book/hello.bin"

static char path[256];                  /*
SET_DEVICE_FULLPATH takes 256 */

static void fail(const char *what)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED");
    for (;;)
        mt_sync();
}

void main(void)
{
    uint8_t state, pct, shown = 0;

    mt_init(' ');
    mt_background(0x00);
```

```
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("BOOT " BOOTPATH);

    strcpy(path, BOOTPATH);
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_set_device_filename(FMREAD,
HOSTSL, DEVSL, path))
        fail("SET_DEVICE_FULLPATH");
    if (!fuji_mount_disk_image(DEVSL, FMREAD))
        fail("MOUNT_IMAGE");

    /* the push may finish before or after
MOUNT_IMAGE returns */
    mt_at(2, 4);
    for (;;) {
        state = fuji_a7800_boot_state();
        pct = fuji_a7800_boot_percent();
        while (shown < pct / 4) {       /* 25
cells for 100 per cent */
            mt_put('#');
            ++shown;
        }
        if (state == FUJI_A7800_BOOT_READY)
            break;
        if (state == FUJI_A7800_BOOT_FAILED)
            fail("PUSH");
        mt_sync();
    }
    fuji_a7800_boot();                  /*
does not return */
}

```

appkey.c

```
/* appkey.c -- read the shared FujiNet
username, then write a key of our own
 * and read it back.
 *
 * Creator 1, app 1, key 0 is the slot every
FujiNet game reads the player's
 * name from. Ours is creator 0x7800 (the
cartridge's own), app 0xFF (this
 * handbook's scratch key), key 0.
fuji_read_appkey() and
 * fuji_write_appkey() each do the OPEN_APPKEY
themselves. */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari7800.h>
#include "maria.h"

static uint8_t value[64 + 2];           /* a
key, plus 2: the lib's rule */
static const char ours[] = "HELLO 7800";

static void show(uint8_t row)
{
    uint16_t n;

    mt_at(4, row);
    if (!fuji_read_appkey(0, &n, value))
        mt_puts("(READ FAILED)");
    else if (n == 0)
        mt_puts("(EMPTY)");
    else
        mt_text((char *)value, n);
}

void main(void)
{
    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("APPKEYS");

    mt_at(2, 5);
    mt_puts("USERNAME: KEY 1:1:0");
    fuji_set_appkey_details(1, 1, DEFAULT);
    show(6);

    mt_at(2, 9);
    mt_puts("OURS: KEY $7800:$FF:0");
    fuji_set_appkey_details(0x7800, 0xFF,
DEFAULT);
    if (!fuji_write_appkey(0, sizeof ours - 1,
(uint8_t *)ours)) {
        mt_at(4, 10);
        mt_puts("(WRITE FAILED)");
    } else
        show(10);
    for (;;)
        mt_sync();
}

```

clock.c

```
/* clock.c -- the time of day, from the adapter's clock.
 *
 * clock_get_time() with TZ_ISO_STRING answers with the local time as text,
 * "2026-10-20T10:45:31" and more, in the adapter's configured time zone;
 * UTC_ISO_STRING the same in UTC. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-clock.h>
#include <fujinet-atari7800.h>
#include "maria.h"

static uint8_t when[32];

static void line(uint8_t row, const char *label, TimeFormat f)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(10, row);
    if (clock_get_time(when, f))
        mt_puts("FAILED");
    else
        mt_text((char *)when, 25);
}

void main(void)
{
    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("CLOCK");
    line(5, "LOCAL", TZ_ISO_STRING);
    line(7, "UTC", UTC_ISO_STRING);
    for (;;)
        mt_sync();
}
```

bank.c

```
/* bank.c -- a C program bigger than the slot. Three 16K banks take turns at
 * $8000 under SuperGame banking; each makes a real transaction from its own
 * code and names itself on the screen. bank.cfg places the banks, and
 * crt0bank.s's claim tells the cartridge to bank the image that way. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-fuji.h>
#include <fujinet-atari7800.h>
#include "maria.h"

static AdapterConfigExtended ac;

/* SuperGame: a store anywhere in $8000-$BFFF picks the bank shown there. */
#define BANK_SELECT (*(volatile uint8_t *)0x8000)

/* One function per bank, each at $8000 in its own bank. Each returns its
 * bank's number, so a wrong bank shows. */
#pragma code-name(push, "BANK0")
uint8_t bank0(void) { return fuji_get_adapter_config_extended(&ac) ? 0 : 0xFF; }
#pragma code-name(pop)
#pragma code-name(push, "BANK1")
uint8_t bank1(void) { return fuji_get_adapter_config_extended(&ac) ? 1 : 0xFF; }
#pragma code-name(pop)
#pragma code-name(push, "BANK2")
uint8_t bank2(void) { return fuji_get_adapter_config_extended(&ac) ? 2 : 0xFF; }
#pragma code-name(pop)

typedef uint8_t (*bankfn)(void);
static const bankfn entry[3] = { bank0, bank1, bank2 };

/* The call lives in the fixed bank, so it is still there after the switch. */
static uint8_t bank_call(uint8_t b)
{
    BANK_SELECT = b;
    return entry[b]();
}

void main(void)
{
    uint8_t b, got;

    mt_init(' ');
    mt_background(0x00);
    mt_palette(0, 0x0F, 0x00, 0x0F);
    mt_at(2, 2);
    mt_puts("FUJINET 7800 C BANKED APP");
    for (b = 0; b < 3; ++b) {
        got = bank_call(b);
        mt_at(2, 5 + 2 * b);
        mt_puts("BANK ");
        mt_put('0' + b);
        mt_puts(got == b ? "  OK  " : " FAIL");
        mt_text(ac.ssid, 16);
    }
    for (;;)
        mt_sync();
}
```

bank.cfg

```
# bank.cfg -- a 64K banked FujiNet client in C: fujinet-lib's
# atari7800-fujinet.cfg with the ROM split SuperGame's way. Three 16K banks
# take turns at $8000; the last bank sits at $C000 for good and holds the C
# runtime, fujinet-lib, the tile engine, the claim and the vectors. The
# cart's RAM at $4000 holds DATA, BSS, the heap and the C stack, as before.
SYMBOLS {
    __STACKSIZE__: type = weak, value = $0800;
}
MEMORY {
    ZP:      file = "", define = yes, start = $0040, size = $00C0, type = rw;
    CONSOLE: file = "", define = yes, start = $1800, size = $0840, type = rw;
    CRAM:    file = "", define = yes, start = $4000, size = $4000 - __STACKSIZE__, type = rw;
    STACK:   file = "", define = yes, start = $8000 - __STACKSIZE__, size = __STACKSIZE__, type = rw;
    BANK0:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    BANK1:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    BANK2:   file = %O, start = $8000, size = $4000, type = ro, fill = yes, fillval = $FF;
    ROM:     file = %O, define = yes, start = $C000, size = $3F70, type = ro, fill = yes, fillval = $FF;
    CLAIM:   file = %O, start = $FF70, size = $000A, type = ro, fill = yes, fillval = $00;
    SIG:     file = %O, start = $FF7A, size = $007E, type = ro, fill = yes, fillval = $FF;
    VEC:     file = %O, start = $FFF8, size = $0008, type = ro;
}
SEGMENTS {
    ZEROPAGE: load = ZP,              type = zp;
    EXTZP:    load = ZP,              type = zp,  optional = yes;
    BANK0:    load = BANK0,           type = ro;
    BANK1:    load = BANK1,           type = ro;
    BANK2:    load = BANK2,           type = ro;
    TILES:    load = ROM,             type = ro,  optional = yes, align = $100, define = yes;
    STARTUP:  load = ROM,             type = ro,  define = yes;
    LOWCODE:  load = ROM,             type = ro,  optional = yes;
    ONCE:     load = ROM,             type = ro,  optional = yes;
    CODE:     load = ROM,             type = ro,  define = yes;
    RODATA:   load = ROM,             type = ro,  define = yes;
    VIDEO:    load = CRAM,            type = bss, optional = yes, align = $100, define = yes;
    DATA:     load = ROM, run = CRAM, type = rw,  define = yes;
    BSS:      load = CRAM,            type = bss, define = yes;
    CONSOLE:  load = CONSOLE,         type = bss, optional = yes, define = yes;
    CLAIM:    load = CLAIM,           type = ro;
    VECTORS:  load = VEC,             type = ro;
}
FEATURES {
    CONDES: type    = constructor,
            label   = __CONSTRUCTOR_TABLE__,
            count   = __CONSTRUCTOR_COUNT__,
            segment = ONCE;
    CONDES: type    = destructor,
            label   = __DESTRUCTOR_TABLE__,
            count   = __DESTRUCTOR_COUNT__,
            segment = RODATA;
    CONDES: type    = interruptor,
            label   = __INTERRUPTOR_TABLE__,
            count   = __INTERRUPTOR_COUNT__,
            segment = RODATA,
            import  = __CALLIRQ__;
}
```

crt0bank.s

```
; crt0bank.s -- fujinet-lib's atari7800 crt0 (bus/atari7800/crt0.s), with
; one byte changed: the claim names the bank scheme, SuperGame with cart RAM
; (kind byte = A78MAP_SG_RAM + 1), so the cartridge banks the image itself.
;
; crt0.s -- startup for a FujiNet Atari 7800 client, in place of cc65's.
;
; cc65's own locks INPTCTRL ($07 to $01); this one never stores to $00-$1F,
; so INPTCTRL stays unlocked and a game can later be handed to the
; console's BIOS. The cart's RAM at $4000 is there because the claim below
; asks for it.

        .export         __STARTUP__ : absolute = 1
        .export         _exit
        .export         _fuji_a7800_nmi
        .import         __STACK_START__, __STACK_SIZE__
        .import         initlib, donelib, zerobss, copydata
        .import         push0, _main

        .include        "zeropage.inc"

CTRL    = $3C
OFFSET  = $38

        .segment "STARTUP"
start:  sei
        cld
        lda     #$7F            ; MARIA's DMA off until the program builds a display
        sta     CTRL
        lda     #0
        sta     OFFSET
        ldx     #$FF
        txs
        lda     #<(__STACK_START__ + __STACK_SIZE__)
        sta     c_sp
        lda     #>(__STACK_START__ + __STACK_SIZE__)
        sta     c_sp+1

        jsr     zerobss
        jsr     copydata
        lda     #$4C            ; JMP rti_only; no NMI comes before a display is on
        sta     _fuji_a7800_nmi
        lda     #<rti_only
        sta     _fuji_a7800_nmi+1
        lda     #>rti_only
        sta     _fuji_a7800_nmi+2
        jsr     initlib
        jsr     push0           ; argc
        jsr     push0           ; argv
        ldy     #4
        jsr     _main
_exit:  jsr     donelib
@hang:  jmp     @hang

rti_only:
        rti

        .segment "BSS"
_fuji_a7800_nmi:
        .res    3               ; JMP to a display-list interrupt handler

        .segment "CLAIM"
        .byte   "FUJI", 1, 5, 1, 0, 0, 0    ; version 1, SuperGame + RAM, cart RAM

        .segment "VECTORS"
        .byte   $FF, $87                    ; $FFF8/$FFF9: a 7800 cart
        .word   _fuji_a7800_nmi, start, rti_only

```

build.sh

```
#!/usr/bin/env bash
# build.sh -- build the C programs: cc65's atari7800 target, fujinet-lib's
# atari7800 target (its crt0, linker config and romstamp), and the MARIA
# tile engine CONFIG and the games use.
#
#   ./build.sh acfg           -> build/acfg.bin (the cart's) and build/acfg.a78 (MAME's)
#   CFG=bank.cfg ./build.sh bank
#
# FNLIB is fujinet-lib with the atari7800 target built (`make atari7800`).
set -euo pipefail
cd "$(dirname "$0")"
BOOK="$(cd ../.. && pwd)"
FNLIB="${FNLIB:-$HOME/Workspace/fnlib-7800}"
LIB="$FNLIB/r2r/atari7800/fujinet.atari7800.lib"
CFG="${CFG:-$FNLIB/makefiles/atari7800-fujinet.cfg}"
CFLAGS=(-t atari7800 -O --cpu 6502 -I"$FNLIB/include" -I"$FNLIB/bus/atari7800")

mkdir -p build
[ build/tiles.s -nt font.txt ] || python3 mktiles.py font.txt build/tiles.s >/dev/null
cl65 -t atari7800 -c -o build/tiles.o build/tiles.s
cl65 -t atari7800 -c -o build/maria.o maria.s

for p in "$@"; do
    objs=()
    for src in "$p".c ${EXTRA:-}; do
        o="build/$(basename "${src%.*}").o"
        cl65 "${CFLAGS[@]}" -c -o "$o" "$src"
        objs+=("$o")
    done
    cl65 -t atari7800 -C "$CFG" -m "build/$p.map" -o "build/$p.bin" \
        "${objs[@]}" build/maria.o build/tiles.o "$LIB"
    if [ "$(stat -c %s "build/$p.bin")" -le 49152 ]; then
        python3 "$FNLIB/makefiles/atari7800-romstamp.py" --stamp --map "build/$p.map" "build/$p.bin"
        python3 "$BOOK/tools/checkrom.py" --claim --notia --map "build/$p.map" "build/$p.bin"
    else                                  # banked: romstamp takes 48K at most
        python3 "$BOOK/tools/checkbanks.py" --map "build/$p.map" "build/$p.bin"
    fi
    python3 "$BOOK/tools/mka78.py" "build/$p.bin" "build/$p.a78" "$p"
done

```

The 7800basic library and programs

fujinet.bas

```
rem fujinet.bas -- the FujiNet cartridge's mailbox, named for 7800basic.
rem incbasic it first; incbasic fujilib.bas after the main loop.
rem
rem In BASIC a register's name is its address: FN_CMD = $C4 is one store,
rem STA $0D01. Use these names only on the left of a plain assignment
rem or the right of a comparison, never in arithmetic: FN_TX = FN_TX + 1
rem would read the page back and write twice.

rem cells the cartridge paints: read them
dim FN_REPLY = $0800
dim FN_ACKSEQ = $0C00
dim FN_STATUS = $0C01
dim FN_ERR = $0C02
dim FN_REPLYCMD = $0C03
dim FN_RXLEN = $0C04
dim FN_RXLENHI = $0C05
dim FN_BOOTSTATE = $0C06
dim FN_BOOTPCT = $0C07
dim FN_BOOTERR = $0C08
dim FN_MAGIC0 = $0C09
dim FN_MAGIC1 = $0C0A
dim FN_MODE = $0C12
dim FN_INPTLOCK = $0C1D

rem registers it watches: write them, one store each
dim FN_DEVICE = $0D00
dim FN_CMD = $0D01
dim FN_NPARAM = $0D02
dim FN_DATA_RST = $0D05
dim FN_SEQ = $0D10
dim FN_BOOTLOCK = $0D11
dim FN_TX = $0F00
```

```
33
34  rem the library's own variables
35  dim fn_dev = var90
36  dim fn_cmd = var91
37  dim fn_npr = var92
38  dim fn_val = var93
39  dim fn_valhi = var94
40  dim fn_res = var95
41  dim fn_seq = var96
42  dim fn_tick = var97
43  dim fn_tmo = var98
44  dim fn_i = var99
45  dim fn_hexbuf = $27FE
46
47  rem devices, and the commands these programs
    use
48  const FN_FUJI = $70
49  const FN_NET = $71
50  const FN_CLOCK = $45
51  const FN_OK = 0
52  const FN_ENOCART = $FD
53  const FN_ENAK = $EE
54  const FN_EWAIT = $FF
55  const FN_BOOT_READY = 2
56  const FN_BOOT_FAILED = $80
57
```

fujilib.bas

```
 1  rem fujilib.bas -- the mailbox subroutines
    for 7800basic, fujilib.s's
 2  rem one for one. incbasic it after the main
    loop, where it is never
 3  rem fallen into; call each with gosub.
 4
 5  rem fn_chk -- fn_res = 0 if a FujiNet
    cartridge answers, else FN_ENOCART.
 6  fn_chk
 7  fn_res = FN_ENOCART
 8  if FN_MAGIC0 <> $46 then return
 9  if FN_MAGIC1 <> $4E then return
10  fn_res = FN_OK
11  return
12
13  rem fn_beg -- begin the transaction in
    fn_dev, fn_cmd, fn_npr. The TX
14  rem stream is rewound last, so parameters can
    follow at once. Sets the
15  rem timeout to 3 (about 13 seconds); raise
    fn_tmo after this for a
16  rem slow command.
17  fn_beg
18  FN_DEVICE = fn_dev
19  FN_CMD = fn_cmd
20  FN_NPARAM = fn_npr
21  FN_DATA_RST = 0
22  fn_tmo = 3
23  return
24
25  rem fn_pb -- append a one-byte parameter,
    fn_val.
26  fn_pb
27  FN_TX = 1
28  FN_TX = fn_val
29  return
30
31  rem fn_pw -- append a two-byte parameter,
    fn_val low and fn_valhi high.
32  fn_pw
33  FN_TX = 2
34  FN_TX = fn_val
35  FN_TX = fn_valhi
36  return
37
38  rem fn_pad -- append NULs from byte fn_i
    until fn_i wraps: after a path
39  rem of fn_i bytes, this makes it exactly 256.
40  fn_pad
41  FN_TX = 0
42  fn_i = fn_i + 1
43  if fn_i <> 0 then goto fn_pad
44  return
45
46  rem fn_go -- launch it and wait, drawing the
    screen meanwhile. fn_res is
47  rem 0 (FN_OK), the cartridge's error, FN_ENAK
    if the server said no, or
48  rem FN_EWAIT if nothing came back in fn_tmo x
    256 frames. The next
49  rem sequence number is always the cartridge's
    ACKSEQ + 1, never a
50  rem counter of ours: 0 is reserved, so 255
    wraps to 1.
51  fn_go
52  fn_seq = FN_ACKSEQ + 1
53  if fn_seq = 0 then fn_seq = 1
54  FN_SEQ = fn_seq
55  fn_tick = 0
56  fn_go_wait
57  if FN_ACKSEQ = fn_seq then goto fn_go_done
58  drawscreen
59  fn_tick = fn_tick + 1
60  if fn_tick <> 0 then goto fn_go_wait
61  fn_tmo = fn_tmo - 1
62  if fn_tmo <> 0 then goto fn_go_wait
63  fn_res = FN_EWAIT
64  return
65  fn_go_done
66  fn_res = FN_ERR
67  if fn_res <> FN_OK then return
68  if FN_REPLYCMD <> $06 then fn_res = FN_ENAK
69  return
70
71  rem fn_hex -- fn_val as two hex digits in
    fn_hexbuf, for plotchars.
72  fn_hex
73  fn_i = fn_val / 16
74  fn_hexbuf[0] = fn_hexdigits[fn_i]
75  fn_i = fn_val & 15
76  fn_hexbuf[1] = fn_hexdigits[fn_i]
77  return
78
79  alphadata fn_hexdigits fujifont
80  '0123456789ABCDEF'
81  end
82
```

hello.bas

```
rem hello.bas -- the smallest FujiNet program in 7800basic: a screen,
rem some text, and proof the cartridge is there. No transaction yet.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

clearscreen
plotchars 'FUJINET ATARI 7800' 0 8 2
plotchars 'HELLO, WORLD' 0 8 4
gosub fn_chk
if fn_res <> FN_OK then goto nocart
plotchars 'A FUJINET CARTRIDGE ANSWERS' 0 8 6
if FN_MODE = 3 then plotchars 'RUNNING AS AN APP' 0 8 7
goto main
nocart
plotchars 'NO FUJINET CARTRIDGE' 0 8 6

main
drawscreen
goto main

incbasic fujilib.bas

```

acfg.bas

```
rem acfg.bas -- First Contact in 7800basic: one GET_ADAPTERCONFIG_EXTENDED,
rem and the SSID, IP address and firmware version on the screen.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

rem two of the reply's fields, by offset: firmware 125, IP text 140
dim FN_R_FW = $087D
dim FN_R_IP = $088C
rem console RAM for the strings, where MARIA can read them
dim ssid = $2200
dim ipaddr = $2220
dim fwver = $2230
dim ix = var0

clearscreen
plotchars 'FUJINET 7800BASIC FIRST CONTACT' 0 8 1
gosub fn_chk
if fn_res <> FN_OK then goto failed

fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
gosub fn_beg
gosub fn_go
if fn_res <> FN_OK then goto failed

for ix = 0 to 31 : ssid[ix] = FN_REPLY[ix] : next
for ix = 0 to 15 : ipaddr[ix] = FN_R_IP[ix] : next
for ix = 0 to 14 : fwver[ix] = FN_R_FW[ix] : next
plotchars 'SSID' 0 8 4
plotchars ssid 0 40 4 24
plotchars 'IP' 0 8 6
plotchars ipaddr 0 40 6 16
plotchars 'FW' 0 8 8
plotchars fwver 0 40 8 15
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

incbasic fujilib.bas

```

netget.bas

```
rem netget.bas -- fetch a URL through the N: device and show the text.
rem OPEN as an HTTP GET, STATUS until the count settles, READ it (224
rem bytes at most), then CLOSE. The text goes into a 32 x 7 map in
rem console RAM a line at a time: LF ends a row, CR is skipped.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

dim scr = $2200
dim avail = var0
dim prev = var1
dim tries = var2
dim src = var3
dim pos = var4
dim ch = var5
dim ix = var6

rem the map starts blank: 224 spaces
for ix = 0 to 223 : scr[ix] = 32 : next
clearscreen
plotchars 'NETGET' 0 8 2
plotmap scr 0 8 4 32 7
gosub fn_chk
if fn_res <> FN_OK then goto failed

rem OPEN: the mode (12, HTTP GET) and the translation (0, none) as
rem one-byte parameters, then the devicespec as the payload
fn_dev = FN_NET : fn_cmd = $4F : fn_npr = 2
gosub fn_beg
fn_val = 12 : gosub fn_pb
fn_val = 0 : gosub fn_pb
ix = 0
open_url
FN_TX = url[ix]
ix = ix + 1
if ix < url_length then goto open_url
fn_tmo = 22
gosub fn_go
if fn_res <> FN_OK then goto failed

rem STATUS until two readings agree and are not zero
prev = $FF : tries = 0
status
fn_dev = FN_NET : fn_cmd = $53 : fn_npr = 2
gosub fn_beg
fn_val = 0 : gosub fn_pb : gosub fn_pb
fn_tmo = 22
gosub fn_go
if fn_res <> FN_OK then goto failed
avail = FN_REPLY[0]
if FN_REPLY[1] <> 0 then avail = 224
if avail > 224 then avail = 224
if avail <> prev then goto status_again
if avail <> 0 then goto read
status_again
prev = avail
tries = tries + 1
if tries = 30 then fn_res = FN_EWAIT : goto failed
drawscreen : drawscreen : drawscreen
goto status

rem READ: ONE two-byte parameter, the count
read
fn_dev = FN_NET : fn_cmd = $52 : fn_npr = 1
gosub fn_beg
fn_val = avail : fn_valhi = 0 : gosub fn_pw
fn_tmo = 22
gosub fn_go
if fn_res <> FN_OK then goto failed

rem into the map, a row per line
src = 0 : pos = 0
show
if src = FN_RXLEN then goto close
ch = FN_REPLY[src]
src = src + 1
if ch = 13 then goto show
if ch <> 10 then goto show_char
pos = pos & $E0
pos = pos + 32
if pos = 224 then goto close
goto show
show_char
ix = pos & 31
if ix = 31 then goto show
scr[pos] = ch
pos = pos + 1
goto show

rem CLOSE: no parameters
close
fn_dev = FN_NET : fn_cmd = $43 : fn_npr = 0
gosub fn_beg
gosub fn_go
plotchars 'CLOSED' 0 8 22
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

alphadata url fujifont
'N:http://127.0.0.1:8765/hello.txt'
end

incbasic fujilib.bas
```

dir.bas

```
rem dir.bas -- list a directory on a FujiNet host.
rem MOUNT_HOST, OPEN_DIRECTORY with the path, READ_DIR_ENTRY until the
rem $7F,$7F marker (or eight names), CLOSE_DIRECTORY. The names go into
rem a 32 x 8 map in console RAM.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

const HOSTSL = 0
const NAMELN = 30
const NAMELAST = 29
dim names = $2200
dim pos = var0
dim ix = var1

for ix = 0 to 255 : names[ix] = 32 : next
clearscreen
plotchars 'DIRECTORY OF SD:/A7800BOOK/' 0 8 2
plotmap names 0 8 4 32 8
gosub fn_chk
if fn_res <> FN_OK then goto failed

rem MOUNT_HOST: one parameter, the host slot
fn_dev = FN_FUJI : fn_cmd = $F9 : fn_npr = 1
gosub fn_beg
fn_val = HOSTSL : gosub fn_pb
gosub fn_go
if fn_res <> FN_OK then goto failed

rem OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
fn_dev = FN_FUJI : fn_cmd = $F7 : fn_npr = 1
gosub fn_beg
fn_val = HOSTSL : gosub fn_pb
fn_i = 0
open_path
FN_TX = path[fn_i]
fn_i = fn_i + 1
if fn_i < path_length then goto open_path
gosub fn_pad
gosub fn_go
if fn_res <> FN_OK then goto failed

rem READ_DIR_ENTRY: the longest name wanted, and the flags
pos = 0
entry
fn_dev = FN_FUJI : fn_cmd = $F6 : fn_npr = 2
gosub fn_beg
fn_val = NAMELN : gosub fn_pb
fn_val = 0 : gosub fn_pb
gosub fn_go
if fn_res <> FN_OK then goto failed
if FN_REPLY[0] <> $7F then goto entry_show
if FN_REPLY[1] = $7F then goto close
entry_show
for ix = 0 to NAMELAST
  if FN_REPLY[ix] = 0 then goto entry_next
  names[pos] = FN_REPLY[ix]
  pos = pos + 1
next
entry_next
pos = pos & $E0
pos = pos + 32
if pos <> 0 then goto entry

rem CLOSE_DIRECTORY: no parameters
close
fn_dev = FN_FUJI : fn_cmd = $F5 : fn_npr = 0
gosub fn_beg
gosub fn_go
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

alphadata path fujifont
'/a7800book/'
end

incbasic fujilib.bas
```

boot.bas

```
rem boot.bas -- push a program from the SD card into the cartridge and
rem run it. MOUNT_HOST, SET_DEVICE_FULLPATH, then MOUNT_IMAGE, which makes
rem the FujiNet push the image into the cartridge's RAM. When BOOT_STATE
rem says READY, arm BOOTLOCK and jump to the loader at $0600.
incbasic fujinet.bas

rem leave INPTCTRL unlocked, so a game can still be handed to the
rem console's own BIOS, and so keep every TIA register untouched
const NOTIALOCK = 1
const NOTIALOCKMUTE = 1

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

const HOSTSL = 0
const DEVSL = 0
const FMREAD = 1
dim bar = $2200
dim ix = var0
dim shown = var1

for ix = 0 to 24 : bar[ix] = 32 : next
clearscreen
plotchars 'BOOT /A7800BOOK/HELLO.BIN' 0 8 2
plotchars bar 0 8 4 25
gosub fn_chk
if fn_res <> FN_OK then goto failed

rem MOUNT_HOST: the host slot
fn_dev = FN_FUJI : fn_cmd = $F9 : fn_npr = 1
gosub fn_beg
fn_val = HOSTSL : gosub fn_pb
gosub fn_go
if fn_res <> FN_OK then goto failed

rem SET_DEVICE_FULLPATH: device slot, host slot, mode, then 256 of path
fn_dev = FN_FUJI : fn_cmd = $E2 : fn_npr = 3
gosub fn_beg
fn_val = DEVSL : gosub fn_pb
fn_val = HOSTSL : gosub fn_pb
fn_val = FMREAD : gosub fn_pb
fn_i = 0
set_path
FN_TX = path[fn_i]
fn_i = fn_i + 1
if fn_i < path_length then goto set_path
gosub fn_pad
gosub fn_go
if fn_res <> FN_OK then goto failed

rem MOUNT_IMAGE: device slot and mode. The cartridge allows it 60
rem seconds; wait a little longer than that.
fn_dev = FN_FUJI : fn_cmd = $F8 : fn_npr = 2
gosub fn_beg
fn_val = DEVSL : gosub fn_pb
fn_val = FMREAD : gosub fn_pb
fn_tmo = 16
gosub fn_go
if fn_res <> FN_OK then goto failed

rem the push may finish before or after MOUNT_IMAGE answers; a cell of
rem the bar for every 4 per cent
shown = 0
push
ix = FN_BOOTPCT / 4
push_bar
if shown >= ix then goto push_state
bar[shown] = 35
shown = shown + 1
goto push_bar
push_state
if FN_BOOTSTATE = FN_BOOT_READY then goto go
if FN_BOOTSTATE = FN_BOOT_FAILED then fn_res = FN_BOOTERR : goto failed
drawscreen
goto push

go
FN_BOOTLOCK = $B5
asm
  sei
  lda #$7F
  sta CTRL
  jmp $0600
end

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

alphadata path fujifont
'/a7800book/hello.bin'
end

incbasic fujilib.bas
```

appkey.bas

```
rem appkey.bas -- read the shared FujiNet username, then write a key of
rem our own and read it back. Creator 1, app 1, key 0 is the player's
rem name; ours is creator $7800, app $FF, key 0. OPEN_APPKEY takes ONE
rem six-byte struct as its payload: creator (low, high), app, key, mode,
rem and a reserved zero.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

rem READ_APPKEY's value starts after its two-byte length
dim FN_R_VALUE = $0802
dim line1 = $2200
dim line2 = $2220
dim akcrelo = var0
dim akcrehi = var1
dim akapp = var2
dim akmode = var3
dim ix = var4
dim len = var5
dim last = var6

for ix = 0 to 63 : line1[ix] = 32 : next
clearscreen
plotchars 'APPKEYS' 0 8 2
plotchars 'USERNAME: KEY 1:1:0' 0 8 5
plotchars line1 0 16 6 30
plotchars 'OURS: KEY $7800:$FF:0' 0 8 9
plotchars line2 0 16 10 30
gosub fn_chk
if fn_res <> FN_OK then goto failed

akcrelo = 1 : akcrehi = 0 : akapp = 1 : akmode = 0
gosub akopen
if fn_res <> FN_OK then goto failed
gosub akread
if fn_res <> FN_OK then goto failed
len = FN_REPLY[0]
if len > 30 then len = 30
if len = 0 then goto ours
last = len - 1
for ix = 0 to last : line1[ix] = FN_R_VALUE[ix] : next

ours
akcrelo = $00 : akcrehi = $78 : akapp = $FF : akmode = 1
gosub akopen
if fn_res <> FN_OK then goto failed
rem WRITE_APPKEY: one parameter, the length, then the bytes
fn_dev = FN_FUJI : fn_cmd = $DE : fn_npr = 1
gosub fn_beg
fn_val = value_length : gosub fn_pb
last = value_length - 1
for ix = 0 to last : FN_TX = value[ix] : next
gosub fn_go
if fn_res <> FN_OK then goto failed
akmode = 0
gosub akopen
if fn_res <> FN_OK then goto failed
gosub akread
if fn_res <> FN_OK then goto failed
last = FN_REPLY[0] - 1
for ix = 0 to last : line2[ix] = FN_R_VALUE[ix] : next
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

rem akopen -- OPEN_APPKEY: no parameters, the struct as the payload
akopen
fn_dev = FN_FUJI : fn_cmd = $DC : fn_npr = 0
gosub fn_beg
FN_TX = akcrelo
FN_TX = akcrehi
FN_TX = akapp
FN_TX = 0
FN_TX = akmode
FN_TX = 0
gosub fn_go
return

rem akread -- READ_APPKEY: the reply is the length (2 bytes), then the value
akread
fn_dev = FN_FUJI : fn_cmd = $DD : fn_npr = 0
gosub fn_beg
gosub fn_go
return

alphadata value fujifont
'HELLO 7800'
end

incbasic fujilib.bas

```

clock.bas

```
rem clock.bas -- the time of day, from the adapter's clock ($45).
rem GET_ISO_LOCAL ('I') answers with the local time as text, and
rem GET_ISO_UTC ('Z') the same in UTC.
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
incgraphic fujifont.png 320A
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

dim local = $2200
dim utc = $2220
dim ix = var0

clearscreen
plotchars 'CLOCK' 0 8 2
plotchars 'LOCAL' 0 8 5
plotchars local 0 40 5 25
plotchars 'UTC' 0 8 7
plotchars utc 0 40 7 25
gosub fn_chk
if fn_res <> FN_OK then goto failed

fn_dev = FN_CLOCK : fn_cmd = $49 : fn_npr = 0
gosub fn_beg
gosub fn_go
if fn_res <> FN_OK then goto failed
for ix = 0 to 24 : local[ix] = FN_REPLY[ix] : next

fn_dev = FN_CLOCK : fn_cmd = $5A : fn_npr = 0
gosub fn_beg
gosub fn_go
if fn_res <> FN_OK then goto failed
for ix = 0 to 24 : utc[ix] = FN_REPLY[ix] : next
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
drawscreen
goto main

incbasic fujilib.bas

```

bank.bas

```
rem bank.bas -- a 7800basic program bigger than the slot. With romsize
rem 128k the ROM is eight 16K banks, SuperGame's way: the last, bank 8,
rem sits at $C000 for good, and the others take turns at $8000. Banks 2
rem and 3 each make a real transaction from their own code; the mailbox
rem subroutines live in bank 8, where every bank can reach them.
set romsize 128k
incbasic fujinet.bas

displaymode 320A
set zoneheight 8
characterset fujifont
alphachars ASCII
BACKGRND = $00
P0C2 = $0F

dim ssid1 = $2200
dim ssid2 = $2220
dim ssid3 = $2240
dim ix = var0

for ix = 0 to 95 : ssid1[ix] = 32 : next
clearscreen

plotchars 'FUJINET 7800BASIC BANKED APP' 0 8 2
plotchars 'BANK 1' 0 8 5
plotchars ssid1 0 40 5 20
plotchars 'BANK 2' 0 8 7
plotchars ssid2 0 40 7 20
plotchars 'BANK 3' 0 8 9
plotchars ssid3 0 40 9 20
gosub fn_chk
if fn_res <> FN_OK then goto failed

gosub acfg
if fn_res <> FN_OK then goto failed
for ix = 0 to 19 : ssid1[ix] = FN_REPLY[ix] : next
gosub fetch2 bank2
if fn_res <> FN_OK then goto failed
gosub fetch3 bank3
if fn_res <> FN_OK then goto failed
goto main

failed
fn_val = fn_res
gosub fn_hex
plotchars 'FAILED: E' 0 8 22
plotchars fn_hexbuf 0 44 22 2

main
```

```
 drawscreen
 goto main

 bank 2
fetch2
 gosub acfg
 if fn_res <> FN_OK then return otherbank
 for ix = 0 to 19 : ssid2[ix] = FN_REPLY[ix] :
next
 return otherbank

 bank 3
fetch3
 gosub acfg
 if fn_res <> FN_OK then return otherbank
 for ix = 0 to 19 : ssid3[ix] = FN_REPLY[ix] :
next
 return otherbank

 bank 8
 rem the font lives here too: MARIA reads it
every line, and a font in
 rem bank 1 would turn to garbage while bank 2
or 3 is switched in
 incgraphic fujifont.png 320A

 rem acfg -- one GET_ADAPTERCONFIG_EXTENDED,
from whichever bank calls
acfg
 fn_dev = FN_FUJI : fn_cmd = $C4 : fn_npr = 0
 gosub fn_beg
 gosub fn_go
 return

 incbasic fujilib.bas

```

build.sh

```
#!/usr/bin/env bash
# build.sh -- build a 7800basic program as a
FujiNet app.
#
#   ./build.sh acfg            -> build/
acfg.a78
#
# The 7800basic steps are
7800basic.native.sh's, run directly (that
script
# runs the compiler under valgrind); then
 tools/basfix.py stamps the FujiNet
# claim at $FF70 and checks what the cartridge
needs checked, and 7800header
# writes the .a78 header for the bank scheme
7800basic chose.
set -euo pipefail
cd "$(dirname "$0")"
HERE="$PWD"
BOOK="$(cd ../.. && pwd)"
export bas7800dir="${bas7800dir:-$HOME/
Workspace/7800basic}"
export PATH="$bas7800dir:$PATH"

for p in "$@"; do
    B="build/$p"
    rm -rf "$B"; mkdir -p "$B"
    cp "$p.bas" "$B/"
    cp ./*.png ./fujinet.bas ./fujilib.bas
"$B/"
    cp ./*.asm "$B/" 2>/dev/null || true
    (
        cd "$B"
        7800preprocess < "$p.bas" >
"$p.bas.pre"
        7800basic -i "$bas7800dir" -b "$p.bas"
-p "$p.bas.pre" > "$p.log"
        7800postprocess -i "$bas7800dir" >
"$p.bas.asm"
        dasm "$p.bas.asm" -I"$bas7800dir/
includes" -f3 -l"$p.bas.list.txt" -p20 \
            -s"$p.bas.symbol.txt" -
o"$p.bas.bin" | 7800filter | tee dasm.log
    )
    python3 "$BOOK/tools/basfix.py" "$B/
$p.bas.bin" "$B/$p.bas.list.txt" "$B/dasm.log"
    (cd "$B" && 7800header -o -f a78info.cfg
"$p.bas.bin" > /dev/null)
    cp "$B/$p.bas.a78" "build/$p.a78"
done

```

basfix.py

```
#!/usr/bin/env python3
"""basfix.py -- make a 7800basic ROM a FujiNet app.

    basfix.py prog.bas.bin prog.bas.list.txt dasm.log

7800basic fills its last 4K upward from $F000 and keeps $FF7E-$FFFF for its
checksum, the signature and the vectors. The FujiNet claim is ten bytes at
$FF70: "FUJI", version 1, kind+1 (0 = let the cartridge decide), flags
($01 = 16K of cart RAM at $4000), three zeros. This tool

  - refuses if 7800basic's code reaches $FF70 (dasm's output says how many
    bytes of the reserved area are left below $FF7E),
  - stamps the claim,
  - and refuses any read-modify-write instruction aimed at $0D00-$0FFF, read
    from dasm's listing so graphics are never mistaken for code.

The image is stamped in place; 7800header then writes the .a78 header, with
the bank scheme 7800basic chose.
"""
import re
import sys

CLAIM = 0xFF70
WR_LO, WR_HI = 0x0D00, 0x0FFF
RMW = {0x0E: "ASL", 0x1E: "ASL", 0x2E: "ROL", 0x3E: "ROL", 0x4E: "LSR",
       0x5E: "LSR", 0x6E: "ROR", 0x7E: "ROR", 0xCE: "DEC", 0xDE: "DEC",
       0xEE: "INC", 0xFE: "INC"}


def main():
    binp, listp, logp = sys.argv[1:4]
    img = bytearray(open(binp, "rb").read())
    if len(img) % 1024:
        sys.exit("basfix: %s is %d bytes, not whole KiB" % (binp, len(img)))

    m = re.search(r"(\d+) bytes left in the 7800basic reserved area", open(logp).read())
    if not m:
        sys.exit("basfix: %s does not say where 7800basic's code ends" % logp)
    last = 0xFF7E - int(m.group(1)) - 1

    # Every listing line that assembled bytes:  "  line  addr  bytes  source".
    bad = []
    rx = re.compile(r"^\s*\d+\s+([0-9a-f]{4,5})\s+((?:[0-9a-f]{2}\s)+)\s*(\S.*)?$", re.I)
    for line in open(listp, errors="replace"):
        m = rx.match(line.rstrip("\n"))
        if not m:
            continue
        code = [int(b, 16) for b in m.group(2).split()]
        src = (m.group(3) or "").strip().lower()
        if len(code) == 3 and code[0] in RMW and not src.startswith((".byte", "dc", ".word", "hex")):
            tgt = code[1] | code[2] << 8
            if WR_LO <= tgt <= WR_HI:
                bad.append("%s: %s $%04X writes the old value first -- the cartridge "
                           "counts both writes" % (m.group(1), RMW[code[0]], tgt))
    for msg in bad:
        print("basfix: %s %s" % (binp, msg), file=sys.stderr)
    if bad:
        sys.exit(1)

    at = len(img) - (0x10000 - CLAIM)  # the last bank is at the top
    if last >= CLAIM or any(img[at + i] not in (0, 0xFF) for i in range(10)):
        sys.exit("basfix: 7800basic's code reaches $%04X; no room for the claim at $FF70" % last)
    img[at:at + 10] = b"FUJI" + bytes([1, 0, 0, 0, 0, 0])
    vec = img[-4] | img[-3] << 8
    if vec < 0x4000:
        sys.exit("basfix: reset vector $%04X is below $4000" % vec)
    open(binp, "wb").write(img)
    print("basfix: %s: claim at $FF70, code ends at $%04X, %d bytes" % (binp, last, len(img)))


if __name__ == "__main__":
    main()
```

mkbasfont.py

```
#!/usr/bin/env python3
"""mkbasfont.py -- the assembly programs' font as a 7800basic 320A graphic.

7800basic takes its characters from a PNG (`incgraphic fujifont.png 320A`):
one 8 x 8 cell per character, side by side, colour index 1 the ink. With
`alphachars ASCII` a character's code is its cell, so this writes the 128
cells of the bring-up's own 5 x 7 set (tools/mkfont.py), codes $00-$7F, and
the BASIC programs look like the assembly ones.

    mkbasfont.py out.png
"""
import os
import struct
import sys
import zlib

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from mkfont import rows  # noqa: E402

N = 128


def chunk(kind, data):
    c = kind + data
    return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))


def main():
    glyph = [rows(chr(c)) if 0x20 <= c < 0x80 else [0] * 8 for c in range(N)]
    raw = b""
    for r in range(8):
        raw += b"\x00" + bytes(glyph[c][r] for c in range(N))   # filter 0, 1 bpp
    png = (b"\x89PNG\r\n\x1a\n"
           + chunk(b"IHDR", struct.pack(">IIBBBBB", N * 8, 8, 1, 3, 0, 0, 0))
           + chunk(b"PLTE", bytes([0, 0, 0, 255, 255, 255]))
           + chunk(b"IDAT", zlib.compress(raw))
           + chunk(b"IEND", b""))
    open(sys.argv[1], "wb").write(png)


if __name__ == "__main__":
    main()
```

checkbanks.py

```
#!/usr/bin/env python3
"""checkbanks.py -- checkrom.py's rules for a SuperGame image built by ld65.

    checkbanks.py --map prog.map prog.bin

fujinet-lib's romstamp takes flat images of at most 48K, and checkrom.py reads
an image top-aligned, so neither can see code in a switched bank: every bank
runs at $8000 but lives at its own place in the file. This scans each BANKn
segment the map lists at file offset n * $4000, and the fixed bank's code
segments at their own addresses, with checkrom.py's scanner (no RMW on the
mailbox's write pages, no store to $00-$1F, no JMP ($xxFF)), then runs
checkrom.py itself for the claim and the vectors.
"""
import os
import re
import subprocess
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from checkrom import scan, CODE_SEGS  # noqa: E402


def main():
    args = sys.argv[1:]
    mapfile = args[args.index("--map") + 1]
    path = [a for a in args if not a.startswith("--")][0]
    img = open(path, "rb").read()
    if len(img) % 0x4000:
        sys.exit("checkbanks: %s is not whole 16K banks" % path)
    text = open(mapfile).read()
    seglist = re.search(r"Segment list:\s*\n-+\s*\n(.*?)\n\n", text, re.S).group(1)
    bad = []
    for line in seglist.splitlines():
        parts = line.split()
        if len(parts) < 4 or not re.fullmatch(r"[0-9A-Fa-f]{6}", parts[1]):
            continue
        name, lo, hi = parts[0], int(parts[1], 16), int(parts[2], 16)
```

```
        m = re.fullmatch(r"BANK(\d+)", name)
        if m:
            off = int(m.group(1)) * 0x4000 + (lo - 0x8000)
        elif name in CODE_SEGS:
            off = len(img) - (0x10000 - lo)
        else:
            continue
        bad += scan(img[off:], hi + 1 - lo, lo, "%s $%04X-$%04X" % (name, lo, hi), True)
    for b in bad:
        print("checkbanks: %s: %s" % (path, b), file=sys.stderr)
    if bad:
        sys.exit(1)
    rc = subprocess.call([sys.executable, os.path.join(os.path.dirname(__file__), "checkrom.py"),
                          "--claim", "--notia", "--map", mapfile, path])
    sys.exit(rc)


if __name__ == "__main__":
    main()
```

emu/run.sh

```
#!/usr/bin/env bash
# run.sh -- run a 7800 image in the FujiNet MAME cartridge model.
#
#   emu/run.sh build/asm/fujitest.a78 interactive
#   emu/run.sh build/asm/fujitest.a78 shot headless, driven by emu/shot.lua
#   SYSTEM=a7800p emu/run.sh x.a78   a PAL console (default a7800, NTSC)
#   THROTTLE=1 emu/run.sh x.a78 script headless, but in real time
#
# fujinet-pc must already be serving bus-over-IP on 127.0.0.1:9986 (the
# book's private copy; FUJINET_TCP overrides), and only one MAME may talk to
# it at a time. MAME is the single-driver build with the cart grafted in
# ($MAME, default ~/Workspace/mame-a7800, binary ./a7800); it must run from
# its own tree or -autoboot_script is ignored.
set -euo pipefail
HERE="$(cd "$(dirname "$0")/.." && pwd)"
ROM="$(realpath "${1:?usage: run.sh image.a78 [script]}")"
SCRIPT="${2:-}"

MAME="${MAME:-$HOME/Workspace/mame-a7800}"
BIN="${MAMEBIN:-./a7800}"
ROMS="${ROMS:-$HOME/Workspace/mame/roms}"
SYSTEM="${SYSTEM:-a7800}"
SNAP="${SNAP:-$HERE/build/snap}"

[ -x "$MAME/$BIN" ] || { echo "run.sh: no $BIN in $MAME" >&2; exit 1; }
export A78_EMU_DIR="$HERE/emu"
export FUJINET_TCP="${FUJINET_TCP:-127.0.0.1:9986}"
[ -n "${IMAGE:-}" ] && export FUJINET_IMAGE="$IMAGE"
[ -n "${HSC:-}" ] && export FUJINET_HSC="$HSC"
[ -n "${DISPLAY:-}${WAYLAND_DISPLAY:-}" ] || export SDL_VIDEODRIVER=dummy
mkdir -p "$SNAP" "$HERE/build/nvram" "$HERE/build/cfg"

# one client per bus-over-IP listener: a stale MAME holds the port
pkill -f "^[.]/a7800 $SYSTEM .*-cart $HERE/" 2>/dev/null && sleep 1 || true

args=("$SYSTEM" -rompath "$ROMS" -cartslot fujinet -cart "$ROM"
      -snapshot_directory "$SNAP" -nvram_directory "$HERE/build/nvram"
      -cfg_directory "$HERE/build/cfg")
[ -n "${BIOS:-}" ] && args+=(-bios "$BIOS")
if [ -n "$SCRIPT" ]; then
    args+=(-autoboot_script "$HERE/emu/$SCRIPT.lua"
           -video none -sound none -seconds_to_run "${SECS:-120}")
    # a game server counts real seconds: THROTTLE=1 runs at the console's speed
    [ -n "${THROTTLE:-}" ] || args+=(-nothrottle)
else
    args+=(-window -nomax)
fi
cd "$MAME"
exec "$BIN" "${args[@]}"

```

```
-- screen.lua -- read the screen the way MARIA does, and check it.
--
-- Write taps catch where the program points MARIA's display-list list
-- (DPPH $2C, DPPL $30) and its CTRL ($3C). At SHOT_AT seconds the script
-- walks the DLL zone by zone and every display list in it, and turns each
-- INDIRECT object (character text: 320A one-byte, or 320B two-byte as the
-- tile engine uses) into text at its column: so the same reader serves the
-- assembly programs' fujidisp, the C programs' tile engine and 7800basic's
-- plotchars and plotmap. Codes 32-126 are text (two-byte codes are halved).
--
-- Prints the cartridge's status line and the screen, snapshots it, and with
-- TEXT_WANT="ONE|TWO|..." passes only if every piece appears on some row
-- (case does not matter: the assembly font has no lower case).
if _G.__screen then return end
_G.__screen = { dll = 0, ctrl = 0, done = false }
local S = _G.__screen
local at = tonumber(os.getenv("SHOT_AT") or "6")
local want = os.getenv("TEXT_WANT")
local cpu = manager.machine.devices[":maincpu"]
local mem = cpu.spaces["program"]
local function b(a) return mem:read_u8(a & 0xFFFF) end

S.tap = mem:install_write_tap(0x0020, 0x003F, "fn_screen", function(off, data)
  if off == 0x2C then S.dll = (S.dll & 0x00FF) | (data << 8)
  elseif off == 0x30 then S.dll = (S.dll & 0xFF00) | data
  elseif off == 0x3C then S.ctrl = data end
end)

-- one display list: { {col, text}, ... }
local function objects(dl)
  local out, a = {}, dl
  for _ = 1, 64 do
    local m = b(a + 1)
    if b(a) == 0 and m == 0 then break end
    if m & 0x5F == 0x40 then
  -- five-byte header
      local lo, hi, pw, hp = b(a), b(a + 2), b(a + 3), b(a + 4)
      if m & 0x20 ~= 0 then
  -- indirect: characters
        local w = 32 - (pw & 0x1F)
        if pw & 0x1F == 0 then w = 32 end
        local two = (S.ctrl & 0x10) ~= 0
  -- two-byte characters
        local s = {}
        for i = 0, w - 1 do
          local c = b((hi << 8 | lo) + i)
          if two then c = c >> 1 end
          s[#s + 1] = (c >= 32 and c < 127) and string.char(c) or " "
        end
        local cols = two and 1 or 1
        out[#out + 1] = { (hp // 4) * cols, table.concat(s) }
      end
      a = a + 5
    elseif m == 0 then break
    else a = a + 4 end
  -- four-byte header: graphics
  end
  return out
end

local function screen()
  local rows, y = {}, 0
  for z = 0, 40 do
    local e = S.dll + 3 * z
    local h = (b(e) & 0x0F) + 1
    local dl = b(e + 1) << 8 | b(e + 2)
    local objs = objects(dl)
    if #objs > 0 then
      local line = {}
      for _, o in ipairs(objs) do
        local col, txt = o[1], o[2]
        while #line < col do line[#line + 1] = " " end
        for i = 1, #txt do line[col + i] = txt:sub(i, i) end
      end
      for i = 1, #line do line[i] = line[i] or " " end
      local s = table.concat(line):gsub("%s+$", "")
      if s ~= "" then rows[#rows + 1] = string.format("%3d %s", y, s) end
    end
    y = y + h
    if y >= 262 then break end
  end
  return rows
end

S.sub = emu.add_machine_frame_notifier(function()
  if S.done or manager.machine.time:as_double() < at then return end
  S.done = true
  print(string.format("STATUS magic=%c%c%c mode=%d ackseq=%d err=%d replycmd=$%02X rxlen=%d inptlock=%d",
    b(0x0C09), b(0x0C0A), b(0x0C12), b(0x0C00), b(0x0C02), b(0x0C03),
    b(0x0C04) | b(0x0C05) << 8, b(0x0C1D)))
  local rows = screen()
  for _, r in ipairs(rows) do print(r) end

manager.machine.screens[":screen"]:snapshot()
  if want then
    local all = table.concat(rows, "\n"):upper()
    local missing = {}
    for piece in want:gmatch("[^|]+") do
      if not all:find(piece:upper(), 1, true) then missing[#missing + 1] = piece end
    end
    print(#missing == 0 and "VERDICT PASS" or ("VERDICT FAIL: missing " .. table.concat(missing, ", ")))
  end
  manager.machine:exit()
end)

```

emu/verify.sh

```
#!/usr/bin/env bash
# verify.sh -- run every program in the FujiNet MAME cartridge model against
# the live fujinet-pc and check what each puts on the screen.
#
#   emu/verify.sh                  all 24
#   emu/verify.sh c/netget         one
#
# Needs: fujinet-pc serving bus-over-IP on $FUJINET_TCP (default
# 127.0.0.1:9986) with its SD card at $FNPC_SD, and the netget fixture served
# on 127.0.0.1:8765 (python3 -m http.server 8765 --bind 127.0.0.1
# --directory listings/fixtures). Results also go to build/verify.txt.
set -uo pipefail
cd "$(dirname "$0")/.."
BOOK="$PWD"
FNPC_SD="${FNPC_SD:-$HOME/Workspace/fnpc-7800-a78/SD}"

# The SD card: the fixtures, and the programs the dir and boot programs see.
mkdir -p "$FNPC_SD"
cp -r listings/fixtures/sd/. "$FNPC_SD/"
cp listings/asm/build/hello.bin "$FNPC_SD/a7800book/hello.bin"
cp listings/asm/build/netget.bin "$FNPC_SD/a7800book/netget.bin"

# program | seconds | text that must be on the screen
TABLE="
asm/hello    | 4  | HELLO, WORLD
asm/fujitest | 6  | DUMMY CAFE|127.0.0.1|ERR:00
asm/netget   | 8  | HELLO FROM|OVER HTTP|CLOSED
asm/fujidir  | 8  | HELLO.BIN|NETGET.BIN|README.TXT
asm/fujiboot | 15 | HELLO, WORLD|M0: BOOT BLOCK
asm/appkey   | 8  | PLAYER 1|HELLO 7800
asm/clock    | 6  | LOCAL   20|UTC     20
asm/fujibank | 4  | BANK 00 00 DUMMY CAFE|BANK 01 00 DUMMY CAFE|BANK 02 00 DUMMY CAFE
c/hello      | 4  | HELLO, WORLD|A FUJINET CARTRIDGE ANSWERS|RUNNING AS AN APP
c/acfg       | 6  | DUMMY CAFE|127.0.0.1
c/netget     | 8  | HELLO FROM|OVER HTTP|CLOSED
c/dir        | 8  | HELLO.BIN|NETGET.BIN|README.TXT
c/boot       | 15 | HELLO, WORLD|M0: BOOT BLOCK
c/appkey     | 8  | PLAYER 1|HELLO 7800
c/clock      | 6  | LOCAL   20|UTC     20
c/bank       | 6  | BANK 0  OK  DUMMY CAFE|BANK 1  OK  DUMMY CAFE|BANK 2  OK  DUMMY CAFE
bas/hello    | 4  | HELLO, WORLD|A FUJINET CARTRIDGE ANSWERS|RUNNING AS AN APP
bas/acfg     | 6  | DUMMY CAFE|127.0.0.1
bas/netget   | 8  | HELLO FROM|OVER HTTP|CLOSED
bas/dir      | 8  | HELLO.BIN|NETGET.BIN|README.TXT
bas/boot     | 15 | HELLO, WORLD|M0: BOOT BLOCK
bas/appkey   | 8  | PLAYER 1|HELLO 7800
bas/clock    | 6  | LOCAL   20|UTC     20
bas/bank     | 6  | BANK 1  DUMMY CAFE|BANK 2  DUMMY CAFE|BANK 3  DUMMY CAFE
"

mkdir -p build
: > build/verify.txt
pass=0 fail=0
while IFS='|' read -r prog secs want; do
    prog=$(echo "$prog" | xargs)
    [ -n "$prog" ] || continue
    [ $# -eq 0 ] || [[ " $* " == *" $prog "* ]] || continue
    secs=$(echo "$secs" | xargs)
    want=$(echo "$want" | sed 's/^ *//; s/ *$//')
    lang=${prog%%/*} name=${prog#*/}
    out=$(SHOT_AT=$secs SECS=$((secs + 20)) TEXT_WANT="$want" \
          SNAP="$BOOK/build/snap/$lang-$name" \
          timeout 150 emu/run.sh "listings/$lang/build/$name.a78" screen 2>&1)
    v=$(echo "$out" | grep -m1 "^VERDICT" || echo "VERDICT FAIL: no verdict")
    s=$(echo "$out" | grep -m1 "^STATUS" || true)
    printf '%-14s %s   %s\n' "$prog" "$v" "$s" | tee -a build/verify.txt
    if [[ $v == "VERDICT PASS"* ]]; then pass=$((pass + 1)); else
        fail=$((fail + 1)); echo "$out" | grep -E "^ *[0-9]+ " | sed 's/^/    /'; fi
done <<< "$TABLE"
echo "verify: $pass passed, $fail failed" | tee -a build/verify.txt
[ $fail -eq 0 ]
```

D Battleship Listing

The 7800 layer of Fuji Battleship, `src/atari7800/` at the game's commit `87b8cc7`, in the order Section 20 discusses it. The shared game (`src/main.c`, `gamelogic.c`, `screens.c`, `misc.c`, `stateclient.c`) is every port's and is not printed; the tile engine is in Section 15.

fujinet.c

```
#ifdef BUILD_ATARI7800

/**
 * @brief   FujiNet calls - Atari 7800 (CUSTOM_FUJINET_CALLS, and the lobby boot)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * Everything that talks to the cartridge through fujinet-lib is in this one
 * file, because it has to see the LIBRARY's <fujinet-fuji.h>: as on the NES,
 * the atari7800 target implements most fuji_* calls as macros over its bus
 * call, so the prototypes in src/fujinet-fuji.h link against nothing. This
 * file therefore includes none of the shared headers (src/misc.h,
 * src/platform-specific/), and the shared code reaches the cartridge through
 * the CUSTOM_FUJINET_CALLS hooks instead.
 */

#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <fujinet-fuji.h>
#include <fujinet-network.h>
#include <fujinet-atari7800.h>
#include "vars.h"
#include "maria.h"

// graphics.c - declared here rather than through ../platform-specific, which
// would bring in src/fujinet-fuji.h
void drawText(uint8_t x, uint8_t y, const char *s);
void drawSpace(uint8_t x, uint8_t y, uint8_t w);

#define FUJI_HOST_SLOT_COUNT 8
#define LOBBY_DEVICE_SLOT 0
#define LOBBY_MODE_READ 1

int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    // Show what has been drawn (palette changes wait for a frame) before
    // the transaction holds the 6502.
    mt_sync();

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}

int16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char *destination)
{
    uint16_t read = 0;

    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
        read = 0;
    return read;
}

void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count, char *data)
{
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, count, (uint8_t *)data);
}

static void quitStatus(const char *s)
{
    drawSpace(0, HEIGHT - 1, WIDTH);
    drawText((WIDTH - (unsigned char)strlen(s)) / 2, HEIGHT - 1, s);
    mt_sync();
}

static bool sameHost(const char *a, const char *b)
{
    while (*a && *b)
    {
        if ((*a | 0x20) != (*b | 0x20))
            return false;
        a++;
        b++;
    }
    return *a == *b;
}

/*
  SET_DEVICE_FULLPATH is a fixed 256-byte payload, so the path is padded
  here rather than at the call site. The path is CONFIG's lobby.
*/
static const char lobbyPath[MAX_FILENAME_LEN] = "/atari7800/lobby.a78";
static const char lobbyHost[] = "ec.tnfs.io";
static HostSlot slots[FUJI_HOST_SLOT_COUNT];

static uint8_t findLobbyHost(void)
{
    uint8_t i;

    if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
        return FUJI_HOST_SLOT_COUNT;

    for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
        if (sameHost(lobbyHost, (const char *)slots[i]))
            return i;

    return FUJI_HOST_SLOT_COUNT;
}

// CONFIG's wait: the cart gives a push 60 seconds.
#define LOBBY_FRAMES 3900u

void quit(void)
{
```

```
    uint8_t slot, state, pct = 0xFF;
    uint16_t frames = 0;
    char pctText[5];

    quitStatus("LOADING LOBBY...");

    if (!fuji_a7800_present())
    {
        quitStatus("NO FUJINET CARTRIDGE");
        return;
    }

    // The slot is used as found, never created: a player who came through
    // the FujiNet Lobby already has the host.
    slot = findLobbyHost();
    if (slot == FUJI_HOST_SLOT_COUNT)
    {
        quitStatus("ADD EC.TNFS.IO IN CONFIG");
        return;
    }

    if (!fuji_mount_host_slot(slot)
        || !fuji_set_device_filename(LOBBY_MODE_READ, slot, LOBBY_DEVICE_SLOT, (char *)lobbyPath)
        || !fuji_mount_disk_image(LOBBY_DEVICE_SLOT, LOBBY_MODE_READ))
    {
        quitStatus("LOBBY NOT AVAILABLE");
        return;
    }

    // Some FujiNets push the image after answering MOUNT_IMAGE, so watch the
    // cart's own progress rather than trust the reply.
    for (;;)
    {
        state = fuji_a7800_boot_state();
        if (state == FUJI_A7800_BOOT_READY)
            break;
        if (state == FUJI_A7800_BOOT_FAILED || ++frames > LOBBY_FRAMES)
        {
            quitStatus("LOBBY LOAD FAILED");
            return;
        }
        if (fuji_a7800_boot_percent() != pct)
        {
            pct = fuji_a7800_boot_percent();
            utoa(pct, pctText, 10);
            strcat(pctText, "%");
            drawText(25, HEIGHT - 1, pctText);
        }
        mt_sync();
    }

    // Hand the console to the cartridge's loader. It does not return.
    fuji_a7800_boot();
}

#endif /* BUILD_ATARI7800 */

```

vars.h

```
#ifdef BUILD_ATARI7800

#ifndef KEYMAP_H
#define KEYMAP_H

#include <joystick.h>

// Screen dimensions for platform: the MARIA engine's 32x24 (src/atari7800/maria.h),
// the ColecoVision and NES layout.
#define WIDTH 32
#define HEIGHT 24

#define GAMEOVER_PROMPT_Y HEIGHT - 2

// Icons - msdos tile sheet indices; graphics.c maps them through sheetTile[]
#define ICON_TEXT_CURSOR 0x3A
#define ICON_PLAYER 0x2A
#define ICON_MARK 0x2B
#define ICON_MARK_ALT 0x19
#define ICON_ACTIVE_PLAYER 0x05

/* readJoystick() (src/atari7800/input.c) returns the stick only, in cc65's
   joystick bits. The buttons never appear in the joystick byte: input.c
   reports them as key codes behind kbhit()/cgetc(), as on the NES. */
#undef JOY_BTN_1
#undef JOY_BTN_2
#define JOY_BTN_1(v) ((v) & 0)
#define JOY_BTN_2(v) ((v) & 0)

/**
 * Platform specific key map for common input
 */

// No keyboard: a joystick and the console's switches. Direction comes from
```

```
// the stick, so the arrow codes are unreachable placeholders for
// readCommonInput()'s switch.
#define KEY_LEFT_ARROW      0xF1
#define KEY_LEFT_ARROW_2    0xF2
#define KEY_LEFT_ARROW_3    0xF3

#define KEY_RIGHT_ARROW     0xF4
#define KEY_RIGHT_ARROW_2   0xF5
#define KEY_RIGHT_ARROW_3   0xF6

#define KEY_UP_ARROW        0xF7
#define KEY_UP_ARROW_2      0xF8
#define KEY_UP_ARROW_3      0xF9

#define KEY_DOWN_ARROW      0xFA
#define KEY_DOWN_ARROW_2    0xFB
#define KEY_DOWN_ARROW_3    0xFC

#define KEY_RETURN 0x0D
#define KEY_ESCAPE 0x1B
#define KEY_ESCAPE_ALT 0x03
#define KEY_SPACEBAR 0x20
#define KEY_BACKSPACE 0x08

// What the shared screens call the escape key
#define ESCAPE "PAUSE"
#define ESC "PAUSE"

// Countdown can reach two digits
#define TIMER_WIDTH 2

// Name entry happens on src/atari7800/osk.c's on-screen keyboard.
#define USE_PLATFORM_NAME_ENTRY 1

// The buttons stand in for the letter shortcuts the other ports type.
// src/atari7800/input.c maps them; keep the two in step. Button 2 is a
// ProLine's right button, or RESET for a one-button stick.
//   1 (fire)        RETURN   join / fire / place
//   2 or RESET      'r'      refresh the table list, rotate a ship
//   PAUSE           ESCAPE   the in-game menu
//   SELECT          'n'      change name (on release, alone)
//   SELECT+1        'h'      how to play
//   SELECT+2        's'      sound on/off
//   SELECT+PAUSE    'q'      quit
// Capitals draw in red, each group a palette run, and a row holds at most 12
// runs (maria.h MT_MAXRUNS): keep these to a few capital groups.
#define TABLE_STATUS_TEXT "2:Refresh SEL:Name SEL+1:Help"
#define MENU_QUIT_TEXT "  SEL+PAUSE: quit"
#define MENU_HELP_TEXT "  SEL+1: help"
#define MENU_SOUND_ON_TEXT "  SEL+2: sound ON "
#define MENU_SOUND_OFF_TEXT "  SEL+2: sound OFF"
#define ROTATE_PROMPT_TEXT "press 2/RESET to rotate"
#define READY_PROMPT_TEXT "press FIRE when ready"
#define MENU_CLOSE_TEXT "press FIRE to close"

#endif /* KEYMAP_H */

#endif /* BUILD_ATARI7800 */

```

graphics.c

```
#ifdef BUILD_ATARI7800

/*
  Graphics functionality - Atari 7800 MARIA

  The shared text and tile engine (maria.h): 32x24 cells, each a tile and a
  palette select. The layout and the drawing calls are the NES port's
  (src/nes/graphics.c), drawn from the same MS-DOS sheet; src/atari7800/mkchr.py
  fits it into the engine's 128 tiles by letting the two palettes do what the
  NES did with extra glyph sets (see its header).

  Each palette change starts a new run in the row's display list, and a row
  holds at most MT_MAXRUNS. So cells keep runs long: text sets its whole
  string, white-only art leaves the cell's palette alone, and drawBoard()
  turns the fields and drawers to the art palette before anything lands in
  them.
*/

```

```c
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include "vars.h"
#include "tiles.h"
#include "maria.h"
#include "pad.h"
#include "../misc.h"

/* MARIA colours, hue << 4 | luminance: the MS-DOS client's CGA palette 1,
   high intensity. A PAL console's hues sit one higher. */
#define C_BLUE     0x82
#define C_CYAN     0xAC
#define C_RED      0x46
#define C_WHITE    0x0F
#define PAL_HUE    0x10

/* The whole screen, tiles then palettes, for saveScreenBuffer(). */
static unsigned char saved[2][HEIGHT * WIDTH];

/**
 * @brief Top left of each playfield quadrant (32x24 layout)
 */
static const unsigned char quadrant_offset[4][2] =
    {
        {5, 12}, // bottom left
        {5, 1},  // Top left
        {17, 1}, // top right
        {17, 12} // bottom right
    };

/**
 * @brief offset of legends for each player
 */
static const signed char legendShipOffset[5][2] =
    {
        {2, 0},
        {1, 0},
        {0, 0},
        {0, 5},
        {1, 6},
    };

/**
 * @brief Horizontal Field offset
 */
static unsigned char fieldX = 0;

/**
 * @brief Number of active players
 */
static unsigned char playerCount = 0;

/**
 * @brief set n cells from column x, row y to palette pal
 */
static void paint(unsigned char x, unsigned char y, unsigned char n, unsigned char pal)
{
    mt_at(x, y);
    mt_setpal(pal, n);
}

/**
 * @brief put tile t in palette pal (or AGNOSTIC: leave it) at column x, row y
 */
static void put(unsigned char x, unsigned char y, unsigned char t, unsigned char pal)
{
    if (x >= WIDTH || y >= HEIGHT)
        return;

    mt_at(x, y);
    mt_put(t);
    if (pal != AGNOSTIC && mt_attr[y * WIDTH + x] != pal)
        paint(x, y, 1, pal);
}

/**
 * @brief plot two sheet tiles merged at column x, row y. Used for the one row
 * the top and bottom board borders share on a 24-row screen (msdos has 25).
 * The merged bitmaps are pre-built by mkchr.py; k is the PAIRS index.
 */
static void plotTilePair(unsigned char k, unsigned char add, unsigned char x, unsigned char y)
{
    put(x, y, pairTile[k], add ? PAL_ART : PAL_TEXT);
}

#define PAIR_CORNER_L_BOT 0 /* (0x08,0x0A) */
#define PAIR_CORNER_L_TOP 1 /* (0x0A,0x08) */
#define PAIR_EDGE_BOT 2     /* (0x27,0x29) */
#define PAIR_EDGE_TOP 3     /* (0x29,0x27) */
#define PAIR_CORNER_R_BOT 4 /* (0x09,0x0B) */
#define PAIR_CORNER_R_TOP 5 /* (0x0B,0x09) */

/**
 * @brief The font tile for c: the font is at its ASCII codes, upper case
 * only, and anything it lacks is a blank
 */
static unsigned char glyph(char c)
{
    if (c >= 'a' && c <= 'z')
        return (unsigned char)(c - ('a' - 'A'));
    if (c < ' ' || c >= 0x60)
        return T_BLANK;
    return (unsigned char)c;
}

/**
 * @brief plot name text on its plate. The font has no plate-coloured glyphs,
 * so the name is white text in a window cut into the plate, spaces included:
 * one run per name, however many words it has.
 */
static void plotName(unsigned char x, unsigned char y, const char *s)
{
    char c;

    while (c = *s++)
        put(x++, y, glyph(c), PAL_TEXT);
}

/**
 * @brief Clear screen to the blue field
 */
void resetScreen(void)
{
    mt_clear(T_BLANK);
}

/**
 * @brief cycle to next color palette
 */
unsigned char cycleNextColor()
{
    return 0;
}

/**
 * @brief Initialize Graphics mode
 */
void initGraphics()
{
    unsigned char hue;

    initInput();
    mt_init(T_BLANK);
    hue = mt_pal ? PAL_HUE : 0;
    mt_background(C_BLUE + hue);
    mt_palette(PAL_TEXT, C_WHITE, C_RED + hue, C_WHITE);
    mt_palette(PAL_ART, C_RED + hue, C_CYAN + hue, C_WHITE);
}

/**
 * @brief Reset graphics mode - nothing to reset, the cart never exits
 */
void resetGraphics(void)
{
}

/**
 * @brief Store screen buffer - the engine's map is the screen, so keep a copy
 */
bool saveScreenBuffer()
{
    memcpy(saved[0], mt_map, sizeof saved[0]);
```

```
    memcpy(saved[1], mt_attr, sizeof saved[1]);
    return true;
}

void restoreScreenBuffer()
{
    unsigned char x, y, n;
    unsigned int i = 0;

    for (y = 0; y < HEIGHT; y++, i += WIDTH)
    {
        mt_at(0, y);
        for (x = 0; x < WIDTH; x++)
            mt_put(saved[0][i + x] >> 1);

        // One setpal per run of a palette
        for (x = 0; x < WIDTH; x += n)
        {
            for (n = 1; x + n < WIDTH && saved[1][i + x + n] == saved[1][i + x]; n++)
                ;
            paint(x, y, n, saved[1][i + x]);
        }
    }
}

/**
 * @brief Text output, clipped at the screen edges. No wrap: the shared code
 * centres some strings wider than 32 columns, which start at x = 255.
 */
void drawText(unsigned char x, unsigned char y, const char *s)
{
    char c;
    while (c = *s++)
        put(x++, y, glyph(c), PAL_TEXT);
}

void drawTextAlt(unsigned char x, unsigned char y, const char *s)
{
    char c;
    while (c = *s++)
        put(x++, y, glyph(c), c >= 'A' && c <= 'Z' ? PAL_ART : PAL_TEXT);
}

/**
 * @brief draw icon (tile) from the msdos sheet
 */
void drawIcon(unsigned char x, unsigned char y, unsigned char icon)
{
    put(x, y, sheetTile[icon], sheetPal[icon]);
}

void drawBlank(unsigned char x, unsigned char y)
{
    put(x, y, T_BLANK, AGNOSTIC);
}

void drawSpace(unsigned char x, unsigned char y, unsigned char w)
{
    while (w--)
        put(x++, y, T_BLANK, AGNOSTIC);
}

void drawClock(void)
{
    drawIcon(WIDTH - 1, HEIGHT - 1, 0x1D);
}

void drawConnectionIcon(bool show)
{
    if (show)
    {
        drawIcon(0, HEIGHT - 1, 0x1E);
        drawIcon(1, HEIGHT - 1, 0x1F);
    }
    else
    {
        drawBlank(0, HEIGHT - 1);
        drawBlank(1, HEIGHT - 1);
    }
}

/**
 * @brief true when this player's board shares its middle border row (row 11)
 * with the board above/below it, which happens whenever that counterpart is
 * actually drawn
 */
static bool sharedBorderRow(unsigned char player)
{
    if (player == 0)
        return playerCount >= 2;
    if (player == 2)
        return playerCount == 4;
    return true; // 1 always has 0 below; 3 only exists in 4-player
}

/**
 * @brief Draw Player Name plate and board chrome
 */
void drawPlayerName(unsigned char player, const char *name, bool active)
{
    uint8_t x = quadrant_offset[player][0] - 1;
    uint8_t y = quadrant_offset[player][1] - 1;
    uint8_t add = active ? 0x00 : 0x80;
    uint8_t i = 0;
    bool shared = sharedBorderRow(player);

    x += fieldX;

    if (player == 0 || player == 3)
    {
        // Bottom player boards

        // Thin horizontal border (row 11, shared with the top board's lower
        // border when both are on screen - merge the two tile sets)
        if (shared)
        {
            plotTilePair(PAIR_CORNER_L_BOT, add, x, y);
            for (i = 1; i < 11; i++)
            {
                plotTilePair(PAIR_EDGE_BOT, add, x + i, y);
            }
            plotTilePair(PAIR_CORNER_R_BOT, add, x + 11, y);
        }
        else
        {
            drawIcon(x, y, 0x08 + add);
            for (i = 1; i < 11; i++)
            {
                drawIcon(x + i, y, 0x27 + add);
            }
            drawIcon(x + 11, y, 0x09 + add);
        }

        // Name label
        drawIcon(x, y + 11, 0x5E + add);
        for (i = 1; i < 11; i++)
        {
            drawIcon(x + i, y + 11, 0x60 + add);
        }
        drawIcon(x + 11, y + 11, 0x5F + add);
        plotName(x + 2, y + 11, name);

        // Active indicator
        if (active)
            drawIcon(x + 1, y + 11, 0x5B);

        // Bottom border below name label
        drawIcon(x, y + 12, 0x20 + add);
        for (i = 1; i < 11; i++)
        {
            drawIcon(x + i, y + 12, 0x28 + add);
        }
        drawIcon(x + 11, y + 12, 0x21 + add);
    }
    else
    {
        // Top player boards

        // Name label
        drawIcon(x, y, 0x5C + add);
```

```
        for (i = 1; i < 11; i++)
        {
            drawIcon(x + i, y, 0x60 + add);
        }
        drawIcon(x + 11, y, 0x5D + add);
        plotName(x + 2, y, name);

        // Active indicator
        if (active)
            drawIcon(x + 1, y, 0x5B);

        // Thin Horizontal Border (row 11, see above)
        if (shared)
        {
            plotTilePair(PAIR_CORNER_L_TOP, add, x, y + 11);
            for (i = 1; i < 11; i++)
            {
                plotTilePair(PAIR_EDGE_TOP, add, x + i, y + 11);
            }
            plotTilePair(PAIR_CORNER_R_TOP, add, x + 11, y + 11);
        }
        else
        {
            drawIcon(x, y + 11, 0x0A + add);
            for (i = 1; i < 11; i++)
            {
                drawIcon(x + i, y + 11, 0x29 + add);
            }
            drawIcon(x + 11, y + 11, 0x0B + add);
        }
    }

    // Draw left/right borders and drawers
    if (player > 1 || playerCount == 2 && player > 0)
    {
        // Right drawer
        // top
        drawIcon(x + 11, y + 1, 0x25 + add);
        drawIcon(x + 12, y + 1, 0x31 + add);
        drawIcon(x + 13, y + 1, 0x31 + add);
        drawIcon(x + 14, y + 1, 0x31 + add);
        drawIcon(x + 15, y + 1, 0x2D + add);
        drawIcon(x, y + 1, 0x22 + add);

        // Edges
        for (i = 0; i < 8; i++)
        {
            drawIcon(x + 11, y + 2 + i, 0x03 + add);
            drawIcon(x + 15, y + 2 + i, 0x02 + add);
            drawIcon(x, y + 2 + i, 0x22 + add);
        }

        // bottom
        drawIcon(x, y + 10, 0x22 + add);
        drawIcon(x + 11, y + 10, 0x25 + add);
        drawIcon(x + 12, y + 10, 0x31 + add);
        drawIcon(x + 13, y + 10, 0x31 + add);
        drawIcon(x + 14, y + 10, 0x31 + add);
        drawIcon(x + 15, y + 10, 0x2F + add);
    }
    else
    {
        // Left drawer
        drawIcon(x - 4, y + 1, 0x2C + add);
        drawIcon(x - 3, y + 1, 0x31 + add);
        drawIcon(x - 2, y + 1, 0x31 + add);
        drawIcon(x - 1, y + 1, 0x31 + add);
        drawIcon(x, y + 1, 0x24 + add);
        drawIcon(x + 11, y + 1, 0x23 + add);

        // Edges
        for (i = 0; i < 8; i++)
        {
            drawIcon(x - 4, y + 2 + i, 0x02 + add);
            drawIcon(x, y + 2 + i, 0x02 + add);
            drawIcon(x + 11, y + 2 + i, 0x23 + add);
        }

        drawIcon(x - 4, y + 10, 0x2E + add);
        drawIcon(x - 3, y + 10, 0x31 + add);
        drawIcon(x - 2, y + 10, 0x31 + add);
        drawIcon(x - 1, y + 10, 0x31 + add);
        drawIcon(x, y + 10, 0x24 + add);
        drawIcon(x + 11, y + 10, 0x23 + add);
    }
}

/**
 * @brief Draw the board
 */
void drawBoard(unsigned char currentPlayerCount)
{
    unsigned char i, j, x, y;
    playerCount = currentPlayerCount;

    fieldX = playerCount > 2 ? 0 : 6;

    for (i = 0; i < playerCount; i++)
    {
        // The field and its drawer only ever hold art: one palette, so the
        // blank sea between hits starts no runs.
        x = quadrant_offset[i][0] + fieldX;
        y = quadrant_offset[i][1];
        for (j = 0; j < 10; j++)
        {
            paint(x, y + j, 10, PAL_ART);
            if (j < 8)
                paint(i > 1 || (i > 0 && fieldX) ? x + 11 : x - 4, y + 1 + j, 3, PAL_ART);
        }

        drawPlayerName(i, "", false);
    }
}

/**
 * @brief draw a horizontal line of w characters at x,y
 */
void drawLine(unsigned char x, unsigned char y, unsigned char w)
{
    while (w--)
        drawIcon(x++, y, 0x3F);
}

/**
 * @brief draw ship, shared between a few routines
 */
static void drawShipInternal(unsigned char x, unsigned char y, unsigned char size, unsigned char delta)
{
    uint8_t c = delta ? 0x37 : 0x32;

    if (delta)
    {
        // Vertical
        drawIcon(x, y++, c--); // top

        while (size > 2) // middle
        {
            drawIcon(x, y++, c);
            size--;
        }

        c--; // bottom
        drawIcon(x, y++, c);
    }
    else
    {
        // Horizontal
        drawIcon(x++, y, c++); // Left

        while (size > 2) // middle
        {
            drawIcon(x++, y, c);
            size--;
        }

        c++; // Right
        drawIcon(x++, y, c);
    }
}

/**
 * @brief Draw ship at position on sea
 */
void drawShip(unsigned char quadrant, unsigned char size, unsigned char pos, bool hide)
{
```

```
    uint8_t delta = 0;
    uint8_t x = 0, y = 0, i = 0;

    if (pos > 99)
    {
        delta = 1;
        pos -= 100;
    }

    x = pos % 10;
    y = pos / 10;

    x += fieldX + quadrant_offset[quadrant][0];
    y += quadrant_offset[quadrant][1];

    if (hide)
    {
        if (!delta)
        {
            for (i = 0; i < size; i++)
            {
                drawIcon(x++, y, 0x38);
            }
        }
        else
        {
            for (i = 0; i < size; i++)
            {
                drawIcon(x, y++, 0x38);
            }
        }
    }
    else
    {
        drawShipInternal(x, y, size, delta);
    }
}

/**
 * @brief Draw a ship in the drawer
 */
void drawLegendShip(uint8_t player, uint8_t index, uint8_t size, uint8_t status)
{
    uint8_t i = 0;
    uint8_t x = quadrant_offset[player][0] + fieldX + legendShipOffset[index][0];
    uint8_t y = quadrant_offset[player][1] + legendShipOffset[index][1];

    if (player > 1 || (player > 0 && fieldX > 0))
    {
        y++;
        x += 11;
    }
    else
    {
        y++;
        x -= 4;
    }

    if (status)
    {
        drawShipInternal(x, y, size, 1); // draw a nice vertical ship
    }
    else
    {
        // Draw the red splats, instead
        for (i = 0; i < size; i++)
            drawIcon(x, y + i, 0x1C);
    }
}

/**
 * @brief Draw game field for given quadrant
 */
void drawGamefield(uint8_t quadrant, uint8_t *field)
{
    uint8_t ix = 0, iy = 0;
    uint8_t x = quadrant_offset[quadrant][0] + fieldX;
    uint8_t y = quadrant_offset[quadrant][1];

    for (iy = 0; iy < 10; iy++)
    {
        for (ix = 0; ix < 10; ix++)
        {
            if (*field)
            {
                drawIcon(x + ix, y + iy, *field == 1 ? 0x39 : 0xE1);
            }
            field++;
        }
    }
}

void drawGamefieldUpdate(uint8_t quadrant, uint8_t *gamefield, uint8_t attackPos, uint8_t anim)
{
    uint8_t x = quadrant_offset[quadrant][0] + fieldX + (attackPos % 10);
    uint8_t y = quadrant_offset[quadrant][1] + (attackPos / 10);
    uint8_t c = gamefield[attackPos];

    // Animate attack
    if (anim > 9)
    {
        drawIcon(x, y, 217 + anim);
        return;
    }

    if (c == FIELD_ATTACK)
    {
        drawIcon(x, y, anim ? 0x1B : 0x39);
    }
    else if (c == FIELD_MISS)
    {
        drawIcon(x, y, 0xE1);
    }
}

/**
 * @brief Draw game field cursor
 */
void drawGamefieldCursor(uint8_t quadrant, uint8_t x, uint8_t y, uint8_t *gamefield, uint8_t blink)
{
    unsigned char ex = quadrant_offset[quadrant][0] + fieldX + x;
    unsigned char ey = quadrant_offset[quadrant][1] + y;
    unsigned char pos = (y * 10) + x;
    unsigned char c;

    switch (gamefield[pos])
    {
    case FIELD_ATTACK:
        c = 0x43;
        break;
    case FIELD_MISS:
        c = 0x46;
        break;
    default:
        c = 0x40;
        break;
    }

    drawIcon(ex, ey, c + blink);
}

/**
 * @brief Draw end game message
 */
void drawEndgameMessage(const char *message)
{
    uint8_t i, x, ix;
    i = (uint8_t)strlen(message);
    x = WIDTH / 2 - i / 2;

    for (ix = 0; ix < WIDTH; ix++)
        drawIcon(ix, HEIGHT - 2, 0xE2);

    drawSpace(0, HEIGHT - 1, WIDTH);
    drawText(x, HEIGHT - 1, message);
}

/**
 * @brief Draw a box
 */
void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h)
{
    drawIcon(x, y, 0x3B);
    drawIcon(x + w + 1, y, 0x3C);
    drawIcon(x, y + h + 1, 0x3D);
    drawIcon(x + w + 1, y + h + 1, 0x3E);
}

#endif /* BUILD_ATARI7800 */
```

```
#ifdef BUILD_ATARI7800

/**
 * @brief Atari 7800 input routines
 * @author Thomas Cherryhomes
 * @license gpl v.3
 */

#include <joystick.h>
#include "vars.h"
#include "pad.h"
#include "../platform-specific/input.h"
#include "../platform-specific/graphics.h"

/*
  The left joystick and the console's switches, read straight from RIOT and
  TIA's input ports; the TIA is only ever read ($00-$1F is never written).
  A ProLine stick is read in two-button mode; a 2600 stick's one button
  arrives on INPT4, and RESET stands in for button 2, as in CONFIG.

  Only the stick travels in readJoystick(). The buttons are synthesized as
  key codes behind kbhit()/cgetc(), as on the NES (src/nes/input.c), with
  SELECT as the shift for the extra letter shortcuts (vars.h lists them):

    1             RETURN    2 / RESET    'r'
  PAUSE         ESCAPE
    SELECT+1      'h'       SELECT+2     's'
  SELECT+PAUSE 'q'
  SELECT alone 'n', reported when it is let go without having shifted
    anything.
*/

#define REG(a)  (*(volatile unsigned char *)(a))
#define INPT0   REG(0x08)   /* button 2, two-button mode: bit 7 set */
#define INPT1   REG(0x09)   /* button 1, two-button mode: bit 7 set */
#define INPT4   REG(0x0C)   /* a 2600 stick's button: bit 7 clear */
#define SWCHA   REG(0x280)  /* bits 7-4: right, left, down, up; low */
#define SWCHB   REG(0x282)  /* bit 0 RESET, 1 SELECT, 3 PAUSE; low */
#define CTLSWB  REG(0x283)

#define PB2     0x04        /* left port's two-button mode, driven low */

#define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)

static unsigned char lastButtons;
static unsigned char shifted;
static unsigned char pendingKey;

void initInput(void)
{
    CTLSWB = PB2;
    SWCHB = 0;
}

unsigned char readPad(void)
{
    unsigned char b = (unsigned char)~SWCHA & DIRS;
    unsigned char sw = (unsigned char)~SWCHB;

    if ((INPT1 & 0x80) || !(INPT4 & 0x80))
        b |= PAD_FIRE;
    if ((INPT0 & 0x80) || (sw & 0x01))
        b |= PAD_BACK;
    if (sw & 0x02)
        b |= PAD_SELECT;
    if (sw & 0x08)
        b |= PAD_PAUSE;
    return b;
}

unsigned char readJoystick()
{
    return readPad() & DIRS;
}

static unsigned char poll(void)
{
    unsigned char buttons = readPad() & ~DIRS;
    unsigned char pressed = buttons & ~lastButtons;
    unsigned char released = lastButtons & ~buttons;
    unsigned char sel = buttons & PAD_SELECT;

    lastButtons = buttons;

    if (pressed & PAD_SELECT)
        shifted = 0;

    if (pressed & PAD_FIRE)
    {
        shifted = 1;
        return sel ? 'h' : KEY_RETURN;
    }
    if (pressed & PAD_BACK)
    {
        shifted = 1;
        return sel ? 's' : 'r';
    }
    if (pressed & PAD_PAUSE)
    {
        shifted = 1;
        return sel ? 'q' : KEY_ESCAPE;
    }
    if ((released & PAD_SELECT) && !shifted)
        return 'n';

    return 0;
}

unsigned char kbhit(void)
{
    if (!pendingKey)
        pendingKey = poll();

    return pendingKey != 0;
}

unsigned char cgetc(void)
{
    unsigned char k;

    while (!kbhit())
        waitvsync();

    k = pendingKey;
    pendingKey = 0;
    return k;
}

#endif /* BUILD_ATARI7800 */
```

pad.h

```
#ifdef BUILD_ATARI7800

/**
 * @brief   Atari 7800 joystick and console switches - shared by input.c,
 *          osk.c and graphics.c
 * @license gpl v. 3, see LICENSE for details
 */

#ifndef PAD_H
#define PAD_H

// The stick in cc65's JOY_*_MASK bits (0xF0), the buttons below them.
#define PAD_FIRE   0x01 /* button 1 */
#define PAD_BACK   0x02 /* button 2, or RESET */
#define PAD_SELECT 0x04
#define PAD_PAUSE  0x08

void initInput(void);
unsigned char readPad(void);

#endif /* PAD_H */

#endif /* BUILD_ATARI7800 */

```

osk.c

```
#ifdef BUILD_ATARI7800

/**
 * @brief   Atari 7800 on-screen keyboard
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <joystick.h>
#include <string.h>
#include "vars.h"
#include "pad.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/sound.h"

/*
  The 7800 has no keyboard, so the player name is typed on screen: the NES
  port's grid (src/nes/osk.c), on the joystick. Button 1 types the cell under
  the cursor, button 2 (or RESET) backspaces and PAUSE accepts. There is no
  cancel: the caller only reaches this screen when a name is required.

  Entry is uppercase-only (the server lowercases names anyway - see
  welcomeActionVerifyPlayerName()).
*/

#define OSK_COLS 10
#define OSK_ROWS 4
#define OSK_PITCH 3 /* marker column + two label columns */
#define OSK_X 1
#define OSK_Y 19    /* below the name box showPlayerNameScreen draws at 16-18 */

/* The last row is one short: its tenth cell does not exist. */
#define OSK_LAST_ROW_COLS 9

#define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
#define CELL_DEL (CELL_SPACE + 1)
#define CELL_OK (CELL_SPACE + 2)

static const char cells[] =
    "ABCDEFGHIJ"
    "KLMNOPQRST"
    "UVWXYZ0123"
    "456789";

#define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)

#define REPEAT_FIRST 18 /* vblanks before a held stick repeats */
#define REPEAT_NEXT 5

static unsigned char curX, curY;

static unsigned char cellCols(unsigned char row)
{
    return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS : OSK_COLS;
}

static void drawCell(unsigned char col, unsigned char row, unsigned char marked)
{
    unsigned char idx = row * OSK_COLS + col;
    unsigned char x = OSK_X + col * OSK_PITCH;
    char glyph[3];

    drawText(x, OSK_Y + row, marked ? ">" : " ");

    if (idx == CELL_SPACE)
        drawText(x + 1, OSK_Y + row, "SP");
    else if (idx == CELL_DEL)
        drawText(x + 1, OSK_Y + row, "<-");
    else if (idx == CELL_OK)
        drawText(x + 1, OSK_Y + row, "OK");
    else
    {
        glyph[0] = cells[idx];
        glyph[1] = ' ';
        glyph[2] = 0;
        drawText(x + 1, OSK_Y + row, glyph);
    }
}

static void drawGrid(void)
{
    unsigned char row, col;

    for (row = 0; row < OSK_ROWS; row++)
        for (col = 0; col < cellCols(row); col++)
            drawCell(col, row, row == curY && col == curX);
}

static void clearGrid(void)
{
    unsigned char row;

    for (row = 0; row < OSK_ROWS; row++)
        drawSpace(0, OSK_Y + row, WIDTH);
}
```

```

/* Repaint the field and its trailing cursor chip, the same shape
   inputFieldCycle() draws on the platforms that have a keyboard. */
static void drawField(unsigned char x, unsigned char y, unsigned char max,
                      const char *buffer)
{
    unsigned char len = (unsigned char)strlen(buffer);

    drawSpace(x, y, max);
    drawText(x, y, buffer);
    if (len < max)
        drawIcon(x + len, y, ICON_TEXT_CURSOR);
}

static void appendChar(char *buffer, unsigned char *len, unsigned char max,
                       char ch)
{
    if (*len >= max)
    {
        soundInvalid();
        return;
    }
    buffer[(*len)++] = ch;
    buffer[*len] = 0;
    soundCursor();
}

void platformNameEntry(unsigned char x, unsigned char y, unsigned char max,
                       char *buffer)
{
    unsigned char len = (unsigned char)strlen(buffer);
    unsigned char hold = 0, lastDir = 0, lastButtons;
    unsigned char pad, dir, move, pressed, idx;

    curX = 0;
    curY = 0;
    drawGrid();
    drawField(x, y, max, buffer);

    // Whatever is already held (the button that opened this screen) is not a press.
    lastButtons = readPad() & ~DIRS;

    for (;;)
    {
        waitvsync();

        pad = readPad();
        dir = pad & DIRS;
        pressed = pad & ~DIRS & ~lastButtons;
        lastButtons = pad & ~DIRS;

        /* Auto-repeat is counted in vblanks. This loop runs once per frame,
           so `hold` is already a frame count. */
        move = 0;
        if (dir != lastDir)
        {
            lastDir = dir;
            hold = REPEAT_FIRST;
            move = dir;
        }
        else if (dir && --hold == 0)
        {
            hold = REPEAT_NEXT;
            move = dir;
        }

        if (move)
        {
            drawCell(curX, curY, 0);

            if ((move & JOY_LEFT_MASK) && curX)
                curX--;
            else if ((move & JOY_RIGHT_MASK) && curX + 1 < cellCols(curY))
                curX++;
            else if ((move & JOY_UP_MASK) && curY)
                curY--;
            else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
                curY++;

            if (curX >= cellCols(curY))
                curX = cellCols(curY) - 1;

            drawCell(curX, curY, 1);
            soundCursor();
        }

        if (!pressed)
            continue;

        /* Button 1 takes whatever the cursor is sitting on; button 2 and
           PAUSE are shortcuts for the backspace and OK cells. */
        if (pressed & PAD_BACK)
            idx = CELL_DEL;
        else if (pressed & PAD_PAUSE)
            idx = CELL_OK;
        else if (pressed & PAD_FIRE)
            idx = curY * OSK_COLS + curX;
        else
            continue;

        if (idx == CELL_OK)
        {
            if (!len)
            {
                soundInvalid();
                continue;
            }
            break;
        }

        if (idx == CELL_DEL)
        {
            if (!len)
            {
                soundInvalid();
                continue;
            }
            buffer[--len] = 0;
            soundCursor();
        }
        else
            appendChar(buffer, &len, max, idx == CELL_SPACE ? ' ' : cells[idx]);

        drawField(x, y, max, buffer);
    }

    clearGrid();
}

#endif /* BUILD_ATARI7800 */

```

```c
#ifdef BUILD_ATARI7800

/**
 * @brief   Atari 7800 Sound Routines (the cartridge's POKEY at $0450)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The NES port's cues (src/nes/sound.c), which are the msdos PC-speaker
 * table. Never the TIA: its registers are INPTCTRL until locked. Tones are
 * channels 1+2 joined into 16 bits off the 1.79MHz clock, which reaches the
 * whole table exactly. The explosions sweep 120-30Hz on channels 3+4 joined
 * the same way, where the NES used its triangle, with the noise burst on
 * channel 1.
 */

#include <stdint.h>
#include <stdlib.h>
#include "../misc.h"

#define POKEY(r)  (*(volatile uint8_t *)(0x0450 + (r)))
#define AUDF1     POKEY(0x0)
#define AUDC1     POKEY(0x1)
#define AUDF2     POKEY(0x2)
#define AUDC2     POKEY(0x3)
#define AUDF3     POKEY(0x4)
#define AUDC3     POKEY(0x5)
#define AUDF4     POKEY(0x6)
#define AUDC4     POKEY(0x7)
#define AUDCTL    POKEY(0x8)
#define SKCTL     POKEY(0xF)

#define CTL_TONE  0x50  /* 1+2 joined, channel 1 at 1.79MHz */
#define CTL_BOOM  0x28  /* 3+4 joined, channel 3 at 1.79MHz; 1 and 2 at 64kHz */
#define PURE      0xA0  /* AUDC: pure tone; the low nibble is the volume */
#define NOISE     0x80  /* AUDC: the 17-bit poly, white noise */

#define TONE_VOL  10
#define BOOM_VOL  8

#define MIN_GATE  3     /* shorter blips are barely audible */

/* 16-bit divider for hz off the 1.79MHz clock */
static uint16_t divider(unsigned int hz)
{
    return (uint16_t)(894886UL / hz - 7);
}

/* The NES noise periods 9-15 (hiss to rumble) as channel 1 dividers at 64kHz */
static const uint8_t noiseF[7] = {8, 13, 17, 26, 35, 72, 144};

/**
 * @brief Beep
 * @param hz Frequency in Hz
 * @param gate Tone-on time in vertical blanks
 * @param postGate Delay after tone off in vertical blanks
 */
static void beep(unsigned int hz, uint8_t gate, uint8_t postGate)
{
    uint16_t n;

    if (prefs.disableSound)
        return;

    n = divider(hz);

    // Stretch short blips, taking the extra out of the rest so the cue
    // keeps its rhythm.
    if (gate < MIN_GATE)
    {
        postGate = postGate > MIN_GATE - gate ? postGate - (MIN_GATE - gate) : 0;
        gate = MIN_GATE;
    }

    AUDCTL = CTL_TONE;
    AUDF1 = n & 0xFF;
    AUDF2 = n >> 8;
    AUDC1 = 0;
    AUDC2 = PURE | TONE_VOL;

    while (gate--)
        waitvsync();

    AUDC2 = PURE;

    while (postGate--)
        waitvsync();
}

/**
 * @brief One step of an explosion: a tone at hz under a noise burst
 * @param hz Tone frequency in Hz, 0 for noise alone
 * @param noise NES noise period index, 9 (hiss) - 15 (rumble)
 * @param vol Noise volume, 0-15
 * @param gate Time in vertical blanks
 */
static void boom(unsigned int hz, uint8_t noise, uint8_t vol, uint8_t gate)
{
    uint16_t n;

    if (prefs.disableSound)
        return;

    AUDCTL = CTL_BOOM;
    if (hz)
    {
        n = divider(hz);
        AUDF3 = n & 0xFF;
        AUDF4 = n >> 8;
        AUDC3 = 0;
        AUDC4 = PURE | BOOM_VOL;
    }
    else
        AUDC4 = PURE;

    AUDF1 = noiseF[noise - 9];
    AUDC1 = NOISE | (vol * 3 >> 2);

    while (gate--)
        waitvsync();

    AUDC4 = PURE;
    AUDC1 = NOISE;
}

/**
 * @brief Let the explosion's noise ring out, fading from vol to silence
 */
static void tail(uint8_t noise, uint8_t vol, uint8_t step)
{
    while (vol)
    {
        boom(0, noise, vol, step);
        vol--;
    }
}

void initSound()
{
    SKCTL = 0;
    SKCTL = 3;      /* out of initialisation: the counters run */
    AUDCTL = 0;
    AUDC1 = 0;
    AUDC2 = 0;
    AUDC3 = 0;
    AUDC4 = 0;
}

void soundJoinGame()
{
    beep(430, 5, 8);
    beep(340, 5, 0);
    beep(500, 5, 0);
```

```
}

void soundMyTurn()
{
    beep(430, 4, 2);
    beep(430, 4, 2);
}

void soundGameDone()
{
    beep(311, 10, 0);
    beep(330, 20, 0);
    beep(392, 10, 0);
    beep(415, 20, 0);
}

void soundCursor()
{
    beep(300, 1, 0);
}

void soundPlaceShip()
{
    beep(300, 3, 1);
    beep(350, 3, 0);
}

void soundTick()
{
    beep(100, 1, 0);
}

void soundSelect()
{
    beep(350, 2, 1);
    beep(250, 2, 0);
    beep(150, 2, 0);
}

void soundMiss()
{
    beep(150, 1, 0);
    beep(170, 1, 0);
}

void soundInvalid()
{
    beep(150, 2, 2);
    beep(150, 2, 0);
}

void soundAttack()
{
    // msdos: three jittered ~90Hz blips, two ~70Hz, one ~60Hz - the shot
    // leaving. The noise starts as a hiss and drops.
    uint8_t i;

    for (i = 0; i < 3; i++)
        boom(90 + (rand() & 1), 0x09, 10, 2);
    for (i = 0; i < 2; i++)
        boom(70 + (rand() & 1), 0x0B, 8, 2);
    boom(60 + (rand() & 1), 0x0C, 6, 2);
}

void soundHit()
{
    // msdos: 70 down to 30Hz in steps of 10
    uint8_t i, n = 0x0B;

    for (i = 70; i >= 30; i -= 10)
        boom(i, n++, 15, 2);
    tail(0x0F, 10, 2);
}

void soundSink()
{
    // msdos: 120 down to 60Hz in steps of 10, then a long rumble
    uint8_t i, n = 0x09;

    for (i = 120; i >= 60; i -= 10)
        boom(i, n++, 15, 2);
    tail(0x0F, 15, 3);
}

// Not applicable to the 7800
void soundStop() {}
void disableKeySounds() {}
void enableKeySounds() {}

#endif /* BUILD_ATARI7800 */

```

util.c

```
#ifdef BUILD_ATARI7800

/**
 * @brief   Utility Functions - Atari 7800
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include "vars.h"
#include "maria.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/util.h"

// No interrupt counts frames on the 7800 (the engine uses no DLIs), so the
// game timer counts the vertical blanks the game waits for. Every loop that
// times something waits a frame per pass.
static uint16_t frames, timerBase;
static bool seeded;

/* mt_sync() also rebuilds the rows whose palettes changed: tiles show at
   once, palette changes only from here. */
void waitvsync(void)
{
    mt_sync();
    frames++;
}

void resetTimer(void)
{
    timerBase = frames;
}

uint16_t getTime(void)
{
    return frames - timerBase;
}

uint8_t getJiffiesPerSecond()
{
    return mt_pal ? 50 : 60;
}

uint8_t getRandomNumber(uint8_t maxExclusive)
{
    if (!seeded)
    {
        // Seeded from the frame count at the first request, which arrives
        // only after some human-paced input
        seeded = true;
        srand(frames);
    }
    return (uint8_t)(rand() % maxExclusive);
}

void housekeeping()
{
}

// quit() is in fujinet.c, with the rest of the cartridge calls.

#endif /* BUILD_ATARI7800 */

```

tiles.h

```
/* GENERATED by src/atari7800/mkchr.py from support/msdos/charset.dat
 * and ascii.dat. Tile numbers in mt_tiles; 126 of 128 used. The font
 * is at its ASCII codes. Include from graphics.c only (it defines
 * tables). Do not edit. */
#ifndef TILES_H
#define TILES_H

#define T_BLANK          0x20

#define PAL_TEXT         0
#define PAL_ART          1
#define AGNOSTIC         2  /* white or blank: keep the cell's palette */

/* MS-DOS sheet index -> tile. Unlisted indices are blank. */
static const unsigned char sheetTile[256] = {
```

```
    0x20, 0x20, 0x00, 0x01, 0x20, 0x6D, 0x20, 0x20, 0x02, 0x03, 0x04, 0x05, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x17, 0x18, 0x6E, 0x6F, 0x70,
    0x06, 0x07, 0x00, 0x01, 0x08, 0x09, 0x20, 0x0A, 0x0B, 0x0C, 0x19, 0x1A, 0x0D, 0x0E, 0x0F, 0x10,
    0x20, 0x11, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F, 0x60, 0x20, 0x61, 0x62, 0x71, 0x72, 0x73, 0x74, 0x63,
    0x20, 0x75, 0x76, 0x61, 0x77, 0x78, 0x65, 0x79, 0x7A, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x64, 0x12, 0x13, 0x14, 0x15,
    0x16, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x00, 0x01, 0x20, 0x20, 0x20, 0x20, 0x02, 0x03, 0x04, 0x05, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x06, 0x07, 0x00, 0x01, 0x08, 0x09, 0x20, 0x0A, 0x0B, 0x0C, 0x20, 0x20, 0x0D, 0x0E, 0x0F, 0x10,
    0x20, 0x11, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x12, 0x13, 0x14, 0x15,
    0x16, 0x65, 0x66, 0x67, 0x68, 0x69, 0x6A, 0x6B, 0x6C, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
    0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
};

/* MS-DOS sheet index -> PAL_TEXT, PAL_ART or AGNOSTIC. */
static const unsigned char sheetPal[256] = {
    0x02, 0x02, 0x00, 0x00, 0x02, 0x02, 0x02, 0x02, 0x00, 0x00, 0x00, 0x00, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x01, 0x01, 0x01, 0x01,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00, 0x01, 0x01, 0x00, 0x00, 0x00,
    0x02, 0x00, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x01, 0x02, 0x01, 0x02, 0x01, 0x01, 0x01, 0x01,
    0x02, 0x01, 0x02, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x01, 0x00, 0x00, 0x00,
    0x00, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x01, 0x01, 0x02, 0x02, 0x02, 0x02, 0x02, 0x01, 0x01, 0x01, 0x01, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x02, 0x01, 0x01, 0x01, 0x02, 0x02, 0x01, 0x01, 0x01,
    0x02, 0x01, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x01, 0x01, 0x01, 0x01,
    0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02,
    0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02, 0x02
```

```
#!/usr/bin/env python3
"""mkchr.py -- build the MARIA tile set for Fuji Battleship on the Atari 7800.

The art is the MS-DOS client's, as on the NES: support/msdos/charset.dat (the
game sheet) and support/msdos/ascii.dat (the font), 8x8 CGA 2bpp. MARIA's
320B cells are 2bpp too, but the engine (maria.h) gives a screen 128 tiles
and two palettes of three colours, so where the NES baked a colour pair into
extra glyph sets this uses the two palettes instead:

  palette 0  1 white  2 red   3 white     text, active board chrome
  palette 1  1 red    2 cyan  3 white     red captials, inactive chrome, art

The font draws in value 1, so the same glyph is white text in palette 0 and
red alternate text in palette 1. Chrome draws red as 2 and white as 3, so the
active (palette 0) and inactive (palette 1, red -> cyan) boards share tiles.
Other art is recoded to palette 1. A tile that is only white or blank looks
the same in both and is marked AGNOSTIC: drawing it leaves the cell's palette
alone. Name plates have no glyph set of their own: names are font text over
the plate tile.

The font sits at its ASCII codes ($20-$5F, so tile $20 is the blank) and the
art fills $00-$1F and $60-$7F. Identical bitmaps share a tile; the script
fails rather than truncate.

Writes build/atari7800/art.chr (NES CHR layout), runs mktiles.py on it into
src/atari7800/tiles.s, and writes src/atari7800/tiles.h and
support/atari7800/tilemap.lua (tile -> character, for the MAME smoke test).
The Makefile runs it before every 7800 build.
"""

import os
import subprocess
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.join(HERE, '..', '..')
SHEET = os.path.join(ROOT, 'support', 'msdos', 'charset.dat')
ASCII = os.path.join(ROOT, 'support', 'msdos', 'ascii.dat')
CHR = os.path.join(ROOT, 'build', 'atari7800', 'art.chr')
TILES_S = os.path.join(HERE, 'tiles.s')
TILES_H = os.path.join(HERE, 'tiles.h')
TILEMAP = os.path.join(ROOT, 'support', 'atari7800', 'tilemap.lua')

BLUE, CYAN, RED, WHITE = 0, 1, 2, 3
PAL_TEXT, PAL_ART, AGNOSTIC = 0, 1, 2

# Board chrome: drawn as-is for the active player (palette 0) and +0x80 for
# everyone else (palette 1). 0x60 is the solid name plate.
CHROME = [0x02, 0x03, 0x08, 0x09, 0x0A, 0x0B, 0x20, 0x21, 0x22, 0x23, 0x24,
          0x25, 0x27, 0x28, 0x29, 0x2C, 0x2D, 0x2E, 0x2F, 0x31, 0x5C, 0x5D,
          0x5E, 0x5F, 0x60]
# Everything else graphics.c or the shared code (vars.h ICON_*) draws.
OTHER = ([0x00, 0x05, 0x19, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F, 0x2A, 0x2B]
         + list(range(0x32, 0x49))      # ships, hit, icons, box, line, cursor
         + [0x5B, 0xE1, 0xE2]
         + list(range(0xE3, 0xE9)))     # attack animation, 217 + 10..15

# The shared border row where two boards meet (graphics.c plotTilePair).
PAIRS = [(0x08, 0x0A), (0x0A, 0x08), (0x27, 0x29), (0x29, 0x27),
         (0x09, 0x0B), (0x0B, 0x09)]

# What the smoke test prints for the art; the font never shows lower case.
READS = {**{c: 's' for c in range(0x32, 0x38)},
         0x39: 'x', 0x1B: 'x', 0x1C: 'x', 0xE1: 'o', 0x2A: 'p', 0x2B: 'm',
         0x3F: '~', 0xE2: '~', 0x5B: '>', 0x3A: '_',
         **{c: 'c' for c in range(0x41, 0x49)},
         **{c: 'a' for c in range(0xE3, 0xE9)}}

ART_SLOTS = list(range(0x00, 0x20)) + list(range(0x60, 0x80))


def load(path):
    data = open(path, 'rb').read()
    tiles = []
    for t in range(256):
        b = data[t * 16:t * 16 + 16]
        px = []
        for r in range(8):
            w = (b[r * 2] << 8) | b[r * 2 + 1]
            px.append([(w >> (14 - 2 * i)) & 3 for i in range(8)])
        tiles.append(px)
    return tiles


def recode(px, values):
    return tuple(tuple(values[p] for p in row) for row in px)


def merge(a, b):
    return [[pa if pa else pb for pa, pb in zip(ra, rb)] for ra, rb in zip(a, b)]


def chr_bytes(px):
    p0 = bytes(sum(((px[r][c] & 1) << (7 - c)) for c in range(8)) for r in range(8))
    p1 = bytes(sum((((px[r][c] >> 1) & 1) << (7 - c)) for c in range(8)) for r in range(8))
    return p0 + p1


FONT_CODE = {BLUE: 0, CYAN: 0, RED: 0, WHITE: 1}
CHROME_CODE = {BLUE: 0, CYAN: 2, RED: 2, WHITE: 3}
ART_CODE = {BLUE: 0, RED: 1, CYAN: 2, WHITE: 3}


def main():
    sheet = load(SHEET)
    font = load(ASCII)

    tiles = {}          # tile -> bitmap
    chars = {}          # tile -> what tilemap.lua reads it as
    index = {}          # bitmap -> tile
    free = list(ART_SLOTS)

    for c in range(0x20, 0x60):
        px = recode(font[c], FONT_CODE)
        tiles[c] = px
        chars[c] = chr(c)
        index.setdefault(px, c)

    def art(px, ch):
        if px in index:
            return index[px]
        if not free:
            raise SystemExit('mkchr: out of tiles: the engine holds 128')
        t = free.pop(0)
        tiles[t] = px
        chars[t] = ch
        index[px] = t
        return t
```

```
    def agnostic(px):
        return all(v in (0, 3) for row in px
for v in row)

    sheetTile = [0x20] * 256
    sheetPal = [AGNOSTIC] * 256
    for n in CHROME:
        t = art(recode(sheet[n], CHROME_CODE),
'#')
        sheetTile[n] = sheetTile[n + 0x80] = t
        sheetPal[n], sheetPal[n + 0x80] =
PAL_TEXT, PAL_ART
    # The tiles the smoke test reads first, so
a cursor that shares a bitmap
    # with a miss reads as the miss.
    for n in sorted(OTHER, key=lambda n: n not
in READS or READS[n] == 'c'):
        px = recode(sheet[n], ART_CODE)
        sheetTile[n] = art(px, READS.get(n,
'#'))
        sheetPal[n] = AGNOSTIC if agnostic(px)
else PAL_ART

    pairTile = [art(recode(merge(sheet[a],
sheet[b]), CHROME_CODE), '#')
                for a, b in PAIRS]

    used = len(tiles)
    blank = tuple(tuple([0] * 8) for _ in
range(8))

    os.makedirs(os.path.dirname(CHR),
exist_ok=True)
    with open(CHR, 'wb') as f:
        for t in range(128):
            f.write(chr_bytes(tiles.get(t,
blank)))
    subprocess.run([sys.executable,
os.path.join(HERE, 'mktiles.py'), '--chr',
CHR,
                    TILES_S], check=True)

    def table(name, values, per=16):
        out = ['static const unsigned char
%s[%d] = {' % (name, len(values))]
        for i in range(0, len(values), per):
            out.append('    ' +
'.join('0x%02X' % v for v in values[i:i +
per]) + ',')
        out.append('};')
        return '\n'.join(out)

    with open(TILES_H, 'w') as f:
        f.write('/* GENERATED by src/
atari7800/mkchr.py from support/msdos/
charset.dat\n'
                ' * and ascii.dat. Tile
numbers in mt_tiles; %d of 128 used. The
font\n'
                ' * is at its ASCII codes.
Include from graphics.c only (it defines\n'
                ' * tables). Do not edit. */
\n'
                '#ifndef TILES_H\n#define
TILES_H\n\n' % used)
        f.write('#define T_BLANK
0x20\n\n')
        f.write('#define PAL_TEXT
%d\n' % PAL_TEXT)
        f.write('#define PAL_ART
%d\n' % PAL_ART)
        f.write('#define AGNOSTIC
%d  /* white or blank: keep the cell\'s
palette */\n\n'
                % AGNOSTIC)
        f.write('/* MS-DOS sheet index ->
tile. Unlisted indices are blank. */\n')
        f.write(table('sheetTile', sheetTile)
+ '\n\n')
        f.write('/* MS-DOS sheet index ->
PAL_TEXT, PAL_ART or AGNOSTIC. */\n')
        f.write(table('sheetPal', sheetPal) +
'\n\n')
        f.write('/* Merged border pairs, in
plotTilePair() order: (08,0A) (0A,08)\n'
                ' * (27,29) (29,27) (09,0B)
(0B,09); palette 1 for the +0x80 set. */\n')
        f.write(table('pairTile', pairTile,
per=6) + '\n\n')
        f.write('#endif /* TILES_H */\n')

    os.makedirs(os.path.dirname(TILEMAP),
exist_ok=True)
    with open(TILEMAP, 'w') as f:
        f.write('-- GENERATED by src/
atari7800/mkchr.py: what each tile reads as.
\n'
                '-- Text is itself; art is a
lower-case letter or a symbol (s ship, x hit,
o miss, c cursor,\n'
                '-- p player, m mark, >
active, a attack, - ~ _ lines, # chrome).\n'
                'return {\n')
        for t in range(128):
            ch = chars.get(t, ' ')
            f.write('  [%d] = %s,\n' % (t, '"\
\""' if ch == '"' else '"%s"' % ch.replace('\
\', '\\\\')))
        f.write('}\n')
    print('mkchr: %d of 128 tiles' % used)


if __name__ == '__main__':
    main()
```

```
/* maria.h -- a text and tile screen for the Atari 7800 on MARIA, shared by
 * CONFIG and the FujiNet games (each copies maria.s, maria.h and
 * mktiles.py into its own tree).
 *
 * 32 x 24 cells of 8 x 8 pixels (256 x 192), centred, in 320B: every pixel
 * is 2 bits, so a tile has three colours and the background, the way NES
 * CHR tiles do. A cell holds a tile number (0-127) and a palette select (0
 * or 1, MARIA palettes 0 and 4: 320B honours only their high bit). Pixel
 * value 0 is BACKGRND in both; values 1-3 are the selected palette's C1-C3.
 *
 * How it is drawn. Each cell row is one 8-line zone whose display list
 * reads the row's 32 map bytes as indirect two-byte characters from the
 * program's tile set. The list has one 5-byte header per run of cells
 * sharing a palette, so writing tiles costs nothing but the store: MARIA
 * reads the map every line. Changing a palette marks the row, and mt_sync()
 * rebuilds that row's list into its spare slot and swaps one DLL byte, so
 * MARIA never reads a half-built list. Map, palettes, lists and the DLL are
 * in the cart's RAM (the VIDEO segment); the tiles are in ROM (TILES).
 * The atari7800-fujinet.cfg linker config places both, page aligned.
 *
 * DMA cost per line: 16 + 10 per run + 9 per cell (3 for the map byte, 6
 * for the two graphics bytes), of the 426 MARIA clocks a line allows: a
 * one-row row takes 314, which leaves the 6502 about a third of each
 * displayed line, about half of a whole frame. Rows are
 * capped at MT_MAXRUNS runs (434 would overrun); cells past the cap take the
 * last run's palette. No display-list interrupts are used: the lib's NMI
 * hook (fuji_a7800_nmi) is free for a game's own.
 *
 * The tile set: mt_tiles, 2K in segment TILES (page aligned), 8 pages of
 * 256 bytes, page 7 the top pixel row, tile t at bytes 2t and 2t+1 each.
 * mktiles.py makes it from font.txt-style art or NES 2bpp CHR.
 *
 * Rules: never store to $00-$1F (it is INPTCTRL until locked). The engine
 * uses MARIA's own WSYNC ($24) and owns CTRL, CHARBASE, DPPH/DPPL, OFFSET,
 * BACKGRND and the palette registers through mt_palette().
 */

#ifndef MARIA_H
#define MARIA_H

#define MT_COLS     32
#define MT_ROWS     24
#define MT_MAXRUNS  12

/* What the cells hold. The map's bytes are tile * 2 (MARIA's character
 * code); use the calls below rather than storing into them. */
extern unsigned char mt_map[MT_ROWS * MT_COLS];
extern unsigned char mt_attr[MT_ROWS * MT_COLS];

/* 1 on a PAL console, measured by mt_init(). */
extern unsigned char mt_pal;

/* Vertical blanks seen by mt_sync(), wrapping at 256. */
extern unsigned char mt_frames;

/* The program's tile set (mktiles.py output). */
extern const unsigned char mt_tiles[];

/* Measure the TV standard, clear the screen to tile `blank` with palette 0,
 * build the lists and turn the display on at the next vertical blank. */
void __fastcall__ mt_init(unsigned char blank);

/* The display off (MARIA's DMA stops, the 6502 runs flat out) and on. */
void mt_off(void);
void mt_on(void);

/* Wait for the start of the next vertical blank, count it, and rebuild the
 * rows whose palettes changed. Call it once a frame. */
void mt_sync(void);

/* Colours: BACKGRND, and palette `p` (0 or 1) C1-C3. Values are MARIA's
 * hue << 4 | luminance. */
void __fastcall__ mt_background(unsigned char c);
void __fastcall__ mt_palette(unsigned char p, unsigned char c1,
                             unsigned char c2, unsigned char c3);

/* Every cell to tile `blank`, palette 0. */
void __fastcall__ mt_clear(unsigned char blank);

/* The write cursor. Writes advance it along the row and stop at its end. */
void __fastcall__ mt_at(unsigned char col, unsigned char row);
void __fastcall__ mt_put(unsigned char tile);
/* A NUL-terminated string at the cursor: character = tile number. */
void __fastcall__ mt_puts(const char *s);
/* Text: as mt_puts, but at most `max` characters, and any outside 32-126
 * shows as a space. Returns how many cells it wrote. */
unsigned char __fastcall__ mt_text(const char *s, unsigned char max);
/* `n` cells of `tile` at the cursor. */
void __fastcall__ mt_fill(unsigned char tile, unsigned char n);
/* `n` cells from the cursor to palette `p`, without moving the cursor. */
void __fastcall__ mt_setpal(unsigned char p, unsigned char n);

/* The tile at (col,row). */
unsigned char __fastcall__ mt_get(unsigned char col, unsigned char row);

#endif /* MARIA_H */
```

```
; maria.s -- the 32 x 24 text and tile screen; see maria.h.
;
; Nothing here stores to $00-$1F: INPTCTRL is left unlocked.

        .include "zeropage.inc"

        .export _mt_init, _mt_off, _mt_on, _mt_sync, _mt_background
        .export _mt_palette, _mt_clear, _mt_at, _mt_put, _mt_puts
        .export _mt_fill, _mt_setpal, _mt_get, _mt_text
        .export _mt_map, _mt_attr, _mt_pal, _mt_frames
        .import _mt_tiles, popa, popax

BACKGRND = $20
P0C1    = $21
MWSYNC  = $24
MSTAT   = $28                   ; bit 7: vertical blank
DPPH    = $2C
DPPL    = $30
P4C1    = $31
CHARBASE = $34
OFFSET  = $38
CTRL    = $3C

COLS    = 32
ROWS    = 24
MAXRUNS = 12
SLOT    = 64                    ; bytes per display list slot
HDR_WM  = $E0                   ; extended header, indirect, write mode 1
DMA_ON  = $56                   ; DMA on, 2-byte characters, kangaroo, 320B
DMA_OFF = $7F
HPOS0   = 16                    ; (160 - 4 * COLS) / 2 in HPOS units: centred
PAL_LINES = 270                 ; NTSC MARIA draws 242 lines, PAL 292
TOP     = 4                     ; blank DLL entries above the rows
BOTTOM  = 4

        .segment "VIDEO"        ; page aligned, in the cart's RAM
_mt_map:  .res  ROWS * COLS     ; tile * 2 per cell
_mt_attr: .res  ROWS * COLS     ; palette select per cell, 3 pages on
dls:    .res    ROWS * 2 * SLOT ; two lists per row, in one page
dll:    .res    (TOP + ROWS + BOTTOM) * 3
nulldl: .res    2
dirty:  .res    ROWS            ; the row's palettes changed
slot:   .res    ROWS            ; the slot MARIA reads

        .zeropage
mrow:   .res    2               ; the cursor's row in the map
arow:   .res    2               ; ...and in the palette selects
ccol:   .res    1
crow:   .res    1

        .segment "BSS"
_mt_pal:    .res 1
_mt_frames: .res 1
brow:   .res    1
rpal:   .res    1
rstart: .res    1
rend:   .res    1
runs:   .res    1
oidx:   .res    1
syncrow: .res   1

        .code

; void __fastcall__ mt_init (unsigned char blank)
.proc _mt_init
        pha
        jsr     _mt_off
        jsr     measure
        lda     #0
        sta     nulldl
        sta     nulldl+1
        sta     _mt_frames

        ldx     #0              ; the blank lines above, NTSC or PAL
        ldy     _mt_pal
        beq     @top
        ldy     #TOP
@top:   lda     toplines,y
        sta     dll,x
        lda     #>nulldl
        sta     dll+1,x
        lda     #<nulldl
        sta     dll+2,x
        inx
        inx
        inx
        iny
        cpx     #TOP * 3
        bne     @top
        ldy     #0              ; the rows, each 8 lines
@row:   lda     #7
        sta     dll,x
        lda     dlhi,y
        sta     dll+1,x
        lda     dllo,y
        sta     dll+2,x
        lda     #0
        sta     slot,y
        inx
        inx
        inx
        iny
        cpy     #ROWS
        bne     @row
@bot:   lda     #15             ; and enough below to reach the blank
        sta     dll,x
        lda     #>nulldl
        sta     dll+1,x
        lda     #<nulldl
        sta     dll+2,x
        inx
        inx
        inx
        cpx     #(TOP + ROWS + BOTTOM) * 3
        bne     @bot

        pla
        jsr     _mt_clear
        jsr     rebuild

        lda     #>_mt_tiles
        sta     CHARBASE
        lda     #0
        sta     OFFSET
        lda     #>dll
        sta     DPPH
        lda     #<dll
        sta     DPPL
        jmp     _mt_on
.endproc

; Count MARIA's lines from the end of one vertical blank to the next.
.proc measure
@in:    bit     MSTAT
        bpl     @in
@out:   bit     MSTAT
        bmi     @out
        ldx     #0
        ldy     #0
@line:  sta     MWSYNC
        inx
        bne     @same
        iny
@same:  bit     MSTAT
        bpl     @line
        lda     #0
        cpy     #>PAL_LINES
        bcc     @set
        bne     @pal
        cpx     #<PAL_LINES
        bcc     @set
@pal:   lda     #1
@set:   sta     _mt_pal
```

```
        rts
.endproc

.proc _mt_off
        lda     #DMA_OFF
        sta     CTRL
        rts
.endproc

; DMA may only start in a vertical blank.
.proc _mt_on
@w:     bit     MSTAT
        bpl     @w
        lda     #DMA_ON
        sta     CTRL
        rts
.endproc

.proc _mt_sync
@in:    bit     MSTAT           ; let a blank already under way end
        bmi     @in
@out:   bit     MSTAT
        bpl     @out
        inc     _mt_frames
        ; fall through
.endproc

; Rebuild every marked row.
.proc rebuild
        ldx     #0
@r:     lda     dirty,x
        beq     @next
        lda     #0
        sta     dirty,x
        stx     syncrow
        jsr     build
        ldx     syncrow
@next:  inx
        cpx     #ROWS
        bne     @r
        rts
.endproc

; Row X's list into its spare slot: one header per run of cells sharing a
; palette, then publish it with one store to the row's DLL entry.
.proc build
        stx     brow
        lda     rowlo,x
        sta     ptr1            ; the row's palette selects
        sta     tmp1            ; ...and map, low byte
        lda     rowhi,x
        sta     tmp2            ; map, high byte
        clc
        adc     #>(_mt_attr - _mt_map)
        sta     ptrl+1
        lda     slot,x
        eor     #1
        sta     slot,x
        beq     @s0
        lda     #SLOT
@s0:    clc
        adc     dllo,x
        sta     ptr2
        lda     dlhi,x
        sta     ptr2+1

        ldy     #0
        sty     runs
        sty     oidx
@run:   cpy     #COLS
        bcs     @term
        lda     (ptr1),y
        sta     rpal
        sty     rstart
        lda     runs
        cmp     #MAXRUNS - 1
        bcc     @scan
        ldy     #COLS           ; the last run allowed takes the rest
        bne     @emit
@scan:  iny
        cpy     #COLS
        bcs     @emit
        lda     (ptr1),y
        cmp     rpal
        beq     @scan
@emit:  sty     rend
        ldy     oidx
        lda     tmp1
        clc
        adc     rstart
        sta     (ptr2),y        ; +0 map, low
        iny
        lda     #HDR_WM
        sta     (ptr2),y        ; +1
        iny
        lda     tmp2
        sta     (ptr2),y        ; +2 map, high
        iny
        lda     rstart
        sec
        sbc     rend            ; -width; the field is 32 - width
        and     #$1F
        ldx     rpal
        beq     @p0
        ora     #$80            ; palette 4
@p0:    sta     (ptr2),y        ; +3
        iny
        lda     rstart
        asl     a
        asl     a
        adc     #HPOS0
        sta     (ptr2),y        ; +4 hpos
        iny
        sty     oidx
        inc     runs
        ldy     rend
        jmp     @run

@term:  ldy     oidx
        lda     #0
        sta     (ptr2),y
        iny
        sta     (ptr2),y
        ldx     brow
        ldy     dllpos,x
        lda     ptr2
        sta     dll,y
        rts
.endproc

; void __fastcall__ mt_background (unsigned char c)
.proc _mt_background
        sta     BACKGRND
        rts
.endproc

; void __fastcall__ mt_palette (unsigned char p, c1, c2, c3)
.proc _mt_palette
        sta     tmp3
        jsr     popa
        sta     tmp2
        jsr     popa
        sta     tmp1
        jsr     popa
        tax
        beq     @p0
        ldx     #P4C1 - P0C1
@p0:    lda     tmp1
        sta     P0C1,x
        lda     tmp2
        sta     P0C1+1,x
        lda     tmp3
        sta     P0C1+2,x
        rts
.endproc

; void __fastcall__ mt_clear (unsigned char blank)
.proc _mt_clear
        asl     a
        ldx     #0
@c:     sta     _mt_map,x
        sta     _mt_map+$100,x
        sta     _mt_map+$200,x
        inx
        bne     @c
        txa
@a:     sta     _mt_attr,x
        sta     _mt_attr+$100,x
        sta     _mt_attr+$200,x
        inx
        bne     @a
        lda     #1
        ldx     #ROWS - 1
@d:     sta     dirty,x
        dex
        bpl     @d
        rts
```

```
.endproc

; void __fastcall__ mt_at (unsigned char col,
  unsigned char row)
.proc _mt_at
        cmp     #ROWS
        bcc     @ok
        lda     #ROWS - 1
@ok:    sta     crow
        tax
        lda     rowlo,x
        sta     mrow
        sta     arow
        lda     rowhi,x
        sta     mrow+1
        clc
        adc     #>(_mt_attr - _mt_map)
        sta     arow+1
        jsr     popa
        sta     ccol
        rts
.endproc

; void __fastcall__ mt_put (unsigned char
  tile)
.proc _mt_put
        ldy     ccol
        cpy     #COLS
        bcs     @done
        asl     a
        sta     (mrow),y
        inc     ccol
@done:  rts
.endproc

; void __fastcall__ mt_puts (const char *s)
; Indexes the string by column: ptr1 = s -
  ccol.
.proc _mt_puts
        sec
        sbc     ccol
        sta     ptr1
        txa
        sbc     #0
        sta     ptr1+1
        ldy     ccol
        cpy     #COLS
@c:     bcs     @done
        lda     (ptr1),y
        beq     @done
        asl     a
        sta     (mrow),y
        iny
        bne     @c
@done:  sty     ccol
        rts
.endproc

; unsigned char __fastcall__ mt_text (const
  char *s, unsigned char max)
; As mt_puts, at most `max` characters, and
  anything outside 32-126 is a
; space. Returns how many it wrote.
.proc _mt_text
        sta     tmp1
        jsr     popax
        sec
        sbc     ccol
        sta     ptr1
        txa
        sbc     #0
        sta     ptr1+1
        lda     ccol
        sta     tmp3
        clc
        adc     tmp1
        bcs     @cap
        cmp     #COLS + 1
        bcc     @end
@cap:   lda     #COLS
@end:   sta     tmp2            ; the column
  to stop at
        ldy     ccol
@c:     cpy     tmp2
        bcs     @done
        lda     (ptr1),y
        beq     @done
        cmp     #' '
        bcc     @blank
        cmp     #127
        bcc     @put
@blank: lda     #' '
@put:   asl     a
        sta     (mrow),y
        iny
        bne     @c
@done:  sty     ccol
        tya
        sec
        sbc     tmp3
        ldx     #0
        rts
.endproc

; void __fastcall__ mt_fill (unsigned char
  tile, unsigned char n)
.proc _mt_fill
        sta     tmp1
        jsr     popa
        asl     a
        ldy     ccol
        ldx     tmp1
        beq     @done
@f:     cpy     #COLS
        bcs     @done
        sta     (mrow),y
        iny
        dex
        bne     @f
@done:  sty     ccol
        rts
.endproc

; void __fastcall__ mt_setpal (unsigned char
  p, unsigned char n)
.proc _mt_setpal
        tax
        jsr     popa
        sta     tmp1
        txa
        beq     @done
        ldy     ccol
@s:     cpy     #COLS
        bcs     @done
        lda     (arow),y
        cmp     tmp1
        beq     @same
        lda     tmp1
        sta     (arow),y
        lda     #1
        sty     tmp2
        ldy     crow
        sta     dirty,y
        ldy     tmp2
@same:  iny
        dex
        bne     @s
@done:  rts
.endproc

; unsigned char __fastcall__ mt_get (unsigned
  char col, unsigned char row)
.proc _mt_get
        tax
        lda     rowlo,x
        sta     ptr1
        lda     rowhi,x
        sta     ptr1+1
        jsr     popa
        tay
        lda     (ptr1),y
        lsr     a
        ldx     #0
        rts
.endproc

        .rodata
toplines:
        .byte   15, 6, 0, 0     ; NTSC: 25
  lines
        .byte   15, 15, 15, 1   ; PAL: 50
rowlo:
        .repeat ROWS, r
        .byte   <(_mt_map + r * COLS)
        .endrepeat
rowhi:
        .repeat ROWS, r
        .byte   >(_mt_map + r * COLS)
        .endrepeat
dllo:
        .repeat ROWS, r
        .byte   <(dls + r * 2 * SLOT)
        .endrepeat
dlhi:
        .repeat ROWS, r
        .byte   >(dls + r * 2 * SLOT)
        .endrepeat
dllpos:
```

```
509         .repeat ROWS, r
510         .byte   (TOP + r) * 3 + 2
511         .endrepeat
512
```

FUJINET

A FujiNet Project Publication

FN7800-PH REV. A

fujinet.online · github.com/FujiNetWIFI
© 2026 the FujiNet Project. CC BY-SA 4.0
Printed in the FujiNet Project